

UNIT-I

INTRODUCTION: History of Java, Byte code, JVM, Java buzzwords, OOP Principles, Data types, Variables, Scope and lifetime of variables, Operators, Control statements, Type conversion and casting, Arrays.

Concepts of classes and objects, Introducing methods, Method overloading, Constructors, Constructor overloading, Usage of static with data and methods, Access control, This key word, Garbage collection, String class.

The History of Java

Java was developed by **James Gosling**, Patrick Naughton, Chris Warth, Ed Frank, and Mike Sheridan at Sun Microsystems, in 1991. It took 18 months to develop the first working version.

This language was initially called “Oak” (Oak is a tree name). By default the name “Oak” is registered by some other company, so it is renamed as “Java” in 1995.

The primary motivation was the need for a **platform-independent** (that is, architecture-neutral) language that could be used to create software to be embedded in various consumer electronic devices, such as microwave ovens and remote controls.

Java’s importance to the Internet

Java is the language which is very important for internet. Java has a profound effect on the Internet. The reason for its importance is: Java expands objects that can move about freely in cyberspace. In a network, two very broad categories of objects are transmitted between the server and the personal computer: passive information and dynamic, active programs.

For example,

- Reading an e-mail, is viewing passive data. Even when the program is downloaded, the program’s code is still only passive data until it is executed.
- However, a second type of object can be transmitted is a dynamic, self-executing program. Such a program is an active agent on the client computer, yet is initiated by the server. For example, a program might be provided by the server to display properly the data that the server is sending.

Dynamic programs which are transmitted over the internet are suffering from two challenges: Security and portability.

Java Applets and Applications

Java can be used to create two types of programs:

- Applications and
- Applets.

An *application* is a program that runs on your computer, under the operating system of that computer. That is, an application created by Java is more or less like one created using C or C++. The ability to create applets makes java as an important language.

An *applet* is an application designed to be transmitted over the Internet and executed by a Java-compatible Web browser. An applet is actually a tiny Java program, dynamically downloaded across the network, just like an image, sound file, or video clip. In other words, an applet is a program that can react to user input and dynamically change.

Security:

- Security is the main problem when program is transmitted over internet. When a program is downloaded there is chance of a viral infection.
- Prior to Java, most users did not download executable programs frequently, and those who downloaded, scanned them for viruses prior to execution.
- In addition to viruses, another type of malicious program, that can gather private information, such as credit card numbers, bank account balances, and passwords, by searching the contents of your computer's local file system.
- Java answers both of these concerns. When a Java-compatible Web browser is used, one can freely download Java applets without fear of viral infection or malicious intent.
- Java achieves this protection by confining a Java program to the Java execution environment and not allowing it access to other parts of the computer.

Portability

As discussed earlier, many types of computers and operating systems are in use throughout the world—and many are connected to the Internet. For programs to be dynamically downloaded to all the various types of platforms connected to the Internet, some means of generating portable executable code is needed. Java is also a portable language.

Indeed, Java's solution to these two problems is both elegant and efficient with the use of java byte code and JVM.

Servelets:

Servlets are small programs that execute automatically at the server. Servlets dynamically extend the functionality of a Web server. Servlets extend a server's functionality by allowing the server to generate dynamic content.

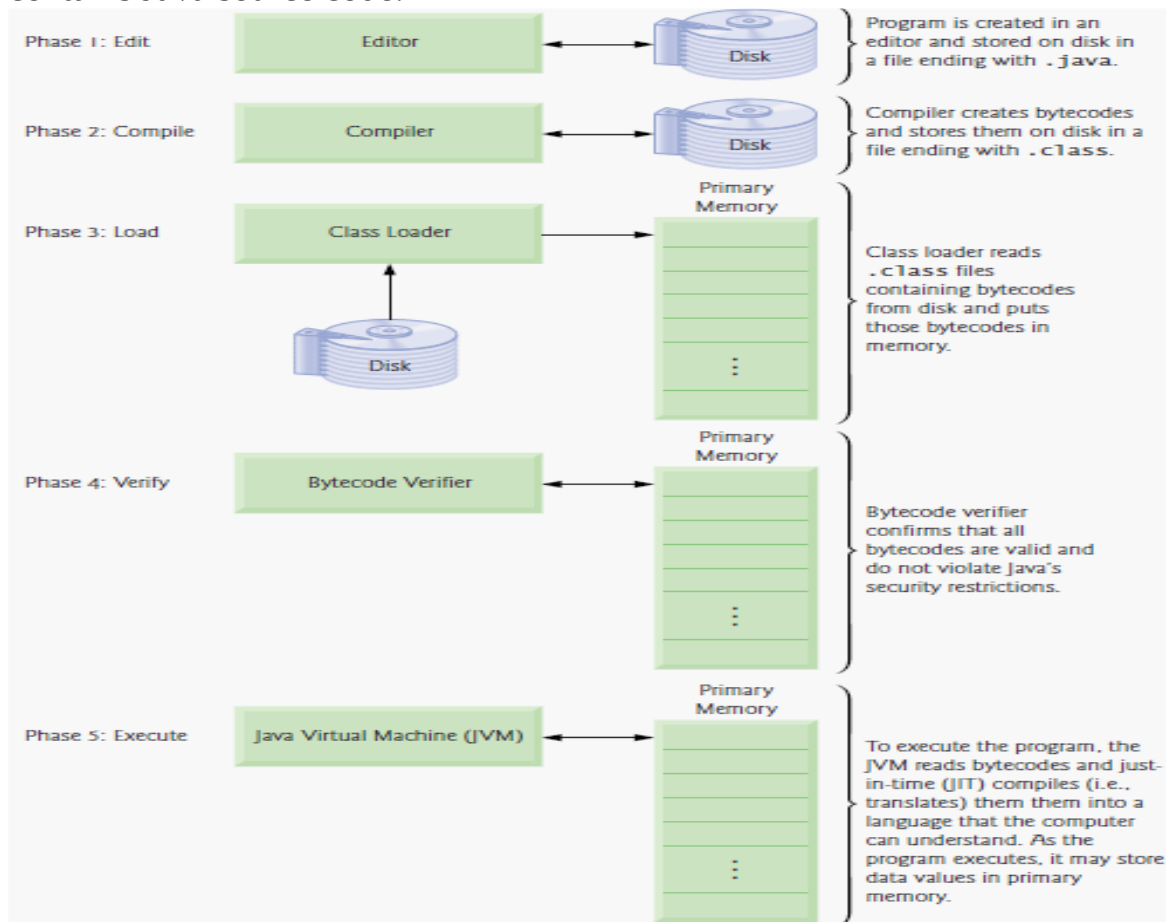
Typical Java Development Environment

Java programs normally go through five phases—

- **Edit**(create)
- **compile**,
- **load**,
- **verify** and
- **execute**.

Phase 1: Creating a Program

Phase 1 consists of editing a file with an **editor**. Type a Java program (typically referred to as **source code**) using the editor, make any necessary corrections and save the program with the **.java extension**, indicates that the file contains Java source code.



Phase 2: Compiling a Java Program into Bytecodes

In Phase 2, use the command **javac to compile** a program. For example, to compile a program called `Welcome.java`, type

➤ **javac Welcome.java**

in the command window of your system. If the program compiles, the compiler produces a **.class** file called `Welcome.class` that contains the compiled version of the program. The Java compiler translates Java source code into **bytecode**.

Phase 3: Loading a Program into Memory

In Phase 3, the program must be placed in memory to execute. This is known as **loading**. The **class loader** takes the .class files containing the program's bytecode and transfers them to main memory(RAM)..

Phase 4: Bytecode Verification

In Phase 4, as the classes are loaded, the **bytecode verifier** examines their byte-code to ensure that they are valid and do not violate Java's security restrictions. Java enforces strong security, to make sure that Java programs arriving over the network do not damage other files on the system (as computer viruses and worms might).

Phase 5: Execution

In Phase 5, the JVM executes the bytecode. The JVM is invoked by the **java** command. For example, to execute a Java application called Welcome, type the command

➔ **java Welcome**

Java Virtual Machine and byte code:

Byte code:

Byte code is nothing but the intermediate representation of Java source code which is produced by the Java compiler by compiling that source code.

This byte code is a machine independent code. It is not completely a compiled code but it is an intermediate code which is later interpreted and executed by JVM.

Byte code is a machine code for JVM. But the machine code is platform specific whereas bytecode is platform independent that is the main difference between them. It is stored in .class file which is created after compiling the source code.

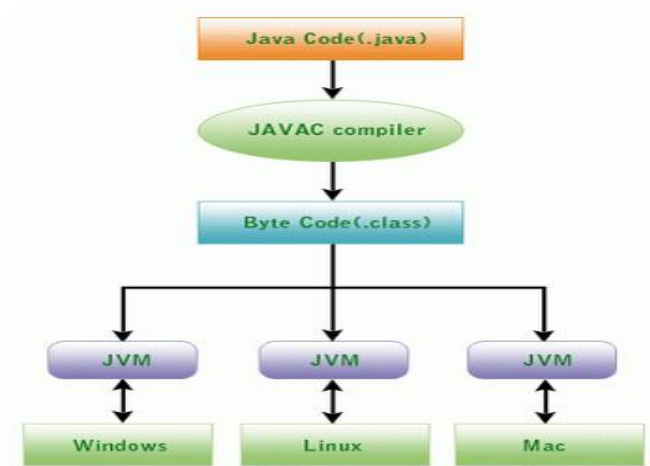
JVM :

JVM (Java Virtual Machine) is an abstract machine. It is a specification that provides runtime environment in which java bytecode can be executed.

JVMs are available for many hardware and software platforms. JVM and JRE are platform dependent because configuration of each OS differs. But, Java is platform independent.

The JVM performs following main tasks:

- Loads code
- Verifies code
- Executes code
- Provides runtime environment



Features of Java (java buzzwords)

Following are the important features of java:

1. Simple
2. Secure
3. Portable
4. Object-oriented
5. Robust
6. Multithreaded
7. Architecture-neutral
8. Interpreted
9. High performance
10. Distributed
11. Dynamic

Simple

- Java is a simple programming language. Java was designed to be easy for the programmer to learn and use effectively.
- Java is very easy to understand because Java **inherits** the C/C++ syntax and many of the object-oriented features of C++.

- Some of the more confusing concepts from C,C++ like pointers and multiple inheritance are eliminated from Java.

Object-Oriented

- Java is a pure object oriented programming language. In java “everything is an object”.
- An object is any thing that really exists can be different from others. Examples are book, student, account e.t.c.
- Each object has properties and behaviours that are implemented inside class.
- The object model in Java is simple and easy to extend, while simple types, such as integers, are kept as high-performance non objects.

Robust

- Java programs are robust (strong) because they don't crash easily like C or C++.
- Two of the main reasons for program failure are: memory management mistakes and mishandled exceptional conditions (that is, run-time errors).

Java programs are robust because of two reasons:

1. Excellent exception handling
2. Proper memory management

An Exception is a run time error such as “divide by zero”, “file not found” e.t.c. Due to exceptions data will be lost. Java helps in this area by providing object-oriented exception handling to handle all run-time errors.

Memory management can be a difficult, tedious task in traditional programming environments like C/C++. In C/C++, the programmer must manually allocate and free all dynamic memory. Java virtually eliminates memory management problems by managing memory allocation and deallocation. (In fact, deallocation is completely automatic, because Java provides garbage collection for unused objects.)

Architecture-Neutral

- A language or technology is said to be architecture neutral when it can be execute on any available processor irrespective of their architecture.
- One of the main problems facing programmers is that if a program written today may not work tomorrow on the same machine.

- Operating system upgrades, processor upgrades, and changes in core system resources can all combine to make a program malfunction. Java Virtual Machine altered this situation.
- The concept of java is, “**write once; run anywhere, any time, forever.**”

Interpreted and High Performance

Java enables the creation of cross-platform programs by compiling into an intermediate code called Java byte-code. This byte code can be interpreted on any system that provides a Java Virtual Machine.

The Java bytecode was carefully designed so that it would be easy to translate directly into native machine code for very high performance by using a **just-in-time compiler**. So java is a “High-performance cross-platform” language.

Distributed

- On internet information is distributed between various computers. Java is designed for the distributed environment of the Internet, because it handles TCP/IP protocols.
- In java, objects from two different computers allows to access methods remotely. Java supports *Remote Method Invocation (RMI)* which enables a program to invoke methods across the network.
- This feature brings an unparalleled level of abstraction to client/ server programming.

Dynamic

Java programs carry with them substantial amounts of run-time type information that is used to verify and resolve accesses to objects at run time. This makes it possible to dynamically link code in a safe manner. This is crucial to the robustness of the applet environment, in which small fragments of bytecode may be dynamically updated on a running system.

Multithreaded

- Multithreading is the ability of an application to perform more than one task at a time.
- A multi threaded program is one in which there exists multiple flows of control called threads.
- A thread represents an individual process to execute a group of statements.

- Whenever we write a java program there exists by default two threads.
 1. Foreground / child thread.
 2. Background / main(parent) thread.
- A fore ground thread is one which always executes user defined subprograms. In a java program there is a possibility of existing 'n' number of foreground threads.
- A background thread is one which always monitors the status of foreground threads. In each and every java program there exists only one background thread.
- First background thread will be created and later foreground threads will be created.

Portable:

- A portable language is one which can run on all operating systems and on all processors irrespective of their architectures.
- Portable means platform independent and architecture neutral.
- Java is a portable language, where as C/C++ are treated as non portable languages.

Secure:

- Security is the main problem when program is transmitted over internet. When a program is downloaded there is chance of a viral infection.
- Prior to Java, most users did not download executable programs frequently, and those who downloaded, scanned them for viruses prior to execution.
- In addition to viruses, another type of malicious program, that can gather private information, such as credit card numbers, bank account balances, and passwords, by searching the contents of your computer's local file system.
- Java answers both of these concerns. When a Java-compatible Web browser is used, one can freely download Java applets without fear of viral infection or malicious intent.
- Java achieves this protection by confining a Java program to the Java execution environment and not allowing it access to other parts of the computer.

First Simple Program

Entering the Program

Java is a case sensitive language. The name you give to a source file is very important. For this example, the name of the source file should be **Example.java**. The Java compiler requires that a source file use the **.java** filename extension.

```
/* simple java program */  
  
import java.lang.*;  
class Example  
{  
    public static void main(String args[])  
    {  
        System.out.println("Welcome to java");  
    }  
}
```

The name of the class defined by the program is also **Example**. In Java, all code must reside inside a class. By convention, the name of that class should match the name of the file that holds the program.

Comments

The contents of a comment are ignored by the compiler.
Java supports three styles of comments.

- Single line comment: //
- Multiline comment: /* */
- Documentation comment: /** */. This type of comment is used to produce an HTML file that documents your program. This type of comment is readable to both, computer and human.

Defining a class

This line uses the keyword **class** to declare that a new class is being defined. **Example** is an *identifier* that is the name of the class. The entire class definition, including all of its members, will be between the opening curly brace and the closing curly brace ({ , }).

public static void main(String args[])

All Java applications begin execution by calling **main()**.

The **public** keyword is an *access specifier*. When a class member is preceded by **public**, then that member may be accessed by code outside the class in which it is declared. (private and protected are other access specifiers).

The function **main()** must be declared as **public**, since it must be called by code outside of its class when the program is started.

The keyword **static** allows **main()** to be called without having to instantiate a particular instance of the class. This is necessary since **main()** is called by the Java interpreter before any objects are made.

Java compiler will compile classes that do not contain a **main()** method. But the Java interpreter has no way to run these classes. In applets you won't use **main()**.

The keyword **void** simply tells the compiler that **main()** does not return a value.

In **main()**, there is only one parameter. **String args[]** declares a parameter named **args**, which is an array of instances of the class **String**. In this case, **args** receives any command-line arguments present when the program is executed.

To display output

```
System.out.println("Welcome to Java");
```

println() displays the string which is passed to it. The line begins with **System.out**. In which **System** is a predefined class that provides access to the system, and **out** is the output stream that is connected to the console.

Note:

- Every statement in Java ends with a semicolon.
- Java is a case sensitive language.
- The built-in method **print()** is just like **println()**, except that it does not output a newline character

Compiling and executing the Program

To compile the **Example** program, execute the following command at command prompt:

```
C:\>javac Example.java
```

The **javac** compiler creates a file called **Example.class** that contains the bytecode version of the program. The Java bytecode is the intermediate representation of a program that contains instructions the Java interpreter will execute. The output of **javac** is not code that can be directly executed.

To actually run the program, use the Java interpreter, called **java**. To do so, pass the class name **Example** as a command-line argument, as shown here:

```
C:\>java Example
```

When the program is run, the following output is displayed:

Welcome to java

Data types

Java is a Strongly Typed Language.

Java defines eight primitive types of data: **byte**, **short**, **int**, **long**, **char**, **float**, **double**, and **boolean**.

These can be put in four groups:

- **Integers:** This group includes **byte**, **short**, **int**, and **long**, which are for whole valued signed numbers.
- **Floating-point numbers:** This group includes **float** and **double**, which represent numbers with fractional precision.
- **Characters:** This group includes **char**, which represents symbols in a character set, like letters and numbers.
- **Boolean:** This group includes **boolean**, which is a special type for representing true/false values.

a. Integers:

Java defines four integer types: **byte**, **short**, **int**, and **long**. All of these are signed, positive and negative values. The width and ranges of these integer types vary widely, as shown in this table:

Name	width	Range
long	64	–9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
int	32	–2,147,483,648 to 2,147,483,647
byte	8	–128 to 127
short	16	–32,768 to 32,767

byte

The smallest integer type is **byte**. This is a signed 8-bit type that has a range from –128 to 127. Byte variables are declared by use of the **byte** keyword.

Example: byte b, c;

short

short is a signed 16-bit type. It has a range from –32,768 to 32,767.

Example: short s;
 short t;

int

The most commonly used integer type is **int**. It is a signed 32-bit type that has a range from –2,147,483,648 to 2,147,483,647. Variables of type **int** are commonly employed to control loops and to index arrays. The **int** type is the most versatile and efficient type.

long:

long is a signed 64-bit type and is useful for those occasions where an **int** type is not large enough to hold the desired value. The range of a **long** is quite large.

b. Floating-Point Types

Floating-point numbers, also known as *real* numbers, are used when evaluating expressions that require fractional precision. For example: square root of a value.

There are two kinds of floating-point types, **float** and **double**, which represent single- and double-precision numbers, respectively.

Name	Width	Range
double	64	4.9e–324 to 1.8e+308
float	32	1.4e–045 to 3.4e+038

float

The type **float** specifies a *single-precision* value that uses 32 bits of storage. Variables of type **float** are useful when you need a fractional component, but don't require a large degree of precision.

Example: float hightemp, lowtemp;

double

Double precision, as denoted by the **double** keyword, uses 64 bits to store a value. All transcendental math functions, such as **sin()**, **cos()**, and **sqrt()**, return **double** values. When you need to maintain accuracy over many iterative calculations **double** is the best choice.

Example: A java program to find area of a circle

```
class Area
{
    public static void main(String args[])
    {
        double pi, r, a;
        r = 10;
        pi = 3.14;
        a = pi * r * r;
        System.out.println("Area of circle is " + a);
    }
}
```

Output : Area of circle is 314.0

c. character types

In Java, the data type used to store characters is **char**. Java uses Unicode to represent characters. In Java **char** is a 16-bit type. The range of a **char** is 0 to 65,536. There are no negative **chars**.

Example:

```
class CharDemo
{
    public static void main(String args[])
    {
        char ch1, ch2;
        ch1 = 88;           // code for X
        ch2 = 'Y';
        System.out.print("ch1 and ch2: ");
        System.out.println(ch1 + " " + ch2);
    }
}
```

Output: ch1 and ch2: X Y

d. boolean

Java has a simple type, called **boolean**, for logical values. It can have only one of two possible values, **true** or **false**. This is the type returned by all relational operators, such as **a < b**.

Example: Boolean result= true;

// Demonstrate boolean values.

```
class BoolTest
{
    public static void main(String args[])
    {
        boolean b;
        b = false;
        System.out.println("b is " + b);
        b = true;
        if(b)
            System.out.println("This is executed.");
    }
}
```

Output:

b is false
This is executed.

Variables

The variable is the basic unit of storage in a Java program. A variable is defined by the combination of an identifier, a type, and an optional initializer. In addition, all variables have a scope, which defines their visibility, and a lifetime.

Declaring a Variable

A variable must be declared before it is used.

The basic form of a variable declaration is:

type identifier [= value][, identifier [= value] ...] ;

- The *type* is one of Java's atomic types, or the name of a class or interface.
- The *identifier* is the name of the variable.
- The variable can be initialized by specifying an equal sign and a value.

- To declare more than one variable of the specified type, use a comma-separated list.

Example:

```
int a, b, c;  
int d = 3, e, f = 5; // declares three more ints, initializing  
byte z = 22;         // initializes z.  
double pi = 3.14159; // declares an approximation of pi.  
char x = 'x';         // the variable x has the value 'x'.
```

Dynamic Initialization

Java allows variables to be initialized dynamically, using any expression valid at the time the variable is declared.

A short program that computes the length of the hypotenuse of a right triangle given the lengths of its two opposing sides:

```
class DynInit  
{  
    public static void main(String args[])  
    {  
        double a = 3.0, b = 4.0;  
  
        // c is dynamically initialized  
        double c = Math.sqrt(a * a + b * b);  
  
        System.out.println("Hypotenuse is " + c);  
    }  
}
```

Here, three local variables—**a**, **b**, and **c**—are declared. The first two, **a** and **b**, are initialized by constants. However, **c** is initialized dynamically.

The Scope and Lifetime of Variables

- Java allows variables to be declared within any block. A block is begun with an opening curly brace and ended by a closing curly brace.
- A block defines a **scope**. Thus, each time you start a new block, you are creating a new scope.
- A scope determines what objects are visible to other parts of your program. It also determines the lifetime of those objects.
- In Java, the two major scopes are those defined by a class and those defined by a method.
- As a general rule, **variables declared inside a scope are not visible (that is, accessible) to code that is defined outside that scope.**
- When you declare a variable within a scope, you are localizing that variable and protecting it from unauthorized access and/or modification. Indeed, the scope rules provide the foundation for **encapsulation**.
- Scopes can be nested by adding a block within another block. Variables or objects declared in the outer scope will be visible to code within the inner scope. However, the reverse is not true. Variables or objects declared within the inner scope will not be visible outside it.

Example:

```
class Scope
{
    public static void main(String args[])
    {
        int x; // known to all code within main
        x = 10;
        if(x == 10)
        {
            // start new scope

            int y = 20; // known only to this block, x and y both known here.
            System.out.println("x and y: " + x + " " + y);

        }
        // y = 100; // Error! y not known here
        System.out.println("x is " + x); // x is still known here.
    }
}
```

Output: x and y : 10 20
X is 10

Here is another important point to remember:

variables are created when their scope is entered, and destroyed(life) when their scope is left. This means that a variable will not hold its value once it has gone out of scope.

Therefore, variables declared within a method will not hold their values between calls to that method. Also, a variable declared within a block will lose its value when the block is left. Thus, the lifetime of a variable is confined to its scope.

```
class LifeTime
{
    public static void main(String args[])
    {
        int x;
        for(x = 0; x < 3; x++)
        {
            int y = 10; // y is initialized each time block is entered
            System.out.println("y is: " + y); // this always prints -1
        }
    }
}
```

output :

```
y is : 10
y is : 10
y is : 10
```

As you can see, **y** is reinitialized to 10 each time the inner **for** loop is entered.

Type Conversion and Casting:

Java's Automatic Conversions

When one type of data is assigned to another type of variable, an *automatic type conversion* will take place if the following two conditions are met:

1. **The two types are compatible.**
2. **The destination type is larger than the source type.**

When these two conditions are met, a *widening conversion* takes place. For example, the **int** type is always large enough to hold all valid **byte** values, so no explicit cast statement is required.

For widening conversions, the numeric types, including integer and floating-point types, are compatible with each other.

However, there are no automatic conversions from the numeric types to **char** or **boolean**. Also, **char** and **boolean** are not compatible with each other.

As mentioned earlier, Java also performs an automatic type conversion when storing a literal integer constant into variables of type **byte**, **short**, **long**, or **char**.

Casting Incompatible Types

To create a conversion between two incompatible types, you must use a cast. A *cast* is simply an explicit type conversion.

syntax: **(target-type) value**

Here, *target-type* specifies the desired type to convert the specified value to. For example, the following fragment casts an **int** to a **byte**. If the integer's value is larger than the range of a **byte**, it will be reduced modulo (the remainder of an integer division by the) **byte**'s range.

```
int a;  
byte b;  
// ...  
b = (byte) a;
```

A different type of conversion will occur when a floating-point value is assigned to an integer type: *truncation*. As you know, integers do not have fractional components. Thus, when a floating-point value is assigned to an integer type, the fractional component is lost.

For example, if the value 1.23 is assigned to an integer, the resulting value will simply be 1. The 0.23 will have been truncated. Of course, if the size of the whole number component is too large to fit into the target integer type, then that value will be reduced modulo the target type's range.

The following program demonstrates some type conversions that require casts:

```
// Demonstrate casts.  
class Conversion  
{
```

```

public static void main(String args[])
{
    byte b;
    int i = 257;
    double d = 323.142;

    System.out.println("\nConversion of int to byte.");
    b = (byte) i;
    System.out.println("i and b " + i + " " + b);

    System.out.println("\nConversion of double to int.");
    i = (int) d;
    System.out.println("d and i " + d + " " + i);

    System.out.println("\nConversion of double to byte.");
    b = (byte) d;
    System.out.println("d and b " + d + " " + b);
}
}

```

output:

Conversion of int to byte.
 i and b 257 1
 Conversion of double to int.
 d and i 323.142 323
 Conversion of double to byte.
 d and b 323.142 67

Let's look at each conversion. When the value 257 is cast into a **byte** variable, the result is the remainder of the division of 257 by 256 (the range of a **byte**), which is 1 in this case.

When the **d** is converted to an **int**, its fractional component is lost. When **d** is converted to a **byte**, its fractional component is lost, *and* the value is reduced modulo 256, which in this case is 67.

Operators :

Most of its operators can be divided into the following four groups: arithmetic, bitwise, relational, and logical.

Arithmetic Operators

Arithmetic operators are used in mathematical expressions in the same way that they are used in algebra. The following table lists the arithmetic operators:

Operator	Result
+	Addition
-	Subtraction (also unary minus)
*	Multiplication
/	Division
%	Modulus
++	Increment
+=	Addition assignment
-=	Subtraction assignment
*=	Multiplication assignment
/=	Division assignment
%=	Modulus assignment
--	Decrement

The operands of the arithmetic operators must be of a numeric type. You cannot use them on **boolean** types, but you can use them on **char** types, since the **char** type in Java is a subset of **int**.

The Basic Arithmetic Operators

- The basic arithmetic operations—addition, subtraction, multiplication, and division— all behave as you would expect for all numeric types.
- The minus operator also has a unary form that negates its single operand.
- Remember that when the division operator is applied to an integer type, there will be no fractional component attached to the result.

// Demonstrate the basic arithmetic operators.

```
class BasicMath
{
    public static void main(String args[])
    {
        // arithmetic using integers
        System.out.println("Integer Arithmetic");
        int a = 20;
        int b = 10;

        System.out.println("add = " + (a+b) );
        System.out.println("sub = " + (a-b) );
    }
}
```

```

        System.out.println("mul = " + (a*b) );
        System.out.println("div = " + (a/b) );
        System.out.println("mod = " + (a%b) );

    }
}

```

When you run this program, you will see the following output:

```

Integer Arithmetic
add =30
sub=10
mul=200
div=2
mod=0

```

The Modulus Operator

The modulus operator, %, returns the remainder of a division operation. It can be applied to floating-point types as well as integer types.

// Demonstrate the % operator.

```

class Modulus
{
    public static void main(String args[])
    {
        int x = 42;
        double y = 42.25;
        System.out.println("x mod 10 = " + x % 10);
        System.out.println("y mod 10 = " + y % 10);
    }
}

```

output:

```

    x mod 10 = 2
    y mod 10 = 2.25

```

Arithmetic Compound Assignment Operators

Java provides special operators that can be used to combine an arithmetic operation with an assignment are called compound assignment operators.

```

    a = a + 4;

```

In Java, you can rewrite this statement as shown here:

```
a += 4;
```

There are compound assignment operators for all of the arithmetic, binary operators.

Thus, any statement of the form

```
var = var op expression;
```

can be rewritten as

```
var op= expression;
```

The compound assignment operators provide two benefits. First, they save you a bit of typing, because they are “shorthand” for their equivalent long forms. Second, they are implemented more efficiently by the Java run-time system than are their equivalent long forms.

```
class OpEquals
{
    public static void main(String args[])
    {
        int a = 1;
        int b = 2;
        int c = 3;
        a += 5;
        b *= 4;
        c += a * b;
        c %= 6;
        System.out.println("a = " + a);
        System.out.println("b = " + b);
        System.out.println("c = " + c);
    }
}
```

output:

a = 6

b = 8

c = 3

Increment and Decrement

The ++ and the -- are Java’s increment and decrement operators.

The increment operator increases its operand by one. The decrement operator decreases its operand by one.

For example, this statement:

`x = x + 1;`

can be rewritten like this by use of the increment operator: `x++;`

Similarly, this statement:

`x = x - 1;` is equivalent to `x--;`

These operators are unique in that they can appear both in *postfix* form, where they follow the operand as just shown, and *prefix* form, where they precede the operand.

Difference between these two forms:

In the prefix form, the operand is incremented or decremented before the value is obtained for use in the expression.

In postfix form, the previous value is obtained for use in the expression, and then the operand is modified.

```
class IncDec
{
    public static void main(String args[])
    {
        int a = 1;
        int b = 2;
        int c;
        int d;
        c = ++b;
        d = a++;
        c++;
        System.out.println("a = " + a);
        System.out.println("b = " + b);
        System.out.println("c = " + c);
        System.out.println("d = " + d);
    }
}
```

The output of this program follows:

a = 2

b = 3

c = 4

d = 1

The Bitwise Operators

Java defines several *bitwise operators* that can be applied to the integer types, **long**, **int**, **short**, **char**, and **byte**. These operators act upon the individual bits of their operands.

Operator	Result
~	Bitwise unary NOT
&	Bitwise AND
	Bitwise OR
^	Bitwise exclusive OR
>>	Shift right
>>>	Shift right zero fill
<<	Shift left

The Bitwise Logical Operators

The bitwise logical operators are **&**, **|**, **^**, and **~**. The following table shows the outcome of each operation.

A	B	A B	A & B	A ^ B	~A
0	0	0	0	0	1
1	0	1	0	1	0
0	1	1	0	1	1
1	1	1	1	0	0

The Bitwise NOT

Also called the *bitwise complement*, the unary NOT operator, **~**, inverts all of the bits of its operand. For example, the number 42, which has the following bit pattern:

00101010
 becomes
 11010101 after the NOT operator is applied.

The Bitwise AND

The AND operator, **&**, produces a 1 bit if both operands are also 1. A zero is produced in all other cases. Here is an example

```

    00101010  42
&  00001111  15
-----
    00001010  10
  
```


The Bitwise OR

The OR operator, `|`, combines bits such that if either of the bits in the operands is a 1, then the resultant bit is a 1, as shown here:

	00101010	42
	00001111	15
	00101111	47

The Bitwise XOR

The XOR operator, `^`, combines bits such that if exactly one operand is 1, then the result is 1. Otherwise, the result is zero.

	00101010	42
^	00001111	15
	00100101	37

The Left Shift

The left shift operator, `<<`, shifts all of the bits in a value to the left a specified number of times.

It has this general form:

value `<<` *num*

Here, *num* specifies the number of positions to left-shift the value in *value*. That is, the `<<` moves all of the bits in the specified value to the left by the number of bit positions specified by *num*.

For each shift left, the high-order bit is shifted out (and lost), and a zero is brought in on the right. This means that when a left shift is applied to an **int** operand, bits are lost once they are shifted past bit position 31. If the operand is a **long**, then bits are lost after bit position 63.

Since each left shift has the effect of doubling the original value, programmers frequently use this fact as an efficient alternative to multiplying by 2.

// Left shifting as a quick way to multiply by 2.

```
class MultByTwo
{
    public static void main(String args[])
    {
        int a=8;
        a+=a<<2;
    }
}
```

```

        System.out.println("a = " + a);
    }
}

```

The program generates the following output:

A = 16

The Right Shift

The right shift operator, `>>`, shifts all of the bits in a value to the right a specified number of times. Its general form is shown here:

value `>>` *num*

Here, *num* specifies the number of positions to right-shift the value in *value*. That is, the `>>` moves all of the bits in the specified value to the right the number of bit positions specified by *num*.

When a value has bits that are “shifted off,” those bits are lost. For example, the next code fragment shifts the value 35 to the right two positions, which causes the two low-order bits to be lost, resulting again in *a* being set to 8.

```

int a = 35;
a = a >> 2; // a still contains 8

```

Looking at the same operation in binary shows more clearly how this happens:

```

00100011    35
>> 2
00001000    8

```

Each time you shift a value to the right, it divides that value by two—and discards any remainder. the previous contents of the top bit. This is called *sign extension* and serves to preserve the sign of negative numbers when you shift them right. For example, `-8 >> 1` is `-4`, which, in binary, is

```

11111000    -8
>> 1
11111100    -4

```

It is interesting to note that if you shift `-1` right, the result always remains `-1`, since sign extension keeps bringing in more ones in the high-order bits.

The Unsigned Right Shift

As you have just seen, the `>>` operator automatically fills the high-order bit with its previous contents each time a shift occurs. This preserves the sign of the value. However, sometimes this is undesirable.

For example, if you are shifting something that does not represent a numeric value, you may not want sign extension to take place. This situation is common when you are working with pixel-based values and graphics. In these cases, you will generally want to shift a zero into the high-order bit no matter what its initial value was. This is known as an *unsigned shift*.

To accomplish this, you will use Java's unsigned, shift-right operator, `>>>`, which always shifts zeros into the high-order bit.

The following code fragment demonstrates the `>>>`. Here, **a** is set to `-1`, which sets all 32 bits to 1 in binary. This value is then shifted right 24 bits, filling the top 24 bits with zeros, ignoring normal sign extension. This sets **a** to 255.

```
int a = -1;
a = a >>> 24;
```

Here is the same operation in binary form to further illustrate what is happening:

```
11111111 11111111 11111111 11111111 -1 in binary as an int
>>>24
00000000 00000000 00000000 11111111 255 in binary as an int
```

Relational Operators

The *relational operators* determine the relationship that one operand has to the other.

Operator	Result
<code>==</code>	Equal to
<code>!=</code>	Not equal to
<code>></code>	Greater than
<code><</code>	Less than
<code>>=</code>	Greater than or equal to
<code><=</code>	Less than or equal to

The outcome of these operations is a **boolean** value. The relational operators are most frequently used in the expressions that control the **if** statement and the various loop statements.

Boolean Logical Operators

The Boolean logical operators shown here operate only on **boolean** operands. All of the binary logical operators combine two **boolean** values to form a resultant **boolean** value.

Operator	Result
&	Logical AND
	Logical OR
^	Logical XOR (exclusive OR)
	Short-circuit OR
&&	Short-circuit AND
!	Logical unary NOT
&=	AND assignment
=	OR assignment
^=	XOR assignment
==	Equal to
!=	Not equal to
?:	Ternary if-then-else

The following table shows the effect of each logical operation:

A	B	A B	A & B	A ^ B	!A
False	False	False	False	False	True
True	False	True	False	True	False
False	True	True	False	True	True
True	True	True	True	False	False

// Demonstrate the boolean logical operators.

class BoolLogic

{

 public static void main(String args[])

 {

 boolean a = true;

 boolean b = false;

 boolean c = a | b;

 boolean d = a & b;

 boolean e = a ^ b;

 boolean f = (!a & b) | (a & !b);

 boolean g = !a;

 System.out.println(" a = " + a);

 System.out.println(" b = " + b);

 System.out.println(" a|b = " + c);

 System.out.println(" a&b = " + d);

 System.out.println(" a^b = " + e);

 System.out.println("!a&b|a&!b = " + f);

 System.out.println(" !a = " + g);

 }

}

Output:

a = true

b = false
a|b = true
a&b = false
a^b = true
a&b|a&!b = true
!a = false

Short-Circuit Logical Operators

Java provides two interesting Boolean operators not found in many other computer languages. These are secondary versions of the Boolean AND and OR operators, and are known as *short-circuit* logical operators.

If you use the `||` and `&&` forms, Java will not bother to evaluate the right-hand operand when the outcome of the expression can be determined by the left operand alone. This is very useful when the right-hand operand depends on the value of the left one in order to function properly.

For example, the following code fragment shows how you can take advantage of short-circuit logical evaluation to be sure that a division operation will be valid before evaluating it:

```
if (denom != 0 && num / denom > 10)
```

Since the short-circuit form of AND (`&&`) is used, there is no risk of causing a run-time exception when **denom** is zero. If this line of code were written using the single `&` version of AND, both sides would be evaluated, causing a run-time exception when **denom** is zero.

The Assignment Operator

The assignment operator is used to assign a value to variable. The *assignment operator* is the single equal sign, `=`.

It has this general form:

var = expression;

Here, the type of *var* must be compatible with the type of *expression*.

It allows you to create a chain of assignments. For example

```
int x, y, z;  
x = y = z = 100; // set x, y, and z to 100
```

Using a “chain of assignment” is an easy way to set a group of variables to a common value.

The ? Operator

Java includes a special *ternary* (three-way) *operator* that can replace certain types of if-then-else statements. This operator is the **?**.

The **?** has this general form:

$$expression1 \text{ ? } expression2 : expression3$$

Here, *expression1* can be any expression that evaluates to a **boolean** value. If *expression1* is **true**, then *expression2* is evaluated; otherwise, *expression3* is evaluated.

```
class Ternary
{
    public static void main(String args[])
    {
        int a=10,b=20;
        int k;
        k = a > b ? a : b;    // find largest among two numbers

        System.out.println(" Largest value = " + k);
    }
}
```

Output:

Largest value = 10

Operator Precedence

Table 4-1 shows the order of precedence for Java operators, from highest to lowest.

Highest			
()	[]	.	
++	--	~	!
*	/	%	
+	-		
>>	>>>	<<	
>	>=	<	<=
==	!=		
&			
^			
&&			
?:			
=	op=		
Lowest			

Control Statements:

Control statements can be put into the following categories: **selection**, **iteration**, and **jump**.

- **Selection** statements allow your program to choose different paths of execution based upon the outcome of an expression or the state of a variable.
- **Iteration** statements enable program execution to repeat one or more statements (that is, iteration statements form loops).
- **Jump** statements allow your program to execute in a nonlinear fashion.

Java's Selection Statements(conditional statements)

Java supports two selection statements: **if** and **switch**. These statements allow you to control the flow of your program's execution based upon conditions known only during run time.

If:

The **if** statement is Java's conditional branch statement. It can be used to route program execution through two different paths.

Here is the general form of the **if** statement:

```
if (condition) statement1;
else statement2;
```

Here, each *statement* may be a single statement or a compound statement enclosed in curly braces (that is, a *block*). The *condition* is any expression that returns a **boolean** value. The **else** clause is optional.

The **if** works like this: If the *condition* is true, then *statement1* is executed. Otherwise, *statement2* (if it exists) is executed. In no case will both statements be executed. For example, consider the following:

```
int a, b;  
// ...  
if(a < b) a = 0;  
else b = 0;
```

Here, if **a** is less than **b**, then **a** is set to zero. Otherwise, **b** is set to zero. In no case are they both set to zero. Most often, the expression used to control the **if** will involve the relational operators.

However, this is not technically necessary. It is possible to control the **if** using a single **boolean** variable.

```
class BoolIf  
{  
    public static void main(String args[])  
    {  
        boolean b;  
        b = true;  
        if(b)  
            System.out.println("This is executed.");  
    }  
}
```

Output:

This is executed.

Nested ifs

- A *nested if* is an **if** statement that is the target of another **if** or **else**.
- When you nest **ifs**, the main thing to remember is that an **else** statement always refers to the nearest **if** statement that is within the same block as the **else** and that is not already associated with an **else**.
- Here is an example:

```
if(i == 10)
```



```

{
    if(j < 20) a = b;
    if(k > 100) c = d; // this if is
    else a = c; // associated with this else
}
else a = d; // this else refers to if(i == 10)

```

As the comments indicate, the final **else** is not associated with **if(j<20)** because it is not in the same block (even though it is the nearest **if** without an **else**). Rather, the final **else** is associated with **if(i==10)**. The inner **else** refers to **if(k>100)** because it is the closest **if** within the same block.

The if-else-if Ladder

A common programming construct that is based upon a sequence of nested **ifs** is the *if-else-if ladder*. It looks like this:

```

    if(condition)
        statement;
    else if(condition)
        statement;
    else if(condition)
        statement;
    ...
    else
        statement;

```

The **if** statements are executed from the top down. As soon as one of the conditions controlling the **if** is **true**, the statement associated with that **if** is executed, and the rest of the ladder is bypassed.

If none of the conditions is true, then the final **else** statement will be executed. The final **else** acts as a default condition; that is, if all other conditional tests fail, then the last **else** statement is performed.

If there is no final **else** and all other conditions are **false**, then no action will take place.

```

class Test
{
    public static void main(String args[])
    {
        int x = 30;
    }
}

```

```

        if( x == 0 )
        {
            System.out.println(" X is Zero");
        }
        else if( x >0 )
        {
            System.out.println("X is Positive");
        }
        else if( x < 0 )
        {
            System.out.println("X is Negative");
        }
    }
}

```

// Demonstrate if-else-if statements.

```

class IfElse
{
    public static void main(String args[])
    {
        int month = 4;           // April
        String season;
        if(month == 12 || month == 1 || month == 2)
            season = "Winter";
        else if(month == 3 || month == 4 || month == 5)
            season = "Spring";
        else if(month == 6 || month == 7 || month == 8)
            season = "Summer";
        else if(month == 9 || month == 10 || month == 11)
            season = "Autumn";
        else
            season = "Bogus Month";
        System.out.println("April is in the " + season + ".");
    }
}

```

Here is the output produced by the program:

April is in the Spring.

switch

The **switch** statement is Java's multiway branch statement. It often provides a better alternative than a large series of **if-else-if** statements.

Here is the general form of a **switch** statement:

```
switch (expression)
{
    case value1:
        // statement sequence
        break;
    case value2:
        // statement sequence
        break;
    ...
    case valueN:
        // statement sequence
        break;
    default:
        // default statement sequence
}
```

The *expression* must be of type **byte**, **short**, **int**, or **char**; each of the *values* specified in the **case** statements must be of a type compatible with the expression.

Each **case** value must be a unique literal (that is, it must be a constant, not a variable). Duplicate **case** values are not allowed.

The **switch** statement works like this:

The value of the expression is compared with each of the literal values in the **case** statements. If a match is found, the code sequence following that **case** statement is executed. If none of the constants matches the value of the expression, then the **default** statement is executed. However, the **default** statement is optional. If no **case** matches and no **default** is present, then no further action is taken.

The **break** statement is used inside the **switch** to terminate a statement sequence. When a **break** statement is encountered, execution branches to the first line of code that follows the entire **switch** statement. This has the effect of “jumping out” of the **switch**.

// A simple example of the switch.
class SampleSwitch

```

{
    public static void main(String args[])
    {
        int i=2;
        switch(i)
        {
            case 0:      System.out.println("i is zero."); break;
            case 1:      System.out.println("i is one."); break;
            case 2:      System.out.println("i is two."); break;
            case 3:      System.out.println("i is three."); break;
            default:     System.out.println("i is greater than 3.");
        }
    }
}

```

Output :

i is two.

// An improved version of the season program.

```

class Switch
{
    public static void main(String args[])
    {
        int month = 4;
        String season;
        switch (month)
        {
            case 12:
            case 1:
            case 2:      season = "Winter";
                        break;
            case 3:
            case 4:
            case 5:      season = "Spring";
                        break;
            case 6:
            case 7:
            case 8:      season = "Summer";
                        break;
            case 9:

```

```

        case 10:
        case 11: season = "Autumn";
                break;
        default: season = "Bogus Month";
    }
    System.out.println("April is in the " + season + ".");
}
}

```

Nested switch Statements

You can use a **switch** as part of the statement sequence of an outer **switch**. This is called a *nested switch*. Since a **switch** statement defines its own block, no conflicts arise between the **case** constants in the inner **switch** and those in the outer **switch**.

```

switch(count)
{
    case 1:
        switch(target)
        {           // nested switch
            case 0:
                System.out.println("target is zero");
                break;
            case 1: // no conflicts with outer switch
                System.out.println("target is one");
                break;
        }
        break;
    case 2: // ...

```

Here, the **case 1:** statement in the inner switch does not conflict with the **case 1:** statement in the outer switch. The **count** variable is only compared with the list of cases at the outer level. If **count** is 1, then **target** is compared with the inner list cases.

In summary, there are three important features of the **switch** statement to note:

- The **switch** differs from the **if**, that is **switch** can only test for equality, whereas **if** can evaluate any type of Boolean expression. That is, the **switch** looks only for a match between the value of the expression and one of its **case** constants.

- No two **case** constants in the same **switch** can have identical values. Of course, a **switch** statement and an enclosing outer **switch** can have **case** constants in common.
- A **switch** statement is usually more efficient than a set of nested **ifs**.

```
public class NestedSwitch
{
    public static void main(String[] args)
    {
        int i = 0;
        int j = 1;
        switch(i)
        {
            case 0:
                switch(j)
                {
                    case 0:    System.out.println("i is 0, j is 0");
                    case 1:    System.out.println("i is 0, j is 1");
                    default    :    System.out.println("nested default
case!!");
                }
                break;
            default:    System.out.println("No matching case found!!");
        }
    }
}
```

Output:
i is 0, j is 1

Iteration Statements

Java's iteration statements are **for**, **while**, and **do-while**. These statements create *loops*. A loop repeatedly executes the same set of instructions until a termination condition is met.

while

The **while** loop is Java's most fundamental loop statement. It repeats a statement or block while its controlling expression is true.

Here is its general form:

```
while(condition)
{
    // body of loop
}
```

The *condition* can be any Boolean expression. The body of the loop will be executed as long as the conditional expression is true. When *condition* becomes false, control passes to the next line of code immediately following the loop. The curly braces are unnecessary if only a single statement is being repeated.

```
class WhileDemo
{
    public static void main(String args[])
    {
        int i = 1;
        while(i<=5)
        {
            System.out.println("Value = " + i);
            i++;
        }
    }
}
```

Output:

```
Value=1
Value=2
Value=3
Value=4
Value=5
```

do-while

Since the **while** loop evaluates its conditional expression at the top of the loop, the body of the loop will not execute even once if the condition is initially false (the body of the loop will not be executed at all).

However, sometimes it is desirable to execute the body of a loop at least once, even if the conditional expression is false initially. In other words, there are times when you would like to test the termination expression at the end of the loop rather than at the beginning. To serve this purpose java supplies **do-while** loop.

The **do-while** loop always executes its body at least once, because its conditional expression is at the bottom of the loop.

Its general form is:

```
do
{
    // body of loop
} while (condition);
```

Each iteration of the **do-while** loop first executes the body of the loop and then evaluates the conditional expression. If this expression is true, the loop will repeat. Otherwise, the loop terminates.

```
class DoWhile
{
    public static void main(String args[])
    {
        int i = 1;
        do {
            System.out.println("Value = " + i);
            i++;
        } while(i <= 5);
    }
}
```

The **do-while** loop is especially useful when you process a menu selection, because you will usually want the body of a menu loop to execute at least once.

```
Import java.util.scanner;
public class ConsoleMenuDemo
{
    public static void main(String[] args)
    {
        // Local variable
        int choice,a,b;
        Scanner s=new Scanner(System.in);
```



```

do{
    System.out.println("  MENU SELECTION DEMO  ");
    System.out.println("=====");
    System.out.println(" 1.addition ");
    System.out.println(" 2.subtraction ");
    System.out.println(" 3.multiplication");
    System.out.println("=====");
    choice = s.nextInt();
}while(choice<1 || choice >3);

switch (choice)
{
    case 1: System.out.println(" enter value of a and b");
        a = s.nextInt();
        b = s.nextInt();
        System.out.println("a+b =" +(a+b));
        break;

    case 2: System.out.println(" enter value of a and b");
        a = s.nextInt();
        b = s.nextInt();
        System.out.println("a-b =" +(a-b));
        break;

    case 3: System.out.println(" enter value of a and b");
        a = s.nextInt();
        b = s.nextInt();
        System.out.println("a*b =" +(a*b));
        break;

    default:
        System.out.println("Invalid selection");
        break; // This break is not really necessary
}
}
}

```

// Notice that characters are read from the keyboard by calling **System.in.read()**. This is one of Java's console input functions.

for:

There are two forms of the **for** loop. The first is the traditional form that has been in use since the original version of Java. The second is the new “for-each” form.

the general form of the traditional **for** statement:

```
for(initialization; condition; iteration)
{
    // body
}
```

If only one statement is being repeated, there is no need for the curly braces.

The **for** loop operates as follows.

When the loop first starts, the *initialization* portion of the loop is executed. Generally, this is an expression that sets the value of the *loop control variable*, which acts as a counter that controls the loop. It is important to understand that the initialization expression is only executed once.

Next, *condition* is evaluated. This must be a Boolean expression. It usually tests the loop control variable against a target value. If this expression is true, then the body of the loop is executed. If it is false, the loop terminates.

Next, the *iteration* portion of the loop is executed. This is usually an expression that increments or decrements the loop control variable.

The loop then iterates, first evaluating the conditional expression, then executing the body of the loop, and then executing the iteration expression with each pass. This process repeats until the controlling expression is false.

```
class ForTick
{
    public static void main(String args[])
    {
        int n;
        for(n=5; n>0; n--)
            System.out.println("tick = " + n);
    }
}
```

Output:

tick = 5

tick = 4

```
tick = 3
tick = 2
tick = 1
```

Declaring Loop Control Variables Inside the for Loop

Some times, the variable that controls a **for** loop is only needed for the purposes of the loop and is not used elsewhere. When this is the case, it is possible to declare the variable inside the initialization portion of the **for**.

```
// Declare a loop control variable inside the for.
class ForTick
{
    public static void main(String args[])
    {
        // here, n is declared inside of the for loop
        for(int n=5; n>0; n--)
            System.out.println("tick = " + n);
    }
}
```

Output:

```
tick = 5
tick = 4
tick = 3
tick = 2
tick = 1
```

When you declare a variable inside a **for** loop, there is one important point to remember:

The scope of that variable ends when the **for** statement does. (That is, the scope of the variable is limited to the **for** loop.) Outside the **for** loop, the variable will cease to exist. If you need to use the loop control variable elsewhere in your program, you will not be able to declare it inside the **for** loop.

The For-Each Version of the for Loop:

A foreach style loop is designed to cycle through a collection of objects, such as an array, in strictly sequential fashion, from start to finish. The for-each style of **for** is also referred to as the *enhanced* **for** loop.

The general form of the for-each version of the **for** is shown here:

for(type itr-var : collection) statement-block

Here, *type* specifies the type and *itr-var* specifies the name of an *iteration variable* that will receive the elements from a collection, one at a time, from beginning to end. There are various types of collections that can be used with the **for**.

Example:

```
// Use a for-each style for loop.
class ForEach
{
    public static void main(String args[])
    {
        int nums[] = { 1, 2, 3, 4, 5 };
        int sum = 0;
        // use for-each style for to display and sum the values
        for(int x : nums)
        {
            System.out.println("Value is: " + x);
            sum += x;
        }
        System.out.println("Summation: " + sum);
    }
}
```

The output from the program is shown here.

```
Value is: 1
Value is: 2
Value is: 3
Value is: 4
Value is: 5
Summation: 55
```

As this output shows, the for-each style for **automatically cycles through an array in sequence from the lowest index to the highest**.

With each pass through the loop, **x** is automatically given a value equal to the next element in **nums**. Thus, on the first iteration, **x** contains 1; on the second iteration, **x** contains 2; and so on.

Not only is the syntax streamlined, but it also **prevents boundary errors**.

Although the for-each **for** loop iterates until all elements in an array have been examined, it is possible to terminate the loop early by using a **break** statement.

For example, this program sums only the first five elements of **nums**:

// Use break with a for-each style for.

```
class ForEach2
{
    public static void main(String args[])
    {
        int sum = 0;
        int nums[] = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
        // use for to display and sum the values
        for(int x : nums)
        {
            System.out.println("Value is: " + x);
            sum += x;
            if(x == 5) break; // stop the loop when 5 is obtained
        }
        System.out.println("Summation of first 5 elements: " + sum);
    }
}
```

This is the output produced:

Value is: 1

Value is: 2

Value is: 3

Value is: 4

Value is: 5

Summation of first 5 elements: 15

There is one important point to understand about the for-each style loop.

Its iteration variable is “read-only” as it relates to the underlying array. An assignment to the iteration variable has no effect on the underlying array. In other words, you can’t change the contents of the array by assigning the iteration variable a new value.

Example:

// The for-each loop is essentially read-only.

```
class NoChange
{
    public static void main(String args[])
    {
        int nums[] = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
```

```

        for(int x : nums)
        {
            System.out.print(x + " ");
            x = x * 10; // no effect on nums
        }
        System.out.println();
        for(int x : nums)
            System.out.print(x + " ");
        System.out.println();
    }
}

```

The first **for** loop increases the value of the iteration variable by a factor of 10. However, this assignment has no effect on the underlying array **nums**, as the second **for** loop illustrates.

The output, shown here, proves this point:

```

1 2 3 4 5 6 7 8 9 10
1 2 3 4 5 6 7 8 9 10

```

Iterating Over Multidimensional Arrays

The enhanced version of the **for** also works on multidimensional arrays. Remember, multidimensional arrays consist of *arrays of arrays*. (For example, a two-dimensional array is an array of one-dimensional arrays.)

This is important when iterating over a multidimensional array, because each iteration obtains the *next array*, not an individual element. Furthermore, the iteration variable in the **for** loop must be compatible with the type of array being obtained.

For example, in the case of a two-dimensional array, the iteration variable must be a reference to a one-dimensional array. In general, when using the for-each **for** to iterate over an array of N dimensions, the objects obtained will be arrays of $N-1$ dimensions. To understand the implications of this, consider the following program. It uses nested **for** loops to obtain the elements of a two-dimensional array in roworder, from first to last.

```

// Use for-each style for on a two-dimensional array.
class ForEach3
{
    public static void main(String args[])
    {
        int sum = 0,k=0;
    }
}

```

```

int nums[][] = new int[3][5];
// give nums some values
for(int i = 0; i < 2; i++)
    for(int j=0; j < 3; j++)
        nums[i][j] = k++;
    // use for-each for to display and sum the values
for(int x[] : nums)
{
    for(int y : x)
    {
        System.out.println("Value is: " + y);
        sum += y;
    }
}
System.out.println("Summation: " + sum);
}
}

```

The output from this program is shown here:

```

Value is: 0
Value is: 1
Value is: 2
Value is: 3
Value is: 4
Value is: 5
Summation: 15

```

In the program, pay special attention to this line: **for(int x[] : nums)**

Notice how **x** is declared. It is a reference to a one-dimensional array of integers. This is necessary because each iteration of the **for** obtains the next *array* in **nums**, beginning with the array specified by **nums[0]**. The inner **for** loop then cycles through each of these arrays, displaying the values of each element.

// Search an array using for-each style for.

```

class Search
{
    public static void main(String args[])
    {
        int nums[] = { 6, 8, 3, 7, 5, 6, 1, 4 };
        int val = 5;
        boolean found = false;
        // use for-each style for to search nums for val
    }
}

```

```

        for(int x : nums)
        {
            if(x == val)
            {
                found = true;
                break;
            }
        }
        if(found)
            System.out.println("Value found!");
    }
}

```

Jump Statements

Java supports three jump statements: **break**, **continue**, and **return**. These statements transfer control to another part of your program.

break

In Java, the **break** statement has three uses.

1. It terminates a statement sequence in a **switch** statement.
2. It can be used to exit a loop.
3. **break** can be used as a “civilized” form of goto. The last two uses are explained here.

Using break to Exit a Loop:

- By using **break**, you can force immediate termination of a loop, bypassing the conditional expression and any remaining code in the body of the loop.
- When a **break** statement is encountered inside a loop, the loop is terminated and program control resumes at the next statement following the loop.

// Using break to exit a loop.

```

class BreakLoop
{
    public static void main(String args[])
    {
        for(int i=0; i<100; i++)
        {

```



```

        if(i == 5) break; // terminate loop if i is 10
        System.out.println("i = " + i);
    }
    System.out.println("Loop complete.");
}

```

This program generates the following output:

```

i = 0
i = 1
i = 2
i = 3
i = 4
Loop complete.

```

Using *break* as a Form of *goto*:

Java does not have a *goto* statement, but there are a few places where the *goto* is a valuable and legitimate construct for flow control. For example, the *goto* can be useful when you are exiting from a deeply nested set of loops.

To handle such situations, Java defines an expanded form of the **break** statement. By using this form of **break**, you can break out of one or more blocks of code. These blocks need not be part of a loop or a **switch**.

They can be any block. Further, you can specify precisely where execution will resume, because this form of **break** works with a label. As you will see, **break** gives you the benefits of a *goto* without its problems.

The general form of the labeled **break** statement is shown here:

break label;

Most often, *label* is the name of a label that identifies a block of code. This can be a stand-alone block of code but it can also be a block that is the target of another statement.

When this form of **break** executes, control is transferred out of the named block. The labeled block must enclose the **break** statement, but it does not need to be the immediately enclosing block.

To name a block, put a label at the start of it. A *label* is any valid Java identifier followed by a colon. Once you have labeled a block, you can then use this label as

the target of a **break** statement. Doing so causes execution to resume at the *end* of the labeled block.

For example, the following program shows three nested blocks, each with its own label. The **break** statement causes execution to jump forward, past the end of the block labeled **second**, skipping the two **println()** statements.

```
// Using break as a civilized form of goto.
class Break
{
    public static void main(String args[])
    {
        boolean t = true;
        first:
        {
            second:
            {
                third:
                {
                    System.out.println("Before the break.");
                    if(t)
                        break second; // break out of second block
                    System.out.println("This won't execute");
                }
                System.out.println("This won't execute");
            }
            System.out.println("This is after second block.");
        }
    }
}
```

Output:

Before the break.

This is after second block.

One of the most common uses for a labelled **break** statement is to exit from nested loops.

For example, in the following program, the outer loop executes only once:

```
// Using break to exit from nested loops
class BreakLoop4
```

```

{
    public static void main(String args[])
    {
        outer: for(int i=0; i<3; i++)
        {
            System.out.print("Pass " + i + ": ");
            for(int j=0; j<100; j++)
            {
                if(j == 10) break outer; // exit both loops
                System.out.print(j + " ");
            }
            System.out.println("This will not print");
        }
        System.out.println("Loops complete.");
    }
}

```

This program generates the following output:

Pass 0: 0 1 2 3 4 5 6 7 8 9 Loops complete.

Continue:

Sometimes it is useful to force an early iteration of a loop. That is, you might want to continue running the loop but stop processing the remainder of the code in its body for this particular iteration. The **continue** statement performs such an action.

In **while** and **do-while** loops, a **continue** statement causes control to be transferred directly to the conditional expression that controls the loop.

In a **for** loop, control goes first to the iteration portion of the **for** statement and then to the conditional expression. For all three loops, any intermediate code is bypassed.

```

class Continue
{
    public static void main(String args[])
    {
        for(int i=0; i<10; i++)
        {
            if (i%2 == 0) continue;
            System.out.println(i + " ");
        }
    }
}

```

```
    }  
}
```

This code uses the `%` operator to check if `i` is even. If it is, the loop continues without printing

a newline. Here is the output from this program:

```
1  
3  
5  
7  
9
```

As with the **break** statement, **continue** may specify a label to describe which enclosing loop to continue.

Here is an example program that uses **continue** to print a triangular multiplication table for 0 through 9.

// Using continue with a label.

```
class ContinueLabel  
{  
    public static void main(String args[])  
    {  
        outer: for (int i=0; i<=5; i++)  
        {  
            for(int j=0; j<=5; j++)  
            {  
                if(j == i)  
                {  
                    System.out.println();  
                    continue outer;  
                }  
                System.out.print(" " + j);  
            }  
            System.out.println();  
        }  
    }  
}
```

Output:

```
0  
0 1  
0 1 2
```

0 1 2 3
0 1 2 3 4

return:

The **return** statement is used to explicitly return from a method. That is, it causes program control to transfer back to the caller of the method.

At any time in a method the **return** statement can be used to cause execution to branch back to the caller of the method. Thus, the **return** statement immediately terminates the method in which it is executed.

The following example illustrates this point. Here, **return** causes execution to return to the Java run-time system, since it is the run-time system that calls **main()**.

// Demonstrate return.

```
class Return
{
    public static void main(String args[])
    {
        boolean t = true;

        System.out.println("Before the return.");
        if(t) return;           // return to caller
        System.out.println("This won't execute.");
    }
}
```

output:

Before the return.

As you can see, the final **println()** statement is not executed. As soon as **return** is executed, control passes back to the caller.

One last point: In the preceding program, the **if(t)** statement is necessary. Without it, the Java compiler would flag an “unreachable code” error because the compiler would know that the last **println()** statement would never be executed. To prevent this error, the **if** statement is used here to trick the compiler for the sake of this demonstration.

Example programs

1. Write a java program to display a string of output

```
class welcome
{
    public static void main(String args[])
    {
        System.out.println("Welcome to java");
    }
}
```

2. Write a java program to display a string n times

```
class welcome10
{
    public static void main(String args[])
    {
        int i,n=10;
        for(i=0;i<n;i++)
            System.out.println("Welcome to java");
    }
}
```

3. Write a java program to find whether n is even or odd

```
class even
{
    public static void main(String args[])
    {
        int n=20;
        if(n%2==0)
            System.out.println("Number is even");
        else
            System.out.println("Number is odd");
    }
}
```

4. Write a java program to find whether n is prime or not

```
class prime
{
    public static void main(String args[])
    {
        int n=17,i,count=0;
        for(i=1;i<=n;i++)
        {
```

```

        if(n%i==0)
            count++;
    }
    if(count==2)
        System.out.println("Number is prime");
    else
        System.out.println("Number is not prime");
}
}

```

5. Write a java program to find factorial of a given number

```

class fact
{
    public static void main(String args[])
    {
        int n=5,i,f=1;
        for(i=1;i<=n;i++)
            f*=i;
        System.out.println("Factorial of the given number is:"+f);
    }
}

```

6. Write a java program to display even numbers from 1..100

```

class series
{
    public static void main(String args[])
    {
        int n=100,i;
        System.out.println("Even number series (1..100):");
        for(i=1;i<=n;i++)
        {
            if(i%2==0)
                System.out.print(i+"\t");
        }
    }
}

```

Control Structures

Example: Implement a Java program that prompts the user for an integer and then prints out all prime Numbers up to that Integer by using for and If.

import java.io.*;

```

import java.util.*;
class prime
{
    public static void main(String args[])throws IOException
    {
        BufferedReader bf=new BufferedReader(new
InputStreamReader(System.in));
        System.out.println("Enter n value");
        int n=Integer.parseInt(bf.readLine());
        int count=0;

        System.out.println("Lit of prime numbers upto "+n+"are:");
        for(int i=1;i<=n;i++)
        {
            count=0;
            for(int k=2;k<i;k++)
            {
                if(i%k==0)
                {
                    count++;
                    break;
                }
            }
            if(count==2)
                System.out.print("\t"+i);
        }
    }
}

```

Arrays

- An *array* is a collection of variables of homogeneous type that occupy contiguous memory locations.
- Arrays of any type can be created and may have one or more dimensions.
- A specific element in an array is accessed by its index.

- Arrays offer a convenient means of grouping related information. In Java all arrays are dynamically allocated.

One-Dimensional Arrays

- An array with one dimension is called one-dimensional array.
- Obtaining an array is a two-step process in java.
 1. declare a variable of the desired array type.

The general form of a one dimensional array declaration is :

type var-name[];

2. allocate the memory that will hold the array, using **new**, and assign it to the array variable. The general form of allocating memory using **new** is:

array-var = new type[size];

Here, *type* specifies the type of data being allocated, *size* specifies the number of elements in the array, and *array-var* is the array variable that is linked to the array. The elements in the array allocated by **new** will automatically be initialized to zero.

Example: int marks[];
 marks = new int[90];

The above two steps can also be combined into one step as follows;

int marks[]=new int[20];

Accessing Array elements:

One can access a specific element in the array by specifying its index. All array indexes start at zero. For example, to assign the value 55 to the second element of marks use,

marks[1] = 55;

To display the value stored at index 3.

System.out.println(marks[3]);

Initialization

Arrays can be initialized when they are declared. An *array initializer* is a list of comma-separated expressions surrounded by curly braces.

```
int month_days[] = { 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31 };
```

Note: If you try to access elements outside the range of the array (negative numbers or numbers greater than the length of the array), cause a run-time error.

Example: Write a java program to find a given element using linear search

```
import java.util.*;
import java.io.*;
class linearsearch
{
    public static void main(String args[])throws IOException
    {
        BufferedReader br=new BufferedReader(new
        InputStreamReader(System.in));
        System.out.println("Enter n value");
        int n=Integer.parseInt(br.readLine());
        int a[]=new int[n];
        System.out.println("Enter array elements");
        for(int i=0;i<n;i++)
            a[i]=Integer.parseInt(br.readLine());
        System.out.println("Enter search element");
        int s=Integer.parseInt(br.readLine());
        boolean found=false;
        for(int i=0;i<n;i++)
            if(a[i]==s)
            {
                found=true;
                break;
            }
        if(found==true)
            System.out.println("Element Found");
        else
            System.out.println("Element not Found");
    }
}
```

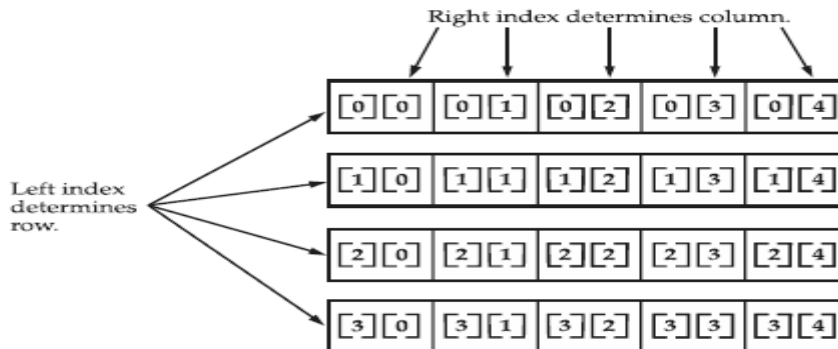
Multidimensional Arrays

An array with more than one dimension is called as a *multidimensional array*. An array with two dimensions is called a two-dimensional array.

For Example consider the following statement,

```
int a[][] = new int[4][5];
```

This creates an array with 4 rows and 5 columns. Internally this matrix is implemented as an *array of arrays* of **int**. Conceptually, this array will look like the one shown below.



Example: To illustrate two dimensional arrays

```
class TwoDArray
{
    public static void main(String args[])
    {
        int a[][]= new int[4][5];
        int i, j, k = 0;
        for(i=0; i<4; i++)
        {
            for(j=0; j<5; j++)
            {
                a[i][j] = k;
                k++;
            }
        }

        for(i=0; i<4; i++)
        {
            for(j=0; j<5; j++)
                System.out.print(a[i][j] + " ");
            System.out.println();
        }
    }
}
```

When memory is allocated for a multidimensional array, we need only specify the memory for the first (leftmost) dimension. We can allocate the remaining dimensions separately.

For example, this following code allocates memory for the first dimension of a when it is declared. It allocates the second dimension manually.

```
int a[][] = new int[4][];  
a[0] = new int[5];  
a[1] = new int[5];  
a[2] = new int[5];  
a[3] = new int[5];
```

We can manually allocate differing size second dimensions.

```
int b[][] = new int[4][];  
b[0] = new int[1];  
b[1] = new int[2];  
b[2] = new int[3];  
b[3] = new int[4];
```

The array b looks like below

[0][0]			
[1][0]	[1][1]		
[2][0]	[2][1]	[2][2]	
[3][0]	[3][1]	[3][2]	[3][3]

It is possible to initialize multidimensional arrays. To do so, simply enclose each dimension's initializer within its own set of curly braces. We can use expressions as well as literal values inside of array initializers.

```
double m[][] = { {0.0, 1.0, 2.0, 3.5},  
                 {0.4, 1.3, 2.0, 3.4},  
                 {0.1, 1.5, 2.5, 3.5},  
                 {0.1, 1.1, 2.2, 3.3}}
```

```
};
```

Example : Implement a Java program to multiply two given matrices.

```
import java.lang.*;
import java.io.*;
class matmul
{
    public static void main(String args[])throws IOException
    {
        BufferedReader bf=new BufferedReader(new
InputStreamReader(System.in));
        System.out.println("Enter dimensions of first matrix");
        int m=Integer.parseInt(bf.readLine());
        int n=Integer.parseInt(bf.readLine());

        System.out.println("Enter dimensions of second matrix");
        int p=Integer.parseInt(bf.readLine());
        int q=Integer.parseInt(bf.readLine());

        if(n==p)
        {
            int a[][]=new int[m][n];
            int b[][]=new int[p][q];
            int c[][]=new int[m][q];

            System.out.println("Enter values of the first matrix");
            for(int i=0;i<m;i++)
                for(int j=0;j<n;j++)
                    a[i][j]=Integer.parseInt(bf.readLine());

            System.out.println("Enter values of the second matrix");
            for(int i=0;i<p;i++)
                for(int j=0;j<q;j++)
                    b[i][j]=Integer.parseInt(bf.readLine());

            for(int i=0;i<m;i++)
            {
                for(int j=0;j<q;j++)
                {
                    c[i][j]=0;
                    for(int k=0;k<n;k++)
                        c[i][j]=c[i][j]+a[i][k]*b[k][j];
                    System.out.print(c[i][j]+"\\t");
                }
            }
        }
    }
}
```

```
        }  
        System.out.println();  
    }  
}
```

Class fundamentals

The class is at the core of Java. The class is a logical construct which defines the shape and nature of an object.

A class defines a new data type. Once defined, this new type can be used to create objects of that type. Thus, a class is a *template* for an object, and an object is an *instance* of a class.

The General Form of a Class

A class contains both the ***data*** and the ***code*** that operates on that data.

A class is declared by use of the **class** keyword.

The general form of a **class** definition is:

```
class classname  
{  
    type instance-variable1;
```

```

type instance-variable2;
// ...
type instance-variableN;
type methodname1(parameter-list)
{
    // body of method
}
type methodname2(parameter-list)
{
    // body of method
}
// ...
type methodnameN(parameter-list)
{
    // body of method
}
}

```

The data, or variables, defined within a **class** are called *instance variables*. The code is contained within *methods*. Collectively, the methods and variables defined within a class are called *members* of the class.

Variables defined within a class are called instance variables because each instance of the class (that is, each object of the class) contains its own copy of these variables. Thus, the data for one object is separate and unique from the data for another.

A Simple Class

Example:

Here is a class called **Box** that defines three instance variables: **width**, **height**, and **depth**. Currently, **Box** does not contain any methods.

```
class Box
{
    double width;
    double height;
    double depth;
}
```

A class defines a new type of data. In this case, the new data type is called Box. This name can be used to declare objects of type Box.

It is important to remember that a **class** declaration only creates a template; it does not create an actual object.

To actually create a **Box** object, you will use a statement like the following:

```
Box mybox = new Box( );
```

```
// create a Box object called mybox
```

After this statement executes, **mybox** will be an instance of **Box**. Thus, it will have “physical” reality. Every **Box** object will contain its own copies of the instance variables **width**, **height**, and **depth**.

To access variables, use the *dot* (.) operator along with the object name. The dot operator links the name of the object with the name of an instance variable. To assign a value to the data member the syntax is as follows.

Object . datamember;

Example: to assign the **width** variable of **mybox** the value 100, you would use the following statement:

```
mybox.width = 100;
```

```
/* A program that uses the Box class.
```

```
Call this file BoxDemo.java
```

```
*/
```



```
class Box
{
    double width;
    double height;
    double depth;
}
```

```
// This class declares an object of type Box.
```

```

class BoxDemo
{
    public static void main(String args[])
    {
        Box mybox = new Box();
        double vol;
        // assign values to mybox's instance variables
        mybox.width = 10;
        mybox.height = 20;
        mybox.depth = 15;
        // compute volume of box
        vol = mybox.width * mybox.height * mybox.depth;
        System.out.println("Volume is " + vol);
    }
}

```

We should save this file that contains this program **BoxDemo.java**, because the **main()** method is in the class called **BoxDemo**, not in the class called **Box**.

To run this program, you must execute **BoxDemo.class**. When you do, you will see the following Output: **Volume is 3000.0**

Example2: To illustrate creation of class and object

```

class student
{
    int no;
    char grade;
    float percent;
}

class demo
{
    public static void main(String args[])
    {
        student s=new student();
        s.no=10;
        s.grade='c';
        s.percent=98;
        System.out.println("number =" +s.no);
        System.out.println("grade =" +s.grade);
        System.out.println("percent =" +s.percent);
    }
}

```

```
    }  
}
```

Output:

Number =10

Grade=c

Percent=98

We should save this file that contains this program **demo.java**, because the **main()** method is in the class called **demo**, not the class called **student**.

Declaring Objects

With a class definition a new data type is created. We can use this type to declare objects of that type.

Obtaining objects of a class is a two-step process.

- First, declare a variable of the class type. This variable does not define an object. Instead, it is simply a variable that can *refer* to an object.

Second, acquire an actual, physical copy of the object and assign it to that variable. This can be done using the **new** operator.

The **new** operator dynamically allocates (that is, allocates at run time) memory for an object and returns a reference to it. This reference is, more or less, the address in memory of the object allocated by **new**. This reference is then stored in the variable. This reference is then stored in the variable.

Example:

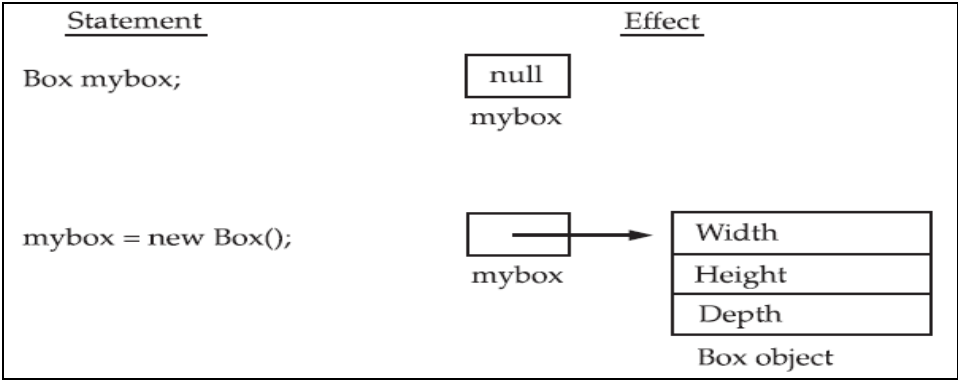
```
Box mybox; // declare reference to object
```

```
mybox = new Box(); // allocate a Box object
```

The first line declares **mybox** as a reference to an object of type **Box**. After this line executes, **mybox** contains the value **null**, which indicates that it does not yet point to an actual object. Any attempt to use **mybox** at this point will result in a compile-time error.

The next line allocates an actual object and assigns a reference to it to **mybox**. After the second line executes, you can use **mybox** as if it were a **Box** object. But in reality, **mybox** simply holds the memory address of the actual **Box** object.

The effect of these two lines of code is depicted in the figure as follows



The above two statements can also be combines into a single statement as follows

```
Box mybox = new Box();
```

The new operator

The **new** operator dynamically allocates memory for an object. It has the general form:

```
class-var = new classname( );
```

Here, ***class-var*** is a variable of the class type being created. The ***classname*** is the name of the class that is being instantiated. The class name followed by parentheses specifies the ***constructor*** for the class. A constructor defines what occurs when an object of a class is created.

It is important to understand that **new** allocates memory for an object during run time. The advantage of this approach (using new) is that the program can create as many or as few objects as it needs during the execution of the program.

The distinction between a class and an object:

- A class creates a new data type that can be used to create objects. That is, a class creates a logical framework that defines the relationship between its members.
- When we declare an object of a class, we are creating an instance of that class. Thus, a class is a logical construct. An object has physical reality. (That is, an object occupies space in memory.)

Assigning Object Reference Variables

Object reference variables act differently than you might expect when an assignment takes place.

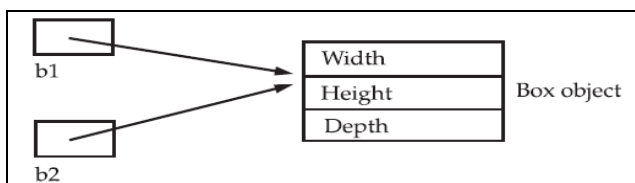
For example:

```
Box b1 = new Box();  
Box b2 = b1;
```

Here **b2** is being assigned a reference to a copy of the object referred to by **b1**. That means **b1** and **b2** will both refer to the *same* object.

The assignment of **b1** to **b2** did not allocate any memory or copy any part of the original object. It simply makes **b2** refer to the same object as does **b1**. Thus, any changes made to the object through **b2** will affect the object to which **b1** is referring, since they are the same object.

This situation is depicted here:



For example, a subsequent assignment to **b1** will simply *unhook* **b1** from the original object without affecting the object or affecting **b2**.

For example:

```
Box b1 = new Box();
Box b2 = b1;
// ...
b1 = null;
```

Here, **b1** has been set to **null**, but **b2** still points to the original object.

Introducing Methods

Classes usually consist of two things: instance variables and methods. In most classes, the instance variables are acted upon and accessed by the methods defined for that class. Thus the methods that determine how a class' data can be used.

The general form of a method is:

```
type name(parameter-list)
{
    // body of method
}
```

- Here, *type* specifies the type of data returned by the method. If the method does not return a value, its return type must be **void**.
- The name of the method is specified by *name*.
- The *parameter-list* is a sequence of type and identifier pairs separated by commas.

Methods that have a return type other than **void** return a value to the calling routine using the following form of the **return** statement:

```
return value;
```

Here, *value* is the value returned.

Adding a Method to the Box Class:

Methods provide access to data. In addition to defining methods that provide access to data, we can also define methods that are used internally by the class itself.

// This program includes a method inside the box class.

```
class Box
{
    double width;
    double height;
    double depth;
    // display volume of a box

void volume()
    {
        System.out.print("Volume is ");
        System.out.println(width * height * depth);
    }
}
```

```

class BoxDemo
{
    public static void main(String args[])
    {
        Box b1 = new Box();
        // assign values to mybox1's instance variables
        b1.width = 10;
        b1.height = 20;
        b1.depth = 15;
        // display volume of box
        b1.volume();
    }
}

```

Output:

Volume is 3000.0

`b1.volume();` invokes the **volume()** method ,that is, it calls **volume()** relative to the **b1** object, using the object's name followed by the dot operator.

When **b1.volume()** is executed, the Java run-time system transfers control to the code defined inside **volume()**. After all the statements inside **volume ()** have executed, control is returned to the calling routine, and execution resumes with the line of code following the call.

Returning a Value

A better way to implement **volume()** is to have it compute the volume of the box and return the result to the caller.

```

class Box
{
    double width;
    double height;

```



```

    double depth;
        // compute and return volume
    double volume()
    {
        return (width * height * depth);
    }
}

class BoxDemo
{
    public static void main(String args[])
    {
        Box b1 = new Box();
        Double vol;
        // assign values to mybox1's instance variables
        b1.width = 10;
        b1.height = 20;
        b1.depth = 15;
        // display volume of box
        vol=b1.volume();
        System.out.println("Volume is " + vol);
    }
}

```

Output:

Volume is 3000.0

There are two important things to understand about returning values:

- The type of data returned by a method must be compatible with the return type specified by the method. For example, if the return type of some method is **boolean**, you could not return an integer.
- The variable receiving the value returned by a method (such as **vol**, in this case) must also be compatible with the return type specified for the method.

Adding a Method That Takes Parameters:

Parameters allow a method to be generalized. That is, a parameterized method can operate on a variety of data and/or be used in a number of slightly different situations.

```

class Box
{
    double width;
    double height;
    double depth;
    // compute and return volume

    double volume()
    {
        return (width * height * depth);
    }

    // sets dimensions of box
    void setDim (double w, double h, double d)
    {
        width = w;
        height = h;
        depth = d;
    }
}

```

```

class BoxDemo
{
    public static void main(String args[])
    {
        Box b1 = new Box();
        Double vol;

        b1.setDim(10, 20, 15);    // initialize box

        System.out.println("Volume is " + b1.volume());    // display volume of
box

```

```
}  
}
```

Output:
Volume is 3000.0

- The **setDim()** method is used to set the dimensions of each box.
- For example, when `mybox1.setDim(10, 20, 15);` is executed, 10 is copied into parameter **w**, 20 is copied into **h**, and 15 is copied into **d**.
- Inside **setDim()** the values of **w**, **h**, and **d** are then assigned to **width**, **height**, and **depth**, respectively.

It is important to keep the two terms *parameter* and *argument* straight.

A **parameter** is a variable defined by a method that receives a value when the method is called.

For example, `setDim(double w, double h, double d)` → where w, h, d are parameters

An **argument** is a value that is passed to a method when it is invoked.

For example, `setDim(10, 20, 15);` passes 10, 20, 30 as arguments.

Constructors

It can be tedious to initialize all of the variables in a class each time an instance is created. Automatic initialization of data members of a class is performed through the use of a constructor.

- A *constructor* initializes an object immediately upon creation.
- A Constructor has the same name as the class name.
- Constructor syntactically similar to a method.
- Constructor is automatically called when the object is created before the **new** operator completes.
- **Constructors have no return type**, not even **void**.

This is because the implicit return type of a class' constructor is the class type itself.

/* Here, Box uses a constructor to initialize the dimensions of a box. */

```
class Box
{
    double width;
    double height;
    double depth;

    // This is the constructor for Box.
    Box()
    {
        System.out.println("Constructing Box");
        width = 10;
        height = 10;
        depth = 10;
    }

    // compute and return volume
    double volume()
    {
        return width * height * depth;
    }
}

class BoxDemo
{
    public static void main(String args[])
```

```

{
    Box b1 = new Box();
    Box b2 = new Box();

    double vol;
    vol = b1.volume();
    System.out.println("Volume is " + vol);

    vol = b2.volume();
    System.out.println("Volume is " +
vol);
}
}

```

Output:

```

Constructing Box
Constructing Box
Volume is 1000.0
Volume is 1000.0

```

let's reexamine the **new** operator. As you know, when you allocate an object, you use the following general form:

class-var = new *classname*();

Now you can understand why the parentheses are needed after the class name. What is actually happening is that the constructor for the class is being called. Thus, in the line

Box b1 = new Box();

new Box() is calling the **Box()** constructor. **When you do not explicitly define a constructor for a class, then Java creates a default constructor for the class.**

The default constructor automatically initializes all instance variables to zero. The default constructor is often sufficient for simple classes, but it usually won't do for more sophisticated ones. Once you define your own constructor, the default constructor is no longer used.

Parameterized Constructors:

The default constructor (Box()) in the preceding example does initialize a **Box** object. But it is not very useful because all boxes have the same dimensions. What is needed is a way to construct **Box** objects of various dimensions. The easy solution is to add parameters to the constructor.

A constructor which takes parameters is called a parameterized constructor.

```
class Box
{
    double width;
    double height;
    double depth;
    // This is the constructor for Box.

    Box(double w, double h, double d)
    {
        width = w;
        height = h;
        depth = d;
    }
    // compute and return volume

    double volume()
    {
        return width * height * depth;
    }
}

class BoxDemo
{
    public static void main(String args[])
    {
        // declare, allocate, and initialize Box objects
        Box b1 = new Box(10, 20, 15);
        Box b2 = new Box(1, 2, 3);

        double vol;

        // get volume of first box
        vol = b1.volume();
        System.out.println("Volume is " + vol);
        // get volume of second box
        vol = b2.volume();
        System.out.println("Volume is " + vol);
    }
}
```

The output from this program is shown here:

Volume is 3000.0

Volume is 6.0

Each object is initialized as specified in the parameters to its constructor.

For example, in the following line,

```
Box b1 = new Box(10, 20, 15);
```

the values 10, 20, and 15 are passed to the **Box()** constructor when **new** creates the object.

Thus, **b1**'s copy of **width**, **height**, and **depth** will contain the values 10, 20, and 15, respectively.

The this Keyword and Instance Variable Hiding:

- The keyword '**this**' is used inside any method to refer to the *current* object.
- That is, '**this**' keyword is always a reference to the object on which the method was invoked.
- You can use **this** anywhere a reference to an object of the current class' type is permitted.
- whenever the formal parameters and data members of a class are similar, to differentiate data members of a class from formal parameters, the data members of the class are preceded by "this"

```
class Test
{
    int a,b;

    Test (int a, int b)
    {
        System.out.println(" In parameterized constructor");
        this.a=a;
        this.b=b;

        System.out.println("a= "+a+" this.a= "+this.a);
        this.a=this.a+10;
        System.out.println("a= "+a+" this.a= "+this.a);
    }
}
```

```
void display()
{
    System.out.println(a); // Internally treated as this.a
    System.out.println(this.a);
}
}
```

```
class ThisDemo
{
    public static void main(String a[])
    {
        Test t=new Test(100,200);
        t.display();
    }
}
```

Output:

```
a= 100  this.a= 100
a= 100  this.a= 110
110
110
```


Garbage Collection

- Since objects are dynamically allocated by using the **new** operator, you might be wondering how such objects are destroyed and their memory released for later reallocation.
- In some languages, such as C++, dynamically allocated objects must be manually released by use of a **delete** operator.
- Java takes a different approach which handles deallocation automatically. The technique that accomplishes this is called *garbage collection*.
- *garbage collection* works like this: when no references to an object exist, that object is assumed to be no longer needed, and the memory occupied by the object can be reclaimed.
- Garbage collection only occurs **sporadically** (periodically) during the execution of your program. It will not occur simply because one or more objects exist that are no longer used.
- Different Java run-time implementations will take varying approaches to garbage collection, but for the most part, you should not have to think about it while writing your programs.

+Overloading Methods:

- In Java it is possible to define two or more methods within the same class that share the same name, as long as their parameter declarations are different. This process is referred to as *method overloading*.
- Method overloading is one of the ways that Java implements polymorphism.
- When an overloaded method is invoked, Java uses the type and/or number of arguments to determine which version of the overloaded method to actually call. Thus, overloaded methods must differ in the type and/or number of their parameters.
- While overloaded methods may have different return types, the return type alone is insufficient to distinguish two versions of a method.

- When Java encounters a call to an overloaded method, it simply executes the version of the method whose parameters match the arguments used in the call.

Here is a simple example that illustrates method overloading:

Example: To illustrate the concept of method overloading in java

```
import java.lang.*;
import java.io.*;

class Overload
{
    void test()
    {
        System.out.println("TEST with out parameters");
    }

    void test(int a)
    {
        System.out.println("parameters:"+a);
    }
}

class OverloadDemo
{
    public static void main(String args[])
    {
        Overload o=new Overload();
        o.test();
        o.test(10);
    }
}
```

In the above example **test()** is overloaded four times.

- The first version takes no parameters, the second takes one integer parameter, the third takes two parameters; one integer and one character, and the fourth takes one character parameter and also returns an integer.
- When an overloaded method is called, Java looks for a match between the arguments used to call the method and the method's parameters.
- In some cases Java's automatic type conversions can play a role. Java will employ its automatic type conversions only if no exact match is found.

Overloading Constructors

Constructor methods can also be overloaded in the same as how methods are overloaded..

```
/* Here, Box defines three constructors to initialize  
the dimensions of a box various ways.  
*/
```

```
class Box
{
    Box()
    {
        System.out.println("default constructor");    }

    Box(int x)
    {
        System.out.println("constructor with one parameter:"+x);
    }
}
```

```
class OverloadCons
{
    public static void main(String args[])
    {

        Box b1 = new Box();
        Box b2 = new Box(7);

    }
}
```

Output:

default constructor

constructor with one parameter:7

Using Objects as Parameters

Normally we pass simple types as parameters to methods. We can also pass objects to methods.

For example, consider the following program:

// Objects may be passed to methods.

```
class Test
{
    int a, b;
    Test(int i, int j)
    {
        a = i;
        b = j;
    }
    // return true if o is equal to the invoking object
    boolean equals(Test o)
    {
        if(o.a == a && o.b == b)
            return true;
        else
            return false;
    }
}
class PassOb
{
    public static void main(String args[])
    {
        Test ob1 = new Test(100, 22);
        Test ob2 = new Test(100, 22);
    }
}
```

```

        Test ob3 = new Test(-1, -1);
        System.out.println("ob1 == ob2: " + ob1.equals(ob2));
        System.out.println("ob1 == ob3: " + ob1.equals(ob3));
    }
}

```

This program generates the following output:

```

ob1 == ob2: true
ob1 == ob3: false

```

- The **equals()** method inside **Test** compares two objects for equality and returns the result. That is, it compares the invoking object with the one that it is passed.
- If they contain the same values, then the method returns **true**. Otherwise, it returns **false**.
- Notice that the parameter **o** in **equals()** specifies **Test** as its type. Although **Test** is a class type created by the program, it is used in just the same way as Java's built-in types.

A Closer Look at Argument Passing:

- There are two ways that a computer language can pass an argument to a subroutine.
 - *call-by-value*
 - *call-by-reference*.

call-by-value:

This approach copies the *value* of an argument into the formal parameter of the subroutine. Therefore, changes made to the parameter of the subroutine have no effect on the argument.

In Java, when you pass a primitive type to a method, it is passed by value. Thus, what occurs to the parameter that receives the argument has no effect outside the method.

// Primitive types are passed by value.

```

class Test
{
    void meth(int i, int j)
    {
        i *= 2;
        j /= 2;
    }
}

```

```

    }
}
class CallByValue
{
    public static void main(String args[])
    {
        Test ob = new Test();
        int a = 15, b = 20;
        System.out.println("a and b before call: " + a + " " + b);
        ob.meth(a, b);
        System.out.println("a and b after call: " + a + " " + b);
    }
}

```

Output:

a and b before call: 15 20
a and b after call: 15 20

As you can see, the operations that occur inside **meth()** have no effect on the values of **a** and **b** used in the call; their values here did not change to 30 and 10.

call-by-reference:

In this approach, a reference to an argument (not the value of the argument) is passed to the parameter. Inside the subroutine, this reference is used to access the actual argument specified in the call. This means that changes made to the parameter will affect the argument used to call the subroutine.

When you pass an object to a method, changes to the object inside the method *do* affect the object used as an argument.

// Objects are passed by reference.

```

class Test
{
    int a, b;
    Test(int i, int j)
    {
        a = i;
        b = j;
    }
    // pass an object
    void meth(Test o)

```

```

        {
            o.a *= 2;
            o.b /= 2;
        }
    }
}
class CallByRef
{
    public static void main(String args[])
    {
        Test ob = new Test(15, 20);
        System.out.println("ob.a and ob.b before call: " + ob.a + " " + ob.b);
        ob.meth(ob);
        System.out.println("ob.a and ob.b after call: " + ob.a + " " + ob.b);
    }
}

```

Output:

ob.a and ob.b before call: 15 20
ob.a and ob.b after call: 30 10

In this case, the actions inside **meth()** have affected the object used as an argument. When an object reference is passed to a method, the reference itself is passed by use of call-by-value. However, since the value being passed refers to an object, the copy of that value will still refer to the same object that its corresponding argument does.

Returning Objects

A method can return any type of data, including class types.

For example, in the following program, the **incrByTen()** method returns an object in which the value of **a** is ten greater than it is in the invoking object.

// Returning an object.

```

class Test
{
    int a;
    Test(int i)
    {
        a = i;
    }
    Test incrByTen()
    {

```



```

        Test temp = new Test(a+10);
        return temp;
    }
}
class RetOb
{
    public static void main(String args[])
    {
        Test ob1 = new Test(2);
        Test ob2;
        ob2 = ob1.incrByTen();
        System.out.println("ob1.a: " + ob1.a);
        System.out.println("ob2.a: " + ob2.a);
        ob2 = ob2.incrByTen();
        System.out.println("ob2.a after second increase: "+ ob2.a);
    }
}

```

The output generated by this program is shown here:

ob1.a: 2

ob2.a: 12

ob2.a after second increase: 22

Each time **incrByTen()** is invoked, a new object is created, and a reference to it is returned to the calling routine.

The preceding program makes another important point: Since all objects are dynamically allocated using **new**, you don't need to worry about an object going out-of-scope because the method in which it was created terminates. The object will continue to exist as long as there is a reference to it somewhere in your program. When there are no references to it, the object will be reclaimed the next time garbage collection takes place.

Recursion

Java supports *recursion*. Recursion is the process of defining something in terms of itself, that is recursion is the attribute that allows a method to call itself. A method that calls itself is said to be *recursive*.

Here is how a factorial can be computed by use of a recursive method:

```

// A simple example of recursion.
class Factorial

```

```

{
    // this is a recursive method
    int fact(int n)
    {
        int result;
        if(n==1) return 1;
        result = fact(n-1) * n;
        return result;
    }
}
class Recursion
{
    public static void main(String args[])
    {
        Factorial f = new Factorial();
        System.out.println("Factorial of 3 is " + f.fact(3));
        System.out.println("Factorial of 4 is " + f.fact(4));
        System.out.println("Factorial of 5 is " + f.fact(5));
    }
}

```

The output from this program is shown here:

Factorial of 3 is 6

Factorial of 4 is 24

Factorial of 5 is 120

When **fact()** is called with an argument of 1, the function returns 1; otherwise, it returns the product of **fact(n-1)*n**. To evaluate this expression, **fact()** is called with **n-1**. This process repeats until **n** equals 1 and the calls to the method begin returning.

How recursion works:

- When a method calls itself, new local variables and parameters are allocated storage on the stack, and the method code is executed with these new variables from the start.
- As each recursive call returns, the old local variables and parameters are removed from the stack, and execution resumes at the point of the call inside the method.
- Recursive methods could be said to “telescope” out and back.

Disadvantages:

- Recursive versions of many routines(methods) may execute a bit more slowly than the iterative equivalent because of the added overhead of the additional function calls.
- Many recursive calls to a method could cause a stack overrun. Because storage for parameters and local variables is on the stack and each new call creates a new copy of these variables, it is possible that the stack could be exhausted. If this occurs, the Java run-time system will cause an exception.

Advantages:

- The **main advantage** to recursive methods is that they can be used to create clearer and simpler versions of several algorithms than can their iterative relatives.
- For example, the Quick-Sort sorting algorithm is quite difficult to implement in an iterative way. Also, some types of AI-related algorithms are most easily implemented using recursive solutions.

Note:

When writing recursive methods, you must have an **if** statement somewhere to force the method to return without the recursive call being executed. If you don't do this, once you call the method, it will never return. This is a very common error in working with recursion.

Example: Write a java program to display Fibonacci series upto n using recursion and without using recursion

```
import java.io.*;
import java.util.*;
class fib
{
    int rfib(int n)
    {
        if(n==0)
            return (0);
        else if(n==1)
            return (1);
        else
            return (rfib(n-1)+rfib(n-2));
    }
    void sfib(int n)
    {
        int a=0,b=1,c=1;
        System.out.println(a+"\n"+b);
    }
}
```

```

        for(int i=2;i<n;i++)
        {
            c=a+b;
            a=b;
            b=c;
            System.out.println(c+"\t");
        }
    }
}
class fibo
{
    public static void main(String[] args) throws Exception
    {
        BufferedReader bf=new BufferedReader(new
InputStreamReader(System.in));
        System.out.println("Enter n value");
        int n=Integer.parseInt(bf.readLine());
        fib f=new fib();
        System.out.println("Fibonacci series using recursion");
        for (int i=0;i<n;i++)
        {
            System.out.println(f.rfib(i));
        }
        System.out.println("Fibonacci series with out using recursion");
        f.sfib(n);
    }
}

```

Access Control

Encapsulation links data with the code that manipulates it. Encapsulation provides another important attribute: *access control*. Through encapsulation, we can control what parts of a program can access the members of a class.

Java offers four access specifiers, listed below

- a. Private
- b. Protected
- c. default (no specifier)
- d. public**

private:

private members can only be accessed within the same class to which they belong to. It is mostly used for encapsulation: data are hidden within the class. Only the methods within that class can access these members.

protected:

Protected members can only be accessed within the same class to which the methods and fields belong, within its subclasses, and within classes of the same package, but not from anywhere else.

default (no specifier):

If no access specifier is used then such a class, method, or field will be accessible from inside the same package to which the class, method, or field belongs, but not from outside this package. This access-level is convenient if you are creating packages.

public:

public classes, methods, and fields can be accessed from everywhere. The only constraint is that a file with Java source code can only contain one public class whose name must also match with the filename.

Example: To illustrate java's access specifiers.

```
class Test
{
    int a;
    public int b;
    private int c;
    void setc(int i)
    {
        c = i;
    }
    int getc()
    {
        return c;
    }
}
class Accesstest
{
    public static void main(String args[])
    {
        Test ob = new Test();
        //a and b may be accessed directly
        ob.a = 10;
    }
}
```

```

        ob.b = 20;
        // ob.c = 100; // Error!
        ob.setc(100);
        System.out.println("a, b, and c: " + ob.a + " , " + ob.b + " and " +
ob.getc());
    }
}

```

Output:

a, b, and c:10 , 20 and 100

Access Specifier	Class	Subclass	Package	Others
Private	√			
Protected	√	√	√	
Public	√	√	√	√
Default(None)	√		√	

Static in java:

- Normally a class member must be accessed using an object of that class.
- It is possible that a class member can be accessed without using an object if it (class member) is declared as static.
- To create a static member, precede its declaration with the keyword **static**.
- Both methods and variables can be declared as **static**.
- The most common example of a **static** member is **main()**. **main()** is declared as **static** because it must be called before any objects exist.
- Instance variables declared as **static** are, essentially, global variables.
- When objects of its class are declared, no copy of a **static** variable is made. All instances of the class share the same **static** variable.

Methods declared as static have the following rules:

- ✓ They can only call other **static** methods.
- ✓ They must only access **static** data.
- ✓ They cannot refer to **this** or **super** in any way.

If you need to do computation in order to initialize your **static** variables, you can declare a **static** block that gets executed exactly once, when the class is first loaded.

// Demonstrate static variables, methods, and blocks.

```
class StaticDemo
{
    static int a = 3;
    static int b;

    static void meth(int x)
    {
        System.out.println("x = " + x);
        System.out.println("a = " + a);
        System.out.println("b = " + b);
    }

    static
    {
        System.out.println("Static block initialized.");
        b = a * 4;
    }

    public static void main(String args[])
    {
        meth(42);
    }
}
```

Output:

Static block initialized.

x = 42

a = 3

b = 12

As soon as the **StaticDemo** class is loaded, all of the **static** statements are **run**.

First, **a** is set to **3**, then the **static** block executes, which prints a message and then initializes **b** to **a * 4** or **12**. Then **main()** is called, which calls **meth()**, passing **42** to **x**. The three **println()** statements refer to the two **static** variables **a** and **b**, as well as to the local variable **x**.

Outside of the class in which they are defined, **static** methods and variables can be used independently of any object. They can be accessed by the name of their class followed by the dot operator.

General form:

classname.method()

Here, *classname* is the name of the class in which the **static** method is declared.

```
class Static1
{
    static int a = 42;
    static int b = 99;
    static void callme()
    {
        System.out.println("a = " + a);
    }
}
class Static2
{
    public static void main(String args[])
    {
        Static1.callme();
        System.out.println("b = " + Static1.b);
    }
}
```

Output:

a = 42

b = 99

final keyword

A variable can be declared as **final**. If a variable is declared as final its contents cannot be modified. A **final** variable must be initialized when it is declared.

For example:

```
final int FILE_NEW = 1;
final int FILE_OPEN = 2;
final int FILE_SAVE = 3;
final int FILE_SAVEAS = 4;
final int FILE_QUIT = 5;
```


In subsequent parts of the program we can use **FILE_OPEN**, etc., as if they were Constants. Variables declared as **final** do not occupy memory on a per-instance basis. Thus, a **final** variable is essentially a constant. The keyword **final** can also be applied to methods, but its meaning is substantially different than when it is applied to variables.

Nested and Inner Classes

A class which is defined within another class is known as *nested class*. If class B is defined within class A, then B is known to A, but not outside of A. A nested class has access to the members, including private members, of the outer class. The outer class does not have access to the members of the nested class. There are two types of nested classes: *static* and *non-static*.

A static nested class is one which has the **static** modifier applied. Because it is static, it must access the members of its outer class through an object.

An inner class is a non-static nested class. It has access to all of the variables and methods of its outer class.

Example:

```
class Outer
{
    int outer_x = 100;
    void test()
    {
        Inner inner = new Inner();
        inner.display();
    }
    class Inner
    {
        void display()
        {
            System.out.println("display: outer_x = " + outer_x);
        }
    }
}
class InnerClassDemo
{
    public static void main(String args[])
    {
        Outer outer = new Outer();
        outer.test();
    }
}
```

```
}
```

Output from this application is shown here:

```
display: outer_x = 100
```

It is possible to define inner classes within any block scope. For example, you can define a nested class within the block defined by a method or even within the body of a **for** loop,

anonymous inner class is an inner and that don't have a name.

Exploring the String Class

String is probably the most commonly used class in Java's class library.

In Java a *string* is a sequence of characters. But, unlike many other languages that implement strings as character arrays, Java implements strings as objects of type **String**.

The first thing to understand about strings is that every string you create is actually an object of type **String**. Even string constants are actually **String** objects.

For example, in the statement

```
System.out.println ("This is a String, too");
```

the string "This is a String, too" is a **String** constant.

The second thing to understand about strings is that objects of type **String** are immutable; once a **String** object is created, its contents cannot be altered. While this may seem like a serious restriction, it is not, for two reasons:

- If you need to change a string, you can always create a new one that contains the modifications.
- Java defines a peer class of **String**, called **StringBuffer**, which allows strings to be altered, so all of the normal string manipulations are still available in Java.

Strings can be constructed in a variety of ways. The easiest is to use a statement like this:

```
String myString = "this is a test";
```

Once you have created a **String** object, you can use it anywhere that a string is allowed.

For example, this statement displays **myString**:

```
System.out.println(myString);
```

Java defines one operator for **String** objects: **+**. It is used to concatenate two strings.

For example, this statement

```
String myString = "I" + " like " + "Java.";
results in myString containing "I like Java."
```

The following program demonstrates the preceding concepts:

```
// Demonstrating Strings.
class StringDemo
{
    public static void main(String args[])
    {
        String strOb1 = "First String";
        String strOb2 = "Second String";
        String strOb3 = strOb1 + " and " + strOb2;
        System.out.println(strOb1);
        System.out.println(strOb2);
        System.out.println(strOb3);
    }
}
```

The output produced by this program is shown here:

```
First String
Second String
First String and Second String
```

The **String** class contains several methods that you can use. Here are a few. You can test two strings for equality by using **equals()**. You can obtain the length of a string by calling the **length()** method. You can obtain the character at a specified index within a string by calling **charAt()**.

The general forms of these three methods are shown here:

```
boolean equals(String object)
int length( )
char charAt(int index)
```

Here is a program that demonstrates these methods:

```
// Demonstrating some String methods.
class StringDemo2
{
```

```

public static void main(String args[])
{
    String strOb1 = "First String";
    String strOb2 = "Second String";
    String strOb3 = strOb1;
    System.out.println("Length of strOb1: " +strOb1.length());
    System.out.println("Char at index 3 in strOb1: " +strOb1.charAt(3));

    if(strOb1.equals(strOb2))
        System.out.println("strOb1 == strOb2");
    else
        System.out.println("strOb1 != strOb2");

    if(strOb1.equals(strOb3))
        System.out.println("strOb1 == strOb3");
    else
        System.out.println("strOb1 != strOb3");
}
}

```

This program generates the following output:

```

Length of strOb1: 12
Char at index 3 in strOb1: s
strOb1 != strOb2
strOb1 == strOb3

```

Of course, you can have arrays of strings, just like you can have arrays of any other type of object. For example:

```

// Demonstrate String arrays.
class StringDemo3
{
    public static void main(String args[])
    {
        String str[] = { "one", "two", "three" };
        for(int i=0; i<str.length; i++)
            System.out.println("str[" + i + "]: " +str[i]);
    }
}

```

Here is the output from this program:

```

str[0]: one
str[1]: two

```

str[2]: three

As you will see in the following section, string arrays play an important part in many Java programs.

The String Class

In java every string is an object of type **String**. Strings can be constructed a variety of ways. The easiest is

```
String str1="welocme";
```

The other ways to create a sting are:

```
String str2=new String("java");
```

```
String str3=new String(str2);
```

```
String str4=str1+str2;
```

The operator '+' is used to concatenate two strings.

Now str4 contains the string: welcome java

Few methods of String class:

1. To compare two strings for equality use **equals()**.
2. boolean equals(String *object*)
3. To obtain the length of a string use **length()**.

```
int length( )
```

4. To obtain the character at a specified index within a string use **charAt()**.

```
char charAt(int index)
```

String vs StringBuffer

java provides the string buffer and string classes and the string class is used to manipulate character strings that cannot be changed. Simply stated objects of type string are read only and immutable. The string buffer class is used to represent characters that can be modified.

Example

```
class str  
{
```

```

public static void main(String[] args)
{
    String str1=new String("welocme");
    String str2=new String("java");
    String str3=str1+str2;
    System.out.println("STRING1:"+str3);
    System.out.println("Length of the string is:"+str3.length());
    System.out.println("s1==s2:"+str1.equals(str2));
    System.out.println("chartAt(3):"+str3.charAt(3));
}
}

```

Command-Line Arguments

Sometimes you will want to pass information into a program when you run it. One of the way to accomplish this is by passing *command-line arguments* to **main()**. A command-line argument is the information that directly follows the program's name on the command line when it is executed. The command line arguments are stored as strings in the **String** array passed to **main()**. All command-line arguments are passed as strings.

```

import java.io.*;
class cmdline
{
    public static void main(String args[])
    {
        System.out.println("Number of arguments are:"+args.length);
        System.out.println("arguments are:");
        for(int i=0;i<args.length;i++)
        {
            System.out.println(args[i]+"\\t");
        }
    }
}
>java cmdline 10 welcome cse x

```

When we give the above ststatement at command line the following output will be given:

```

Number of arguments are: 4
arguments are :

```

Object Oriented Programming through JAVA

UNIT-I - Question Bank

One Mark Questions:

1. The wrapping up of data & methods into a single unit is called as _____
2. _____ is the process by which object of one class acquires the properties of object of another class .
3. The concept of inheritance provides the idea of _____
a. Taking more than one form b. reusability c. data hiding
4. The derived class is known as _____
a. Superclass b. subclass c. parentclass
5. The class from which the subclass derives the properties is called as _____
a. Superclass b. subclass c. base class
6. The property or the ability to take more than one form is called as _____
a. Encapsulation b. Polymorphism c. inheritance
7. The most striking feature of java is that it is _____
a. Java compiler b. java interpreter c. platform independent
8. Java is developed by _____ in 1991 at USA
a. Microsoft b. sun Microsystems c. IBM
9. Java compilers convert source code into _____
a. Unicode b. byte code c. psuedocode

10. _____ translates byte code into machine code
a. Java interpreter b. java compiler c. assembler
11. _____ initializes an object immediately upon creation
12. What is Ternary Operator?
13. What is one-dimensional Array?
14. What is boolean data type?
15. What is JVM?
16. What is a constructor?
17. Define JIT Compiler (Just in –time compiler).
18. What is meant by **abstraction**?
19. Who is the father of **java**?
20. What is the size of **char** data type?

2 Mark Questions:

1. What is **platform independency**?
2. . Write the difference between **interpreter** and **compiler**?
3. What is the use of **final** keyword?
4. What are the different types of **operators** used in Java?
5. Write the syntax of for **control structure**?
6. What is the use of **new** keyword?
7. What is the use of **this** keyword?
8. What is the difference between a **constructor** and a **method**?
9. What is an **access specifier** in java?
10. What is a **class, object** and **method**?

5 Marks Questions

1. Briefly discuss about java program and its execution process.
2. What is a class? How an object is created for a class.
3. What is a constructor? Explain with an example.
4. Write a java program to find given n umber is prime number or not using the concepts class, object and a method.
5. Explain the static keyword with an example.
6. Define overloading. Explain method overloading.
7. Explain in detail about constructor overloading with an example
8. Explain classes and objects of java.
9. Explain the access specifiers (Visibility Controls) supported by Java with an example code.
- 10.Explain scope and lifetime in java.

10 Marks Questions

1. Explain the Java buzzwords (features) in detail.
2. What is array? What are the different types arrays supported by Java? Explain in detail with examples.
3. What are the Operators available with JAVA language? Explain each operator with syntax.
4. What differences are between while, do while and for looping structures.
5. What is data type? Explain data types with example.

UNIT-2

Inheritance, Packages and Interfaces

Inheritance

- Inheritance is an important feature of object oriented programming.
- Inheritance is the feature of **acquiring the properties(instance variables & methods) of other class.**
- The class which inherits the properties of other is known as subclass (derived class, child class)
- The class whose properties are inherited is known as superclass (base class, parent class).
- A *subclass* inherits members of superclass and adds its own, unique elements.

The general form of a **class** declaration that inherits a superclass is shown here:

```
class superclass
{
    //body of super class
}
class subclass extends superclass
{
    // body of sub class
}
```

In java “*extends*” is the keyword used to inherit the properties other class. The class which inherits the properties is called sub class and the class from which the properties are inherited is called the super class.

Inheritance represents the **IS-A relationship**, also known as *parent-child* relationship.

Why use inheritance in java

- For Method Overriding (so runtime polymorphism can be achieved).
- For Code Reusability.

Types of inheritance

- Single inheritance
- Multi level inheritance
- Hierarchical Inheritance
- Multiple inheritance (**possible through interface**)
- Hybrid inheritance(**possible through interface**)

Single Inheritance:

When a class extends another one class only then we call it a single inheritance. The below flow diagram shows that class B extends only one class which is A. Here A is a **parent class** of B and B would be a **child class** of A.



(a) Single Inheritance

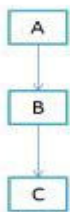
Single Inheritance example program in Java

```
class A
{
    public void mA()
    {
        System.out.println("Base class method");
    }
}

class B extends A
{
    public void mB()
    {
        System.out.println("Child class method");
    }
    public static void main(String args[])
    {
        B b = new B();
        b.mA(); //calling super class method
        b.mB(); //calling local method
    }
}
```

2) Multilevel Inheritance

Multilevel inheritance refers to a mechanism in OOP technology where one can inherit from a derived class, thereby making this derived class the base class for the new class. As you can see in below flow diagram C is subclass or child class of B and B is a child class of A.



(d) Multilevel Inheritance

It's pretty clear with the diagram that in Multilevel inheritance there is a concept of grandparent class. If we take the example of above diagram then class C inherits class B and class B inherits class A which

means B is a parent class of C and A is a parent class of B. So in this case class C is implicitly inheriting the properties and method of class A along with B that's what is called multilevel inheritance.

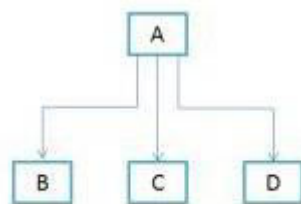
Multilevel Inheritance example program in Java

```
class X
{
    public void mX()
    {
        System.out.println("Class X method");
    }
}
class Y extends X
{
    public void mY()
    {
        System.out.println("class Y method");
    }
}
class Z extends Y
{
    public void mZ()
    {
        System.out.println("class Z method");
    }
    public static void main(String args[])
    {
        Z obj = new Z();
        obj.mX(); //calling grand parent class method
        obj.mY(); //calling parent class method
        obj.mZ(); //calling local method
    }
}
```

3) Hierarchical Inheritance

In this inheritance, one class is inherited by many **sub classes**.

In below example class B, C and D **inherits** the same class A. A is **parent class (or base class)** of B, C & D.



(c) Hierarchical Inheritance

As you can see in the above diagram that when a class has more than one child classes (sub classes) or in other words more than one child classes have the same parent class then such kind of inheritance is known as hierarchical.

Let's have a look at the below example program to understand it better –

I'm using the above figure for implementing hierarchical inheritance in the below example-

```
class A
{
    public void mA()
    {
        System.out.println("method of Class A");
    }
}

class B extends A
{
    public void mB()
    {
        System.out.println("method of Class B");
    }
}

class C extends A
{
    public void mC()
    {
        System.out.println("method of Class C");
    }
}

class Demo
{
    public static void main(String args[])
    {
        B b = new B();
        C c = new C();
        b.mA();
        c.mA();
    }
}
```

The above would run perfectly fine with no errors and the **output** would be –

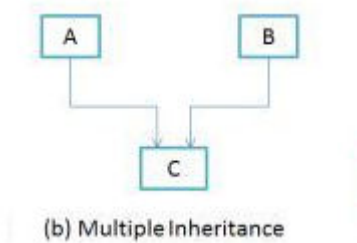
method of Class A
method of Class A

4) Multiple Inheritance

Multiple Inheritance is nothing but **one** class **extending more than** one class.

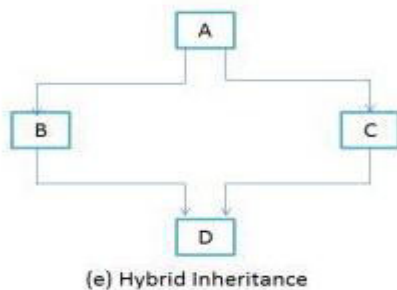
Multiple Inheritance is basically not supported by many **Object Oriented Programming** languages such as **Java, Small Talk, C# etc.. (C++ Supports Multiple Inheritance)**. As the **Child** class has to manage the dependency of more than one **Parent** class.

But you can achieve multiple inheritance in Java using **Interfaces**.



5) Hybrid Inheritance

Hybrid Inheritance is the combination of both Single and Multiple Inheritance. Again Hybrid inheritance is also not directly supported in Java only through interface we can achieve this. Flow diagram of the Hybrid inheritance will look like below. As you can **ClassA** will be acting as the **Parent** class for **ClassB & ClassC** and **ClassB & ClassC** will be acting as **Parent** for **ClassD**.



As you can see in the above diagram that it's a combine form of single and multiple inheritance. Since java doesn't support multiple inheritance, the hybrid inheritance is also not possible.

Case 1: Using classes: If in above figure B and C are classes then this inheritance is not allowed as a single class cannot extend more than one class (Class D is extending both B and C). Reason explained below!!

Case 2: Using Interfaces: If B and C are interfaces then the above hybrid inheritance is allowed as a single class can implement any number of interfaces in java.

Let's understand the above concept with the help of examples:

Example program 1: Using classes to form multiple inheritance or hybrid inheritance

```
public class A
{
    public void mA()
    {
        System.out.println("Class A methodA");
    }
}
```

```

public class B
{
    public void mB()
    {
        System.out.println("Class B methodB");
    }
}

public class C extends A, B
{
    public void methodC()
    {
        System.out.println("Class C methodC");
    }

    public static void main(String args[])
    {
        C c = new C();
        c.mA();
    }
}

```

Output:

Error!!

Why? Most of the times you will find the following explanation of above error – Multiple inheritance is not allowed in java so class C cannot extend two classes(A and B). java. In C++ it's allowed.

What's the solution? Multiple & Hybrid inheritances are supported using interfaces.

Member Access and Inheritance

Although a subclass includes all of the members of its superclass, it cannot access those members of the superclass that have been declared as **private**. In a class hierarchy, private members remain private to their class.

```

class A
{
    private int i
    int j;
    void mA()
    {
        System.out.println("i and j: " + i + " " + j);
    }
}

class B extends A
{
    void mB()
    {
        System.out.println("i: " + i);
    }
}

```

```

        System.out.println("j: " + j);
    }
}
class Sample
{
    public static void main(String args[])
    {
        A o1= new A();
        B o2 = new B();
        System.out.println("Contents of superOb: ");
        o1.showij();
        System.out.println("Contents of subOb: ");
        o2.showjk();
    }
}

```

In the above program the data member `j` is inherited but `i` is not inherited as it is a private member. A class member that has been declared as private will remain private to its class. It is not accessible by any code outside its class, including subclasses.

super :

super is a **keyword**. It is used inside a sub-class method **definition** to call a method **defined** in the **super** class.

Private methods of the **super**-class cannot be called. Only public and protected methods can be called by the **super** keyword.

super is used in two cases.

- It is used to **invoke the superclass** constructor from subclass.
- It is used to access a member of the superclass that has been hidden by a member of a subclass.

It is used to **differentiate the members** of superclass from the members of subclass, if they have same names.

a. Using super to Call Superclass Constructors

A subclass can call a constructor method defined by its immediate superclass by use of the following form of **super**:

```

super( )
super(parameter-list);

```

Here, *parameter-list* specifies any parameters needed by the constructor in the superclass. **super()** must always be the first statement executed inside a subclass' constructor.

Example

```
class A
{
    A()
    {
        System.out.println("parent class constructor ");
    }
}

public class A extends B
{
    B()
    {
        super();
        System.out.println("child class constructor ");
    }

    public static void main(String arg[])
    {
        B b = new B();
    }
}
```

OUTPUT:

parent class constructor
child class constructor

In the above example super keyword is used to call the constructor in the super class. Parameters can also be passed to the constructor by enclosing them with in parentheses.

A Second Use for super (super used to invoke parent class member(variable & method))

If a class is inheriting the properties of another class. And if the members of the superclass have the names same as the sub class, to differentiate these members we use super keyword.

- super.variable
- super.method();

```
class A
{
    int a=10;

    void m()
    {
        System.out.println("Parent class method");
    }
}

class B extends A
{
}
```

```

int a=20;
void m()
{
    super.m( );
    System.out.println("parent class variable = " + super.a);
    System.out.println("child class variable = " + a);
    System.out.println("Child class method");
}

public static void main(String args[])
{
    B b= new B();
    b.m( );
}
}

```

Output:

```

Parent class method
parent class variable = 10
child class variable = 20
Child class method

```

NOTE:

If there is no method overriding then we do not need to use super to call the parent class method.

Order Constructor execution with inheritance

Constructors are called in order of derivation, from superclass to subclass. Further, since **super()** must be the first statement executed in a subclass' constructor, this order is the same whether or not **super()** is used. If **super()** is not used, then the default or parameterless constructor of each superclass will be executed.

Example

```

class A
{
    A()
    {
        System.out.println("Inside A's constructor.");
    }
}
class B extends A
{
    B()
    {
        System.out.println("Inside B's constructor.");
    }
}

```

```

    }
}
class C extends B
{
    C()
    {
        System.out.println("Inside C's constructor.");
    }
}
class multicons
{
    public static void main(String args[])
    {
        C c = new C();
    }
}

```

The output from this program is:

Inside A's constructor

Inside B's constructor

Inside C's constructor

Method Overriding

When a method in a subclass has the same name and type signature as a method in its superclass, then the method in the subclass is said to *override* the method in the superclass.

When an overridden method is called from within a subclass, it will always refer to the version of that method defined by the subclass. The version of the method defined by the superclass will be hidden.

Example:

```

class A
{
    void meth()        // overriding method
    {
        System.out.println(" A-method");
    }
}

class B extends A
{
    void meth()        // overriding method
    {
        System.out.println("B-method");
    }

    public static void main(String args[])
    {

```

```

        B b= new B();
        b.meth();
    }
}

```

Output: B-method

Note: overridden methods allow Java to support run-time polymorphism.

dynamic method dispatch (Run time polymorphism using method overriding)

Runtime polymorphism or **Dynamic Method Dispatch** is a process in which a call to an overridden method is resolved at runtime rather than compile-time.

In this process, an overridden method is called through the reference variable of a superclass. The determination of the method to be called is based on the object being referred to by the reference variable.

Let's first understand the upcasting before Runtime Polymorphism.

Upcasting

When reference variable of Parent class refers to the object of Child class, it is known as upcasting. For example:



```

class A
{
    void meth()
    {
        System.out.println(" A-method");
    }
}
class B extends A
{
    void meth()
    {
        System.out.println("B-method");
    }
}

public static void main(String args[])
{
    A a;
    a= new B();//upcasting
    a.meth();
}

```

Abstraction in Java

Abstraction is a process of hiding the implementation details and showing only functionality to the user.

There are two ways to achieve abstraction in java

1. Abstract class
2. Interface

Abstract Class

A class with abstract keyword is known as abstract class in java.
An abstract class can have both abstract and non-abstract methods (method with body).

- An abstract class must contain at least one abstract method.
- An abstract class **cannot be instantiated**.
- An abstract class contains a child class; the child class usually provides implementations for all of the abstract methods of abstract class.
- However, if the child class does not provides implementations of all abstract methods, then the subclass must also be declared abstract.
- **Should not declare abstract constructors, or abstract static methods.**

Syntax:

```
abstract class classname
{
    //body
}
```

abstract method

A method that is declared as abstract and does not have implementation is known as abstract method.

To declare an abstract method, use this general form:

abstract type name(parameter-list);

Example : abstract void draw();

EXAMPLE:

```
abstract class A
{
    abstract void x();

    void y()
    {
        System.out.println("This is a concrete method.");
    }
}
```

```

    }
}

Class B extends A
{
    void x()
    {
        System.out.println("implementation of abstract method x.");
    }
}

class Demo
{
    public static void main(String args[])
    {
        B b = new B();
        b.y();
        b.x();
    }
}

```

Output:

This is a concrete method
implementation of abstract method x

final with Inheritance

The keyword **final** has three uses.

- It can be used to create a constant. (variable)
- Can be used to prevent a method from being overridden (method)
- Can be used to prevent a class from being inherited (class)

a. Using final to create constant data members

A variable can be declared as **final**. If a variable is declared as final its contents can not be modified. A **final** variable must be initialized when it is declared.

For example:

```

class A
{
    final int x=10; //final variable
    x=30;
}

```

Output: Compile Time Error

b. Using final to Prevent Overriding

To disallow a method from being overridden, use **final** as a modifier at the start of its declaration. Methods declared as **final** cannot be overridden.

Example:

```
class A
{
    final void meth()
    {
        System.out.println("This is a final method.");
    }
}
```

In the class **A**, **meth()** is declared as **final**, it cannot be overridden in subclasses.

c. Using final to Prevent Inheritance

A class can also be prevented from being inherited. Use **final** to do so. Declaring a class as **final** implicitly declares all of its methods as **final**, too.

Note: It is illegal to declare a class as both **abstract** and **final**

Example

```
final class A {
// ...
}
```

The Object Class

Object is the super class of all other classes in java. All other classes are subclasses of **Object** class. So that a variable of type **Object** can refer to an object of any other class. **Object** defines the following methods, which means that they are available in every object.

Method	Purpose
Object clone()	Creates a new object that is the same as the object being cloned.
boolean equals(Object <i>object</i>)	Determines whether one object is equal to another.
void finalize()	Called before an unused object is recycled.
Class getClass()	Obtains the class of an object at run time.
int hashCode()	Returns the hash code associated with the invoking object.
void notify()	Resumes execution of a thread waiting on the invoking object.
void notifyAll()	Resumes execution of all threads waiting on the invoking object.
String toString()	Returns a string that describes the object.
void wait()	Waits on another thread of execution.
void wait(long <i>milliseconds</i>)	
void wait(long <i>milliseconds</i> , int <i>nanoseconds</i>)	

The methods `getClass( )`, `notify( )`, `notifyAll( )`, and `wait( )` are declared as **final**. The `equals( )` method compares the contents of two objects. It returns **true** if the objects are equivalent, and **false** otherwise. The `toString( )` method returns a string that contains a description of the object on which it is called.

Packages

In java a package is a collection of classes. Groups of related classes are often packaged as reusable components.

Java packages are of two types:

- Predefined packages
- User defined packages

Benefits of packages:

- As applications become more complex, package helps to manage the complexity of application components.
- Packages facilitate software reuse by enabling programs to import classes from other packages.
- Packages provide a convention for unique class names, which helps prevent class-name conflicts.

A Java source-code file must have the following order:

1. A package declaration (if any),
2. import declarations (if any), then
3. class declarations.

Only one of the class declarations in a particular file can be public.

User defined packages:

Defining a Package (user defined)

Include a **package** command as the first statement in a Java source file. Any classes declared within that file will belong to the specified package. If the **package** statement is omitted, the class names are put into the default package, which has no name.

Syntax : ***package package-name;***

Example: package pack1;

Here, pack1 is the name of the package and creates a package called **pack1**.

- Java uses file system directories to store packages. The **.class** files for any classes that are declared as part of **pack1** must be stored in a directory called **pack1**.
- More than one file can include the same **package** statement. The **package** statement simply specifies to which package the classes defined in a file belong.
- We can create a hierarchy of packages. To do so, simply separate each package name from the one above it by use of a period.
- The general form of a multileveled package statement is shown here: `package pkg1[.pkg2[.pkg3]];`

Importing Packages

All of the standard classes in java are stored in some named package. Java includes the **import** statement to bring certain classes, or entire packages, into visibility.

Once imported, a class can be referred to directly, using only its name. In a Java source file, **import** statements occur immediately following the **package** statement (if it exists) and before any class definitions.

Syntax:

`import pkg1[.pkg2].(classname|*);`

Here, *pkg1* is the name of a top-level package, and *pkg2* is the name of a sub package inside the outer package separated by a dot (.).

Finally, specify either an explicit *classname* or a star (*), which indicates that the Java compiler should import the entire package.

Example:

```
import java.util.Scanner;  
import java.io.*;
```

All of the standard Java classes included with Java are stored in a package called **java**. Java.lang package is implicitly imported by the compiler for all programs. When a package is imported, only those items within the package declared as **public** will be available to non-subclasses in the importing code.

Example: A program to define & import user defined package

A.java

```
package p1;  
public class A  
{  
  
    public void m()  
    {  
        System.out.println(" class-A - package-1");  
    }  
}
```

demopack.java

```
package p2;  
import p1.*;  
  
class B  
{  
    public static void main(String args[])  
    {  
        A a = new A();  
        a.m();  
    }  
}
```

Output:

```
class-A-package-1
```

Access Protection

Java provides many levels of protection to allow fine-grained control over the visibility of variables and methods within classes, subclasses, and packages. Classes and packages are both means of encapsulating

and containing the name space and scope of variables and methods. Java addresses four categories of visibility for class members:

- Subclasses in the same package
- Non-subclasses in the same package
- Subclasses in different packages
- Classes that are neither in the same package nor subclasses

The three access specifiers, **private**, **public**, and **protected**, provide a variety of ways to produce the many levels of access required by these categories. Anything declared **public** can be accessed from anywhere. Anything declared **private** cannot be seen outside of its class. When a member does not have an explicit access specification (default), it is visible to subclasses as well as to other classes in the same package. A protected member can be accessed outside the current package, but only to the sub classes.

Access Specifier	Private	No modifier	Protected	Public
Same class	√	√	√	√
Same package sub class		√	√	√
Same package non-subclass		√	√	√
Different package subclass			√	√
Different package non-subclass				√

A class has only two possible access levels: default and public. When a class is declared as **public**, it is accessible by any other code. If a class has default access, then it can only be accessed by other code within its same package.

Finding Packages and CLASSPATH

For example, consider the following package specification.

```
package pack1;
```

In order for a program to find **pack1**, one of two things must be true. Either the program is executed from a directory immediately above **pack1**, or **CLASSPATH** must be set to include the path to **pack1**.

Predefined packages

Java provides a rich set of predefined classes that are grouped into categories of related classes called packages. These packages are referred as the Java Application Programming Interface (Java API), or the Java class library. These packages can be imported into programs and reused.

Some packages in java API

java.lang	The Java Language Package contains classes and interfaces that are required by many Java programs. This package is imported by the compiler into all programs, need not to use import keyword.
java.io	The Java Input/Output Package contains classes and interfaces that enable programs to input and output data.
java.util	The Java Utilities Package contains utility classes and interfaces that enable such actions as date and time manipulations, randomnumber processing (class Random), the storing and processing of large amounts of data and the breaking of strings into smaller pieces called tokens (class StringTokenizer).
java.applet	The Java Applet Package contains a class and several interfaces required to create Java applets—programs that execute in Web browsers.
java.awt	The Java Abstract Window Toolkit Package contains the classes and interfaces required to create and manipulate GUIs.

Interfaces

- The interface in java is **a mechanism to achieve fully abstraction**.
- In general an interface specifies *what the* operation is performed but does not specify *how* the operation is performed.
- An interface may not specify any implementation details.
- All methods declared in an interface are implicitly **public abstract methods** and all data members are implicitly **public, static and final**.
- It is used to achieve fully abstraction and multiple inheritance in Java.
- An interface is syntactically similar to the class. An **interface declaration** begins with the keyword **interface**.
- **Interface can only have abstract methods, they cannot have concrete methods**
- **Interface can extend any number of interfaces at a time**

Defining an Interface

An interface is defined much like a class. This is the general form of an interface:

Access-specifier interface interface-name

{

```

    type  method1 (parameter-list);
    type  method2 (parameter-list);
    type  var1 = value;
    type  var2 = value;
    .....
}

```

Here, *access_specifier* is either **public** or none, *interface_name* is the name of the interface. The methods which are declared in the interface have no bodies. Variables declared inside of interface are implicitly **final** and **static**. They must be initialized with a constant value.

Use of interfaces

- **Interfaces can not be instantiated.** With respect to an interface we cannot create an object directly, but we can create indirectly.
- They can be inherited to other interfaces or to other classes.
- Using interfaces the concept of multiple inheritances can be achieved. Once an interface is defined, any number of classes can implement an **interface**. Also, one class can implement any number of interfaces.
- Each class that includes an interface must implement all of the methods declared in the interface.
- The data members declared in the interface cannot be changed by the implementing class.

Implementing Interfaces

- Once an **interface** has been defined, one or more classes can implement that interface.
- To implement an interface, include the **implements** clause in a class definition, and then create the methods defined by the interface.

The general form of a class that includes the **implements** clause looks like this:

```

Access-specifier class classname [extends superclass] [implements interface [,interface...]]
{
    // class-body
}

```

Here, *access_specifier* is either **public** or none. If a class implements more than one interface, the interfaces are separated with a comma. The methods that implement an interface must be declared **public**. Also, the type signature of the implementing method must match exactly the type signature specified in the **interface** definition.

EXAMPLE:

```

interface I1
{
    void m();
}

class A implements I1
{
    public void m()
    {
        System.out.println("Example interface");
        System.out.println(a);
    }
    public static void main(String args[])
    {
        A a = new A();
        a.m();
    }
}

```

Interfaces Can Be Extended

One interface can inherit another interface by use of the keyword **extends**. The syntax is the same as for inheriting classes. When a class implements an interface that inherits another interface, it must provide implementations for all methods defined within

EXAMPLE:

```

interface I1
{
    void m1();
}
interface I2
{
    void m2();
}
interface I3 extends I1 , I2
{
    void m3();
}
class A implements I3
{
    public void m1()
    {
        System.out.println("Implement m1().");
    }
    public void m2()
    {
        System.out.println("Implement m2().");
    }
    public void m3()

```

```

        {
            System.out.println("Implement m3().");
        }
    }
}
class Demo
{
    public static void main(String arg[])
    {
        A a = new A();
        a.m1();
        a.m2();
        a.m3();
    }
}

```

OUTPUT:

Implement m1()

Implement m2()

Implement m3()

Explanation:

I1 is an interface which contains the declaration of method **m1()**.

I2 is an interface which contains the declaration of method **m2()**.

I3 is an interface which extends **I1 & I2** and contains the declaration of method **m3()**.

In the class **A**, which implements interface **I3** and all the methods **m1()**, **m2()** and **m3()** must be **defined**.

Any class that implements an interface must implement all methods defined by that interface, including any that are inherited from other interfaces.

Accessing Implementations through Interface References

We can declare variables as object references that use an interface rather than a class type. Any instance of any class that implements the declared interface can be referred to by such a variable. The following example calls the method **m()** via an interface reference variable:

```

class demo
{
    public static void main(String args[])
    {
        IF1 i = new A( );
        i.m( );
    }
}

```

Notice that variable **i** is declared to be of the interface type **IF1**, yet it was assigned an instance of the class **A**. Although **i** can be used to access the **m()** method, it cannot access any other members of the **A** class.

Partial Implementations

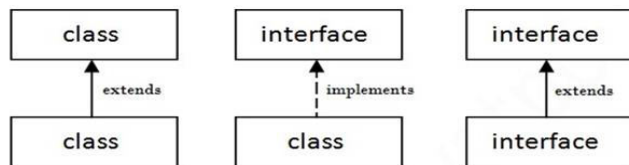
If a class includes an interface but does not fully implement the methods defined by that interface, then that class must be declared as **abstract**.

For example:

```
abstract class A implements interface
{
    // partial implementation of interface methods
}
```

Here, the class A does not implement **all the methods of interface then it** must be declared as abstract.

NOTE:



difference between Abstract Class and Interface:

	abstract Classes	Interfaces
1	abstract class can extend only one class or one abstract class at a time	interface can extend any number of interfaces at a time
2	abstract class can extend from a class or from an abstract class	interface can extend only from an interface
3	abstract class can have both abstract and concrete methods	interface can have only abstract methods
4	A class can extend only one abstract class	A class can implement any number of interfaces
5	In abstract class keyword 'abstract' is mandatory to declare a method as an abstract	In an interface keyword 'abstract' is optional to declare a method as an abstract
6	abstract class can have protected, public and public abstract methods	Interface can have only public abstract methods i.e. by default
7	abstract class can have static, final or static final variable with any access specifier	interface can have only static final (constant) variable i.e. by default

List of Questions in UNIT-II

1. What is a constructor? What is constructor overloading?
2. What is the difference between nested class and inner class?
3. What is the dynamic method dispatch?
4. What is inheritance? Types of inheritance? Give example for each type.
5. Explain Creating and accessing a package?
6. Explain Implementing interface and extending interfaces?

7. What is the use with super keyword? Explain with examples.
8. Explain how multiple inheritance can be implemented in java.
9. Define package. Write 5 methods in any 3 predefined packages.

Exception Handling, Multithreading

UNIT-3

Exception Handling

Definition

An *exception* is an abnormal condition that arises in a code sequence at run time.

In other words, an exception is a run-time error.

Advantages of exception handling:

- Exception handling enables you to create applications that can handle exceptions.
- This feature enables programmers to write **robust** and **fault-tolerant programs** (i.e., programs that are able to deal with problems that may arise and continue executing).

Exception-Handling

Java exception handling is managed via five keywords:

 **try**

 **catch**

 **throw**

 **throws**

 **finally**

- Program statements that are to be monitored for exceptions are contained within a **try** block.
- If an exception occurs within the **try** block, it is thrown.
- The exception can be caught (using **catch**) and handle it in some rational manner.
- To manually throw an exception, use the keyword **throw**.
- Any exception that is thrown out of a method must be specified as such by a **throws** clause.
- Finally block is used to place important code and that code will be executed whether exception is handled or not.

This is the **general form** of an exception-handling block:

```
try
{
    // block of code to monitor for errors
    //code that may contain a run time error
}
catch (ExceptionType1 exOb)
{
```

```

        // exception handler for ExceptionType1
    }
    catch (ExceptionType2 exOb)
    {
        // exception handler for ExceptionType2
    }
    // ...
    finally
    {
        // block of code to be executed before try block ends
    }

```

Here, *ExceptionType* is the type of exception that has occurred. At least **one catch block** or a **finally block** must immediately follow the try block. When an exception occurs in a try block, the catch block that executes is the one whose type matches the type of the exception that occurred

Uncaught Exceptions

When an exception is not handled then what happens is explained using the run time error divide by zero.

```

import java.io.*;
import java.util.*;
class exc
{
    public static void main(String args[])throws IOException
    {
        BufferedReader bf=new BufferedReader(new InputStreamReader(System.in));
        int a=Integer.parseInt(bf.readLine());
        int b=Integer.parseInt(bf.readLine());
        int c=a/b;
        System.out.println(c);
    }
}

```

When the Java run-time system detects the attempt to divide by zero, it constructs a new exception object and then *throws* this exception. The program terminates. Once an exception has been thrown, it must be *caught* by an exception handler. In this example, exception handler is not available. Any exception that is not caught by the program will ultimately be processed by the default handler. The default handler displays a string describing the exception. Here is the output generated when this example is executed.

```
java.lang.ArithmeticException: / by zero at exc.main(exc.java:9)
```

Using try and catch

There are two benefits of handling an exception.

- First, it allows the programmer to fix the error.
- Second, it prevents the program from abnormal termination.

To handle a run-time error, simply enclose the code that is to be monitored inside a **try** block. Immediately following the **try** block, include a **catch** block that specifies the exception type that is to be caught.

ArithmeticException generated by the division-by-zero error:

```
import java.io.*;
import java.util.*;

class try_catch
{
    public static void main(String args[])
    {
        int a;

        try
        {
            a=5/0;
            System.out.println("Inside Try Block");
        }
        catch(ArithmeticException e)
        {
            System.out.println("Arithmetic- divide by zero exception");
        }
        System.out.println("Outside Try & Catch Block");
    }
}
```

Output:

```
Arithmetic- divide by zero exception
Outside Try & Catch Block
```

- Once an exception is thrown, program control transfers out of the **try** block into the **catch** block.
- Execution never “returns” to the **try** block from a **catch**.
- Once the **catch** statement has executed, program control continues with the next line in the program following the entire **try/catch** mechanism.
- The statements that are protected by **try** must be surrounded by curly braces.

Displaying a Description of an Exception

Description of the exception can be displayed with a **println()** statement by simply passing the exception as an argument.

```
catch(ArithmeticException e)
{
    System.out.println("Exception: "+e);
}
```

Output: it displays the following message

Exception: java.lang.ArithmeticException: / by zero

Multiple catch Clauses

In some cases, more than one exception could be raised by a single piece of code. To handle this type of situation, A try statement can be followed by more catch statements and each catch clause catches a different type of exception.

When an exception is thrown, each **catch** statement is inspected in order, and the first one whose type matches that of the exception is executed. The others are bypassed, and execution continues after the **try/catch** block.

Example: To demonstrate multiple catch statements.

```
class multicatch
{
    public static void main(String args[])
    {
        int a,b,c;
        a=Integer.parseInt(args[0]);
        b=Integer.parseInt(args[1]);
        int x[]={ 1,2,3,4};
        try
        {
            c=a/b;
            System.out.println(x[10]);
        }
        catch(ArithmeticException e)
        {
            System.out.println("Arithmetic- divide by zero exception");
        }
        catch(ArrayIndexOutOfBoundsException e)
        {
            System.out.println("ArrayIndexOut of bounds");
        }
    }
}
```

```

        System.out.println("After try/catch blocks.");
    }
}

```

Here is the output generated by running it different ways:

C:\>java multicatch 10 5

X[c]=3

After try/catch blocks

C:\>java multicatch 10 2

Array index oob: java.lang.ArrayIndexOutOfBoundsException

After try/catch blocks

C:\>java multicatch 10 0

Arithmetic- divide by zero exception

After try/catch blocks

Note: When multiple **catch** statements are used, exception subclasses must come before any of their superclasses. This is because a **catch** statement that uses a superclass will catch exceptions of that type plus any of its subclasses. Thus, a subclass would never be reached if it came after its superclass.

Nested try Statements

The **try** statement can be nested. That is, a **try** statement can be inside the block of another **try**. Each time a **try** statement is entered, the context of that exception is pushed on the stack.

If an inner **try** statement does not have a **catch** handler for a particular exception, then the next **try** statement's **catch** handlers are inspected for a match. This continues until one of the **catch** statements succeeds, or until the entire nested **try** statements are exhausted. If no **catch** statement matches, then the Java run-time system will handle the exception.

An example that uses nested **try** statements:

```

import java.io.*;
import java.util.*;
class nesttry
{
    public static void main(String args[])
    {
        int a,b,c;
    }
}

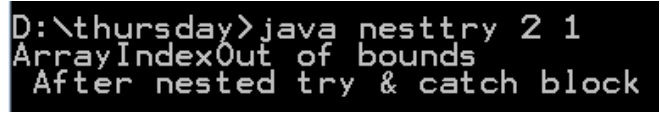
```

```

a=Integer.parseInt(args[0]);
b=Integer.parseInt(args[1]);
int x[]={1,2,3,4};
try
{
    c=a/b;
    try
    {
        System.out.println(x[100]);
    }
    catch(NullPointerException e)
    {
        System.out.println("Null pointer exception");
    }
}
catch(ArithmeticException e)
{
    System.out.println("Arithmetic- divide by zero exception");
}
catch(ArrayIndexOutOfBoundsException e)
{
    System.out.println("ArrayIndexOut of bounds");
}

System.out.println(" After nested try & catch block ");
}
}

```



```

D:\thursday>java nesttry 2 1
ArrayIndexOut of bounds
After nested try & catch block

```

throw:

- In general exceptions are thrown by Java run-time system. They can be handled in the programs.
- The keyword **throw** is used to throw an exception manually. Programmers can throw exceptions by using the throw statement. A throw statement specifies an object to be thrown.

The general form of **throw** is:

throw object;

The operand of a **throw** can be of any class derived from class **Throwable**. The flow of execution stops immediately after the **throw** statement and any subsequent statements are not executed.

The nearest enclosing **try** block is inspected to see if it has a **catch** statement that matches the type of the exception. If it does find a match, control is transferred to that statement. If not, then the next enclosing **try** statement is inspected, and so on. If no matching **catch** is found, then the default exception handler halts the program and prints the stack trace.

Here is a sample program that creates and throws an exception.

```
class throwex
{
    public static void main(String args[])
    {
        try
        {
            NullPointerException n=new NullPointerException("throw example");
            throw n;

            //throw new NullPointerException("throw example");
            //System.out.println("after throw");
        }
        catch(NullPointerException e)
        {
            System.out.println("nullpointer exception"+e);
        }
    }
}
```

Creating Own Exception Subclasses (using **throw** keyword)

In addition to the predefined exception classes programmer can create his/her own exceptions which are suitable for specific applications. These are called as user defined exceptions. To accomplish this, a class must be defined as a subclass of **Exception** class.

Example

```
import java.io.*;
import java.util.*;
```

```
class InvalidNumber extends Exception
{
```



```

        public String toString()
        {
            return ("number must be < 100");
        }
    }

class ThrowDemo
{
    public static void main(String args[])throws IOException
    {
        Scanner s = new Scanner (System.in);
        System.out.println("Enter a number");
        int a=s.nextInt();
        try
        {
            if(a>100)
                //throw new InvalidNumber();
                InvalidNumber i=new InvalidNumber();
                throw i;
            System.out.println("a value is:"+a);
        }

        catch(InvalidNumber e)
        {
            System.out.println("caught:"+e);
        }
    }
}

```

This example defines a subclass of **Exception** called **InvalidNumber**. This subclass is quite simple: it has only a constructor plus an overloaded **toString()** method that displays the value of the exception.

The “**ThrowDemo**” class defines the main method that throws a **InvalidNumber** object. The exception is thrown when user enters a value greater than 100 as input.

throws:

If a method is capable of causing an exception that it does not handle, it must specify this behavior so that callers of the method can guard themselves against that exception. This can be accomplished by adding a **throws** clause in the method’s declaration.

A **throws** clause lists the types of exceptions that a method might throw. *This is necessary for all exceptions, except those of type **Error** or **RuntimeException**, or any of their subclasses.*

All other exceptions that a method can throw must be declared in the **throws** clause. If they are not, a compile-time error will result.

Syntax:

```
type method-name( parameter-list ) throws exception-list
{
    // body of method
}
```

Here, *exception-list* is a comma-separated list of the exceptions that a method can throw.

Example:

FileNotFoundException- signals that a file could not be found.
IOException- produced by failed or interrupted I/O operations

```
import java.io.File;
import java.io.IOException;
import java.io.FileReader;

public class FileNotFound
{
    public static void main(String args[]) throws IOException
    {
        File f=new File("E://file.txt");
        FileReader fr = new FileReader(f); //FileNotFoundException
        fr.read(); //IOException
    }
}
```

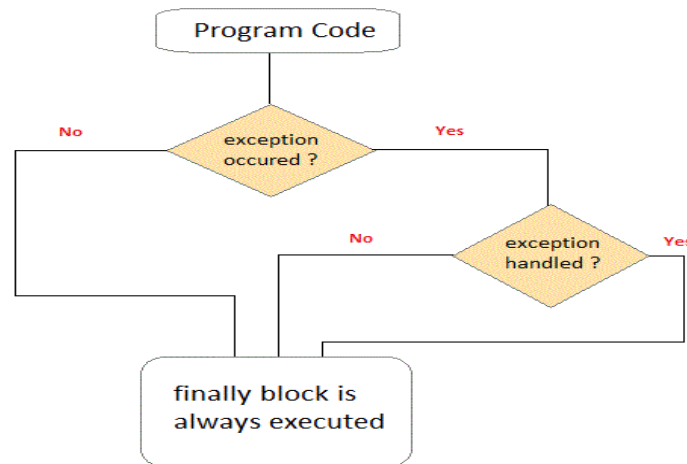
- ➔ Both FileNotFound & IOException are checked exceptions, when you used throws IOException those exceptions will ignore at compile time but they need to be handled at run time.

finally

- **finally** creates a block of code that will be executed after a **try/catch** block has completed and before the code following the **try/catch** block.
- The **finally** block will execute whether or not an exception is thrown.

- If an exception is thrown, the **finally** block will execute even if no **catch** statement matches the exception. The **finally** clause is optional. However, each **try** statement requires at least one **catch** or a **finally** clause.

A finally keyword is used to create a block of code that follows a try block. A finally block of code always executes whether or not exception has occurred. Using a finally block, lets you run any cleanup type statements that you want to execute, no matter what happens in the protected code. A finally block appears at the end of catch block.



Example demonstrating finally Clause

Class ExceptionTest

```
{
    public static void main(String[] args)
    {
        int a[]={1,2,3};
        System.out.println("out of try");
        try
        {
            System.out.println("Access invalid element"+ a[5]);
            /* the above statement will throw ArrayIndexOutOfBoundsException
*/
        }
        finally
        {
            System.out.println("finally is always executed.");
        }
    }
}
```

Output:

Out of try

finally is always executed.

Exception in thread main java. Lang. exception array Index out of bound exception.

You can see in above example even if exception is thrown by the program, which is not handled by catch block, still finally block will get executed.

Difference between throw and throws in Java

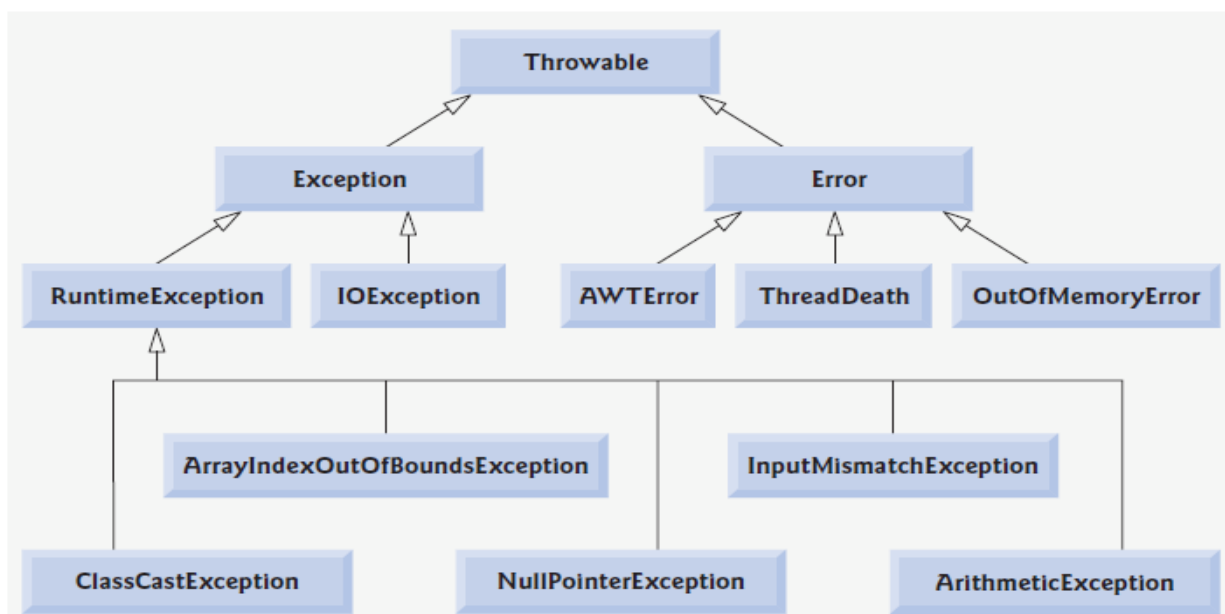
s.no	Throw	Throws
1)	Java throw keyword is used to explicitly throw an exception.	Java throws keyword is used to declare an exception.
2)	Checked exception cannot be propagated using throw only.	Checked exception can be propagated with throws.
3)	Throw is followed by an instance.	Throws is followed by class.
4)	Throw is used within the method.	Throws is used with the method signature.
5)	You cannot throw multiple exceptions.	You can declare multiple exceptions e.g. <code>public void method()throws IOException,SQLException.</code>

Types of Exceptions(Java's Built-in Exceptions):

Java defines several exception classes in java.lang package. **java.lang** is implicitly imported into all Java programs.

All exception types are subclasses of the built-in class **Throwable**. Throwable is a subclass of the Object class.

Throwable has two subclasses that partition exceptions into two distinct branches. They are **Exception** and **Error**



Exception:

- This class is used for exceptional conditions that user programs should catch.
- This is also the class that you will subclass to create your own exception types.
- There is an important subclass of **Exception**, called **RuntimeException**.
- **RuntimeException** has many descendants that include `ArrayIndexOutOfBoundsException`, `ArithmeticException`, `IllegalAccessException`, `NegativeArraySizeException` etc;

Exceptions are again classified into two categories:

Checked exceptions:

A checked exception is an exception that occurs at the compile time, these are also called as compile time exceptions.

These exceptions cannot simply be ignored at the time of compilation, the Programmer should take care of (handle) these exceptions.

Following are some of the checked exceptions in java.lang package

Exception	Meaning
<code>ClassNotFoundException</code>	Class not found
<code>IllegalAccessException</code>	Access to a class is denied
<code>NoSuchMethodException</code>	A requested method does not exist
<code>FileNotFoundException</code>	Access to the file which is not exist

For example, if you use **FileReader** class in your program to read data from a file, if the file specified in its constructor doesn't exist, then an *FileNotFoundException* occurs, and compiler prompts the programmer to handle the exception.

```
import java.io.File;
import java.io.FileReader;

public class FilenotFound_Demo
{
    public static void main(String args[])
    {
        File f=new File("E://file.txt");
        FileReader fr = new FileReader(f);
    }
}
```

```
}
```

If you try to compile the above program you will get exceptions as shown below.

```
C:\>javac FileNotFound_Demo.java
```

FileNotFound_Demo.java:8: error: unreported exception FileNotFoundException; must be caught or declared to be thrown

```
    FileReader fr = new FileReader(file);
```

```
    ^
```

1 error

Note: Since the methods **read()** and **close()** of **FileReader** class throws **IOException**, you can observe that compiler notifies to handle **IOException**, along with **FileNotFoundException**.

Unchecked exceptions:

- An Unchecked exception is an exception that occurs at the time of execution, these are also called as Runtime Exceptions.
- Runtime Exceptions include programming bugs, such as **ArrayIndexOutOfBoundsException**, **ArithmeticException**, **IllegalAccessException**, **NegativeArraySizeException** etc;
- Runtime exceptions are ignored at the time of compilation.

Following are some of the **unchecked** exceptions in **java.lang** package

Exception	Meaning
ArithmeticException	ArithmeticException Arithmetic error, such as divide-by-zero.
ArrayIndexOutOfBoundsException	Array index is out-of-bounds.
ArrayStoreException	Assignment to an array element of an incompatible type.
NegativeArraySizeException	Array created with a negative size
NullPointerException	Invalid use of a null reference.
NumberFormatException	Invalid conversion

For example, if you have declared an array of size 5 in your program, and trying to call the 6th element of the array then an *ArrayIndexOutOfBoundsException* occurs.

```
public class Unchecked_Demo
{
    public static void main(String args[])
    {
        int num[]={ 1,2,3,4};
        System.out.println(num[5]);
    }
}
```

```
}
```

If you compile and execute the above program you will get exception as shown below.

```
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: 5
    at Exceptions.Unchecked_Demo.main(Unchecked_Demo.java:8)
```

Error:

The other branch is topped by **Error**, which defines exceptions that are not expected to be caught under normal circumstances by the program. Exceptions of type **Error** are used by the Java run-time system to indicate errors having to do with the run-time environment, itself. **Stack overflow** is an example of such an error.

Examples:

1) Scenario where ArithmeticException occurs

If we divide any number by zero, there occurs an ArithmeticException.

```
int a=50/0;//ArithmeticException
```

2) Scenario where NullPointerException occurs

If we have null value in any variable, performing any operation by the variable occurs an NullPointerException.

```
String s=null;
System.out.println(s.length());//NullPointerException
```

3) Scenario where NumberFormatException occurs

The wrong formatting of any value may occur NumberFormatException. Suppose I have a string variable that have characters, converting this variable into digit will occur NumberFormatException.

```
String s="abc";
int i=Integer.parseInt(s);//NumberFormatException
```

4) ArrayIndexOutOfBoundsException

```
int num[]={ 1,2,3,4};
System.out.println(num[5]); // ArrayIndexOutOfBoundsException
```

- 5) FileNotFoundException-** signals that a file could not be found.
IOException- produced by failed or interrupted I/O operations

```

import java.io.File;
//import java.io.IOException;
import java.io.FileReader;

public class FileNotFound
{
    public static void main(String args[])//throws IOException
    {
        File f=new File("E://file.txt");
        FileReader fr = new FileReader(f);//FileNotFoundException
        fr.read(); //IOException

    }
}

```

- ➔ Both FileNotFound & IOException are checked exceptions, when you used throws IOException those exceptions will ignore at compile time but they need to be handled at run time.

Example program which covers all topics:

```

import java.io.*;
import java.util.*;
import java.io.IOException;

class InvalidNumber extends Exception
{
    public String toString()
    {
        return ("number must be < 100, you have entered >= 100");
    }
}

class Exceptions
{
    public static void main(String args[]) throws IOException
    {
        Scanner s = new Scanner(System.in);
        int a,b;
        a=s.nextInt();
        b=s.nextInt();

        try
        {
            a =a/b;
            if(a>100)

```



```

        throw new InvalidNumber();
    }
    catch(ArithmeticException e)
    {
        System.out.println("Arithmetic- exception");
    }
    catch(InvalidNumber e)
    {
        System.out.println("Invalid Number ");
    }

    finally
    {
        System.out.println("finally is always executed.");
    }

    System.out.println("Outside Try & Catch Block");
}

```

OUTPUT:

10

0

Arithmetic- exception

Outside Try & Catch Block

MultiThreading

“**Multithreading in java** is a process of executing multiple threads simultaneously.”

Multiprocessing and multithreading, both are used to achieve multitasking.

Multitasking

Multitasking is a process of executing multiple tasks simultaneously. Multitasking can be achieved by two ways:

- Process-based Multitasking(Multiprocessing)
- Thread-based Multitasking(Multithreading)

S.NO	Process	Thread
1	A program in execution is often referred as process.	<i>A thread is a path of execution in a program. A thread is a subset (part) of the process.</i>
2	Multi-processing is the feature that allows to run two or more programs concurrently	Multi-threading is a process of executing multiple threads(different parts of a program/process) simultaneously
3	Process is heavyweight	Thread is a lightweight process.
4	Each process has its own address in memory i.e. each process allocates separate memory area .	Threads share the same address space .
5	Interprocess communication is expensive and limited	Interthread communication is inexpensive .
6	Context switching from one process to another is high cost .	Context switching from one thread to other is low cost .
7	A process does not have control over the sibling process, it has control over its child processes only.	Threads have control over the other threads of the same process

Advantages of Java Multithreading

- Multithreading enables to write very efficient programs that make **maximum use of the CPU**.
- It can perform many operations together so **it saves time**.
- Threads are **independent** so it doesn't affect other threads if exception occurs in a single thread.

- It keeps CPU idle time to minimum.

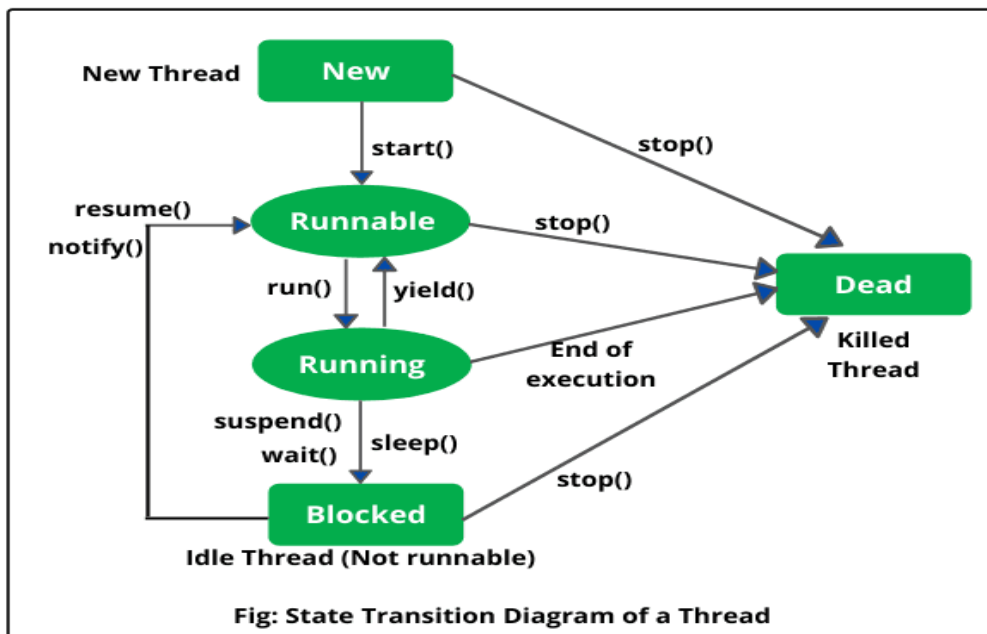
Thread Life cycle (or) Thread States :

A thread goes through various stages in its life cycle.

Thread has following states:

- New
- Runnable
- Running
- Waiting
- Dead (Terminated)

Following diagram shows complete life cycle of a thread



1. New state:

- A new thread begins its life cycle in the new state.
- It remains in this state until the program starts the thread (before calling the start() method).
- It is also referred to as a born thread.
- At this point, the thread is considered not alive.

2. Runnable state:

- Runnable state is also called ready to run stage also called queue.
- A thread start its life from Runnable state.

- A thread first enters runnable state after the invoking of start() method.

3. **Running state:**

- A thread is in running state that means the thread is currently executing.
- A thread enters into a running state by calling run() method.

4. **Waiting (Blocked):**

- A thread can enter in to a waiting state when a thread is waiting for the resources that are hold by another thread.
- A thread enters into a waiting state by calling either sleep() or wait() method.

5. **Dead (Terminate) –**

- A thread can be considered dead when its run() method completes.
- Once a Thread reached dead state it cannot run again.

The Thread Class:

Java's multithreading system is built upon the **Thread** class.

Thread encapsulates a thread of execution.

Thread class provide constructors and methods to create and perform operations on a thread.

Commonly used methods of Thread class:

1. **public void run():** is used to perform action for a thread.
2. **public void start():** starts the execution of the thread.JVM calls the run() method on the thread.
3. **public void sleep(long milliseconds):** The sleep() method is used to sleep(suspend) a thread for the specified amount of time. It throws an InterruptedException
4. **public void join():** waits for a thread to die.
5. **public void join(long milliseconds):** waits for a thread to die for the specified milliseconds.
6. **public int getPriority():** returns the priority of the thread.
7. **public int setPriority(int priority):** changes the priority of the thread.
8. **public String getName():** returns the name of the thread.
9. **public void setName(String name):** changes the name of the thread.
10. **public Thread currentThread():** returns the reference of currently executing thread.

11. **public int getId():** returns the id of the thread.
12. **public Thread.State getState():** returns the state of the thread.
13. **public boolean isAlive():** tests if the thread is alive.
14. **public void yield():** causes the currently executing thread object to temporarily pause and allow other threads to execute.
15. **public void suspend():** is used to suspend the thread(deprecated).
16. **public void resume():** is used to resume the suspended thread(deprecated).
17. **public void stop():** is used to stop the thread(deprecated).
18. **public boolean isDaemon():** tests if the thread is a daemon thread.
19. **public void setDaemon(boolean b):** marks the thread as daemon or user thread.
20. **public void interrupt():** interrupts the thread.
21. **public boolean isInterrupted():** tests if the thread has been interrupted.
22. **public static boolean interrupted():** tests if the current thread has been interrupted.

Commonly used Constructors of Thread class:

- Thread()
- Thread(String name)
- Thread(Runnable r)
- Thread(Runnable r,String name)

Thread Creation:

There are two ways to create a thread:

1. By extending Thread class
2. By implementing Runnable interface.

1. By extending Thread class

- One way to create a thread is to create a new class that extends **Thread**, and then to create an instance of that class.
- The extending class must override the **run()** method, which is the entry point for the new thread.

public void run()

- It has to call **start()** to begin execution of the new thread.

Example: program to extend **Thread**

```

class A extends Thread
{
    public void run()
    {
        System.out.println("thread is running...");
    }
    public static void main(String args[])
    {
        A t=new A();
        t.start();
    }
}

```

Output: thread is running...

2. By implementing Runnable interface.

The second way to create a thread is to create a class that implements the **Runnable** interface. Following are the steps to create a thread by implementing Runnable interface.

- Define a class that implements Runnable interface
- Implement the method run()
public void run()
- Create a thread by passing an object of this runnable class to the Thread class constructor
- Call the thread's start() method to run the thread.

Example:

```

Class A implements Runnable
{
    public void run()
    {
        System.out.println("thread is running...");
    }

    public static void main(String args[])
    {
        A a=new A();
        Thread t=new Thread(a);
        t.start();
    }
}

```

Output:thread is running...

Create multiple threads

The program to create multiple threads using Extending Thread class:

```

class A extends Thread
{
    public void run()
    {
        try
        {
            for(int i=1;i<=3;i++)
            {
                System.out.println("A - "+i);
                Thread.sleep(1000); // thread to sleep for 1 second
            }
        }
        catch(InterruptedException ie)
        {
        }
    }
}

class B extends Thread
{
    public void run()
    {
        try
        {
            for(int i=1;i<=3;i++)
            {
                System.out.println("B - "+i);
                Thread.sleep(2000); // thread to sleep for 2 seconds
            }
        }
        catch(InterruptedException ie){ }
    }
}

class Thread1
{
    public static void main(String args[])
    {
        A t1=new A();
        B t2=new B();
        t1.start();
        t2.start();
    }
}

```

Output:

isAlive()

isAlive() is used to determine whether a thread has finished or not.

Syntax: **final boolean isAlive()**

The **isAlive()** method returns **true** if the thread upon which it is called is still running. It returns **false** otherwise.

join()

The **join()** method waits for a thread to die. In other words, it causes the currently running threads to stop executing until the thread it joins with completes its task.

Syntax:

```
public void join() throws InterruptedException
public void join(long milliseconds) throws InterruptedException
```

Thread Priorities

- Thread priorities are used by the thread scheduler to decide when each thread should be allowed to run.
- Each thread has priority. In theory, higher-priority threads get more CPU time than lower-priority threads.
- Priorities are represented by a number between 1 and 10.
- In most cases, thread scheduler schedules the threads according to their priority (known as preemptive scheduling).

3 constants defined in Thread class:

1. `public static int MIN_PRIORITY`
2. `public static int NORM_PRIORITY`
3. `public static int MAX_PRIORITY`

Default priority of a thread is 5 (`NORM_PRIORITY`). The value of `MIN_PRIORITY` is 1 and the value of `MAX_PRIORITY` is 10.

setPriority() :

setPriority() method is used to set the priority level for a thread.

Syntax: **final void setPriority(int *level*)**

Here, *level* specifies the new priority setting for the calling thread. The value of *level* must be within the range **MIN_PRIORITY** and **MAX_PRIORITY**. Currently, these values are 1 and 10, respectively.

getPriority():

getPriority() method is used to obtain the Current priority of the thread.

Syntax: **final int getPriority()**

```
class A extends Thread
{
    public void run()
    {
        System.out.println(" thread name :"+Thread.currentThread().getName());
        System.out.println(" thread priority :"+Thread.currentThread().getPriority());
    }
    public static void main(String args[])
    {
        A a1=new A();
        A a2=new A();

        a1.setPriority(Thread.MIN_PRIORITY);
        a2.setPriority(Thread.MAX_PRIORITY);
        a1.start();
        a2.start();
    }
}
```

Output:

```
thread name:Thread-0
thread priority:10
thread name:Thread-1
thread priority:1
```

Synchronization in Java

- Synchronization in java is used *to control the access of multiple threads to any shared resource*.
- Java Synchronization is used to allow *only one thread to access the shared resource*.

Concept of Lock in Java

- Synchronization is based on the entity known as the lock or monitor. Every object has a lock associated with it.
- A thread that needs to access an object (resource) has to acquire the object's lock before accessing them. It releases the lock when complete it's work with the acquired resource.

The synchronization is mainly used to

1. To prevent thread interference.
2. To prevent consistency problem.

Synchronization can be done by two ways:

1. synchronized block
2. synchronized method

synchronized block

- Synchronized block can be used to perform synchronization on any specific resource of the method.
- Suppose you have 50 lines of code in your method, but you want to synchronize only 5 lines, you can use synchronized block.
- Scope of synchronized block is smaller than the method.

Syntax for synchronized block

```
synchronized (object )  
{  
    //code block  
}
```

synchronized method

- If you declare any method as synchronized, it is known as synchronized method.
- Synchronized method is used to lock an object for any shared resource.
- When a thread invokes a synchronized method, it automatically acquires the lock for that object and releases it when the thread completes its task.

//example of java synchronized method

Inter-thread communication in Java

Inter-thread communication or **Co-operation** is all about allowing synchronized threads to communicate with each other.

Cooperation (Inter-thread communication) is a mechanism in which a thread is paused running in its critical section and another thread is allowed to enter (or lock) in the same critical section to be executed. It is implemented by following methods of **Object class**:

- wait()
- notify()
- notifyAll()

wait():

This method tells the calling thread to give up the monitor and go to sleep until some other thread enters the same monitor and calls notify()

Method	Description
<code>public final void wait() throws InterruptedException</code>	waits until object is notified.
<code>public final void wait(long timeout) throws InterruptedException</code>	waits for the specified amount of time.

notify():

This method wakes up the first thread that called wait() on the same object.

public final void notify()

notifyAll():

This method wakes up all the threads that called wait() on the same object. The highest priority thread will run first.

public final void notifyAll()

Example:

```

class Customer
{
    int amount=10000;

    synchronized void withdraw(int amount)
    {
        System.out.println("going to withdraw...");

        if(this.amount<amount)
        {
            System.out.println("Less balance; waiting for deposit...");
            try
            {
                wait();
            }
            catch(Exception e){ }
        }
        this.amount-=amount;
        System.out.println("withdraw completed...");
    }

    synchronized void deposit(int amount)
    {
        System.out.println("going to deposit...");
        this.amount+=amount;
        System.out.println("deposit completed... ");
        notify();
    }
}

```

```

    }
}

class Test
{
    public static void main(String args[])
    {
        final Customer c=new Customer();

        new Thread()
        {
            public void run()
            {
                c.withdraw(15000);
            }
        }.start();

        new Thread()
        {
            public void run()
            {
                c.deposit(10000);
            }
        }.start();
    }
}

```

Daemon Thread in Java

- **Daemon thread in java** is a service provider thread that provides services to the user thread.
- Its life depend on user threads i.e. when all the user threads dies, JVM terminates this thread automatically.
- It is a low priority thread.
- There are many java daemon threads running automatically e.g. gc, finalizer etc.

The java.lang.Thread class provides two methods for java daemon thread.

No.	Method	Description
1)	public void setDaemon(boolean status)	is used to mark the current thread as daemon thread or user thread.
2)	public boolean isDaemon()	is used to check that current is daemon.

Simple example of Daemon thread in java

File: MyThread.java

```
public class TestDaemonThread1 extends Thread
```

```

{
    public void run()
    {
        if(Thread.currentThread().isDaemon())
            { //checking for daemon thread
                System.out.println("daemon thread work");
            }
        else
        {
            System.out.println("user thread work");
        }
    }
    public static void main(String[] args)
    {
        TestDaemonThread1 t1=new TestDaemonThread1();//creating thread
        TestDaemonThread1 t2=new TestDaemonThread1();
        TestDaemonThread1 t3=new TestDaemonThread1();
        t1.setDaemon(true); //now t1 is daemon thread
        t1.start(); //starting threads
        t2.start();
        t3.start();
    }
}

```

Output

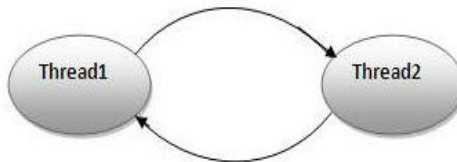
```

daemon thread work
user thread work
user thread work

```

Deadlock in java

Deadlock in java is a part of multithreading. Deadlock can occur in a situation when a thread is waiting for an object lock, that is acquired by another thread and second thread is waiting for an object lock that is acquired by first thread. Since, both threads are waiting for each other to release the lock, the condition is called deadlock.



Example of Deadlock in java

```

public class TestDeadlockExample1
{
    public static void main(String[] args)
    {

```

```

        final String resource1 = "ratan jaiswal";
        final String resource2 = "vimal jaiswal";
        // t1 tries to lock resource1 then resource2
        Thread t1 = new Thread()
        {
            public void run()
            {
                synchronized (resource1)
                {
                    System.out.println("Thread 1: locked resource 1");

                    try { Thread.sleep(100);} catch (Exception e) {}

                    synchronized (resource2)
                    {
                        System.out.println("Thread 1: locked resource 2");
                    }
                }
            }
        };

        // t2 tries to lock resource2 then resource1
        Thread t2 = new Thread()
        {
            public void run()
            {
                synchronized (resource2)
                {
                    System.out.println("Thread 2: locked resource 2");

                    try { Thread.sleep(100);} catch (Exception e) {}

                    synchronized (resource1)
                    {
                        System.out.println("Thread 2: locked resource 1");
                    }
                }
            }
        };

        t1.start();
        t2.start();
    }
}

```

Output: Thread 1: locked resource 1
 Thread 2: locked resource 2

ThreadGroup in Java

Java provides a convenient way to group multiple threads in a single object. In such way, we can suspend, resume or interrupt group of threads by a single method call.

Java thread group is implemented by *java.lang.ThreadGroup* class.

There are only two constructors of ThreadGroup class.

No.	Constructor	Description
1)	ThreadGroup(String name)	creates a thread group with given name.
2)	ThreadGroup(ThreadGroup parent, String name)	creates a thread group with given parent group and name.

ThreadGroup Example

```
public class ThreadGroupDemo implements Runnable
{
    public void run()
    {
        System.out.println(Thread.currentThread().getName());
    }
    public static void main(String[] args)
    {
        ThreadGroupDemo runnable = new ThreadGroupDemo();
        ThreadGroup tg1 = new ThreadGroup("Parent ThreadGroup");

        Thread t1 = new Thread(tg1, runnable, "one");
        t1.start();
        Thread t2 = new Thread(tg1, runnable, "two");
        t2.start();
        Thread t3 = new Thread(tg1, runnable, "three");
        t3.start();

        System.out.println("Thread Group Name: "+tg1.getName());
        tg1.list();
    }
}
```

Output:

one
two
three

Thread Group Name: Parent ThreadGroup

java.lang.ThreadGroup[name=Parent ThreadGroup,maxpri=10]

Thread[one,5,Parent ThreadGroup]

Thread[two,5,Parent ThreadGroup]

Thread[three,5,Parent ThreadGroup]

UNIT – 5

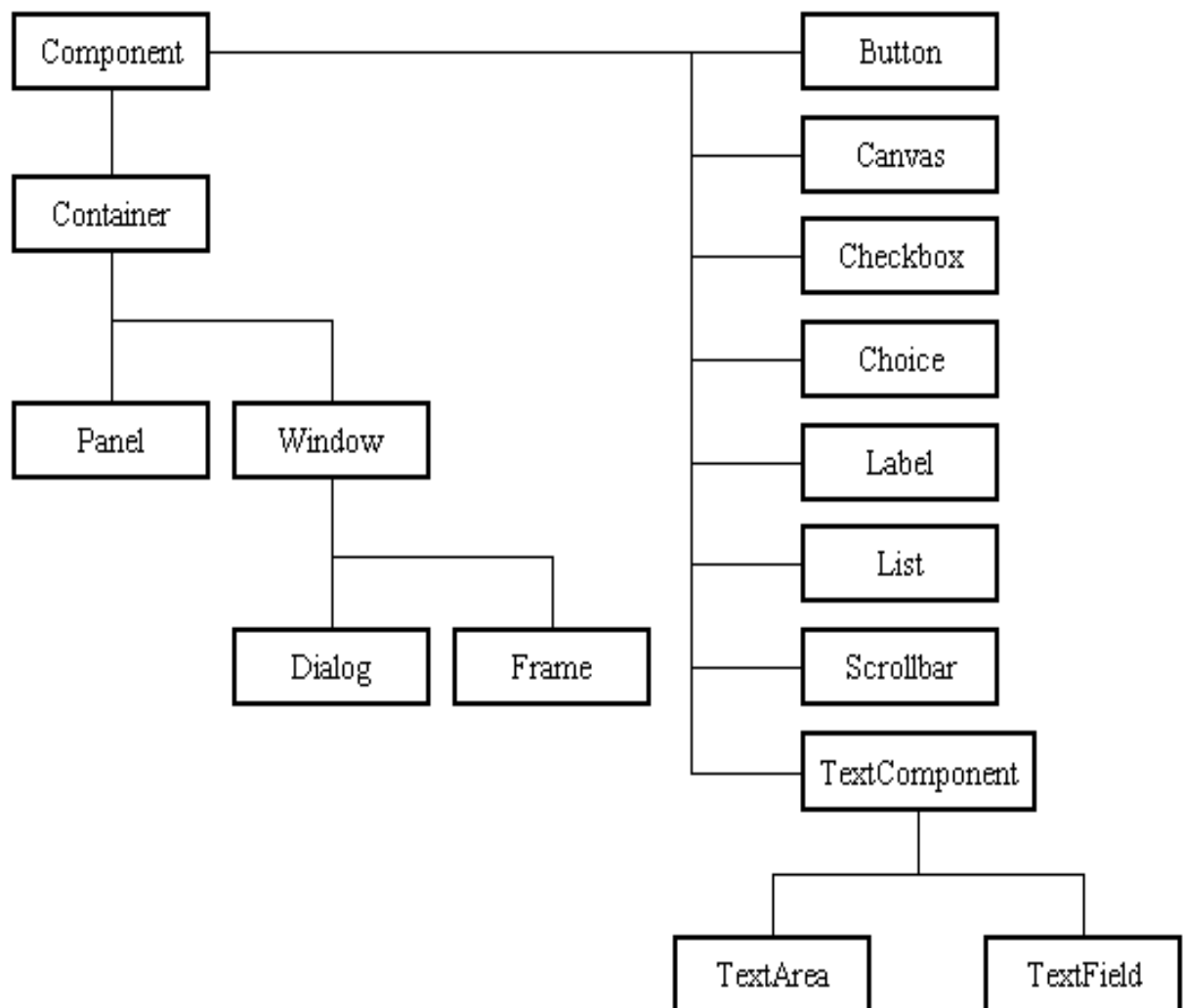
Introduction to AWT (Abstract Window Toolkit)

Java AWT (Abstract Windowing Toolkit) is an API to develop GUI or window-based application in java.

Window Fundamentals:

- The AWT defines windows according to a class hierarchy that adds functionality and specificity with each level.
- The 2 most common windows are :
 1. Those derived from Panel, which is used by **applets**.
 2. Those derived from **Frame**, which creates a standard window.

class hierarchy for Panel and Frame :



Component:

- Component class is at the top most class of the AWT hierarchy.
- A component is an object having graphical representation that can be displayed on the screen and interact with the user.
Examples of components are buttons, checkboxes, label etc.
- All user interface elements that are displayed on the screen and that interact with the user are subclasses of Component.
- Anything that is derived from the component class is can be a component.
- The class contains no of methods for event handling, managing graphics painting and sizing and resizing the window.

Container:

- Container class is a subclass of Component.
- It has additional methods that allow other Component objects to be nested within it.
- A container object is component that can contain other AWT components.
- Some of the containers are Frames, Dialogs, windows, panel(applet)
- Responsibility of a container: To layout (that is, positioning) any components that it contains.

Panel:

- The Panel class is a concrete subclass of Container.
- It doesn't add any new methods; it simply implements Container.
- A panel provides space in which an application can attach any other component, including other panels
- It is a concrete screen component.
- Applet class is the sub class of panel
- Applet's output is drawn on the surface of a Panel object.
- In essence, a Panel is a window that does not contain a title bar, menu bar, or border.
- This is why we don't see these items when an applet is run inside a browser.
- When we run an applet using an appletviewer, the applet viewer provides the title and border.
- Other components can be added to a Panel object by its add() method (inherited from Container).
- Once these components have been added, we can position and resize them manually using the setLocation(), setSize(), or setBounds() methods defined by Component.

Window:

- The Window class creates a top-level window.
- A window objects is a top-level window with no borders and no menu bar.
- It doesn't contain any components.
- It has two subclasses; Frame is window with name and menu bar and Dialog to represent dialog boxes.

Frame:

- It is a subclass of Window and has a title bar, menu bar, borders, and resizing corners.

2 forms of Frame constructors:

1. Frame() : It creates a standard window that does not contain a title.

```
Frame f=new Frame();
```

2. Frame(String *title*) : It creates a window with the title specified by *title*.

```
Frame f=new Frame("my frame");
```

Note: we cannot specify the dimensions of the window. Instead, we must set the size of the window after it has been created.

- Another way is to create a subclass myframe to the Frame class and create an object to the sub class

Ex

```
class Myframe extends frame
{
    Myframe m=new Myframe();
}
```

Methods in Frames :

Setting the Window's Dimensions :

- setSize() : used to set the dimensions of the window.

Syntax : void setSize(int *newWidth*, int *newHeight*)

void setSize(Dimension *newSize*)

where, new size of window is specified by *newWidth* and *newHeight* or by the width and height fields of the Dimension object passed in *newSize*. (dimensions → pixels).

- getSize() : used to obtain the current size of a window.

Syntax : Dimension getSize()

Note : It returns the current size of the window contained within the width and height fields of a Dimension object.

Hiding & Showing a Window :

- setVisible() : After a frame window has been created, it will not be visible until we call setVisible()

Syntax : void setVisible(boolean *visibleFlag*)

Note : The component is visible if the argument to this method is true. Otherwise, it is hidden.

Setting a Window's Title :

- **setTitle()** : We can change the title in a frame window using setTitle()

Syntax : void setTitle(String *newTitle*)

Here, *newTitle* is the new title for the window.

AWT Controls

Labels :

- The easiest control to use is a label.
- A *label* is an object of type **Label**, and it contains a string, which it displays.
- Labels are passive controls that do not support any interaction with the user.

Label defines the following constructors:

```
Label( )  
Label(String str)  
Label(String str, int how)
```

- The first version creates a blank label.
- The second version creates a label that contains the string specified by *str*. This string is left-justified.
- The third version creates a label that contains the string specified by *str* using the alignment specified by *how*. The value of *how* must be one of these three constants: **Label.LEFT**, **Label.RIGHT**, or **Label.CENTER**.

setText() : It is used to set or change the text in a label .

syntax: **void setText(String *str*)**

getText(): It is used to obtain the current label.

syntax: **String getText()**

For **setText()** , *str* specifies the new label. For **getText()** , the current label is returned.

The following example creates three labels and adds them to an applet window:

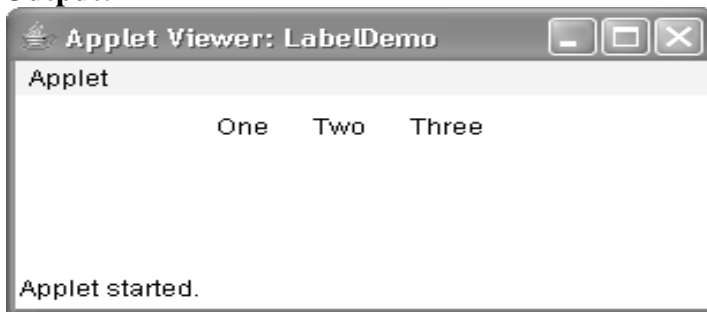
```
// Demonstrate Labels  
import java.awt.*;  
import java.applet.*;  
/*  
    <applet code="LabelDemo" width=300 height=200>  
    </applet>  
*/
```

```

public class LabelDemo extends Applet
{
    public void init()
    {
        Label l1 = new Label("One");
        Label l2 = new Label("Two");
        Label l3 = new Label("Three");
        // add labels to applet window
        add(l1);
        add(l2);
        add(l3);
    }
}

```

Output:



Buttons:

- A **Button** is a component that contains a label and that generates an event when it is pressed.
- buttons are objects of type **Button**.

Button defines these two constructors:

```

Button( )
Button(String str)

```

- The first version creates an empty button.
- The second creates a button that contains *str* as a label.

setLabel():

It is used to set label to button

```
void setLabel(String str)
```

getLabel():

It is used to get the label on the button

```
String getLabel( )
```

Here, *str* becomes the new label for the button.

Handling Buttons

- Each time a button is pressed, an action event is generated.
- This is sent to any **ActionListener** that previously registered to receive action event notifications from that component.
- **ActionListener** interface defines the **actionPerformed()** method, which is called when an event occurs.
- An **ActionEvent** object is supplied as the argument to this method

Programcode:

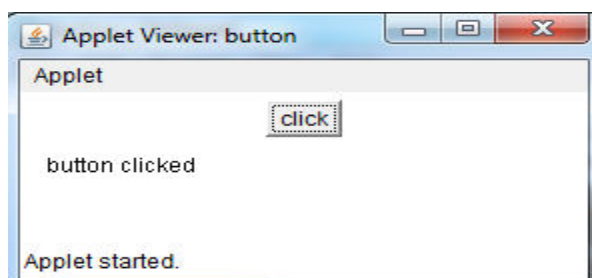
```
import java.awt.*;
import java.applet.*;
import java.awt.event.*;
/*
<applet code=" button " width=500 height=500>
</applet>
*/

public class button extends Applet implements ActionListener
{
    String msg= " ";
    Button b;

    public void init()
    {
        b=new Button("click");
        add(b);
        b.addActionListener(this);
    }

    public void actionPerformed(ActionEvent ae)
    {
        msg+= " button clicked ";
        repaint();
    }

    public void paint(Graphics g)
    {
        g.drawString(msg, 10, 100);
    }
}
```



Check Boxes

- A *check box* is a control that is used to turn an option on or off.
- It consists of a small box that can either contain a check mark or not.
- There is a label associated with each check box that describes what option the box represents. You change the state of a check box by clicking on it.
- Check boxes can be used individually or as part of a group.
- Check boxes are objects of the **Checkbox** class.

Checkbox supports these constructors:

```
Checkbox()  
Checkbox(String str)  
Checkbox(String str, boolean on)  
Checkbox(String str, boolean on, CheckboxGroup cbGroup)  
Checkbox(String str, CheckboxGroup cbGroup, boolean on)
```

- The first form creates a check box whose label is initially blank. The state of the check box is unchecked.
- The second form creates a check box whose label is specified by *str*. The state of the check box is unchecked.
- The third form allows you to set the initial state of the check box. If *on* is **true**, the check box is initially checked; otherwise, it is cleared.
- The fourth and fifth forms create a check box whose label is specified by *str* and whose group is specified by *cbGroup*.

Methods:

getState() : To retrieve the current state of a check box.

setState() : To set its state.

getLabel() : we can obtain current label associated with a check box

setLabel() : To set the label

Syntax :

```
boolean getState()  
void setState(boolean on)  
String getLabel()  
void setLabel(String str)
```

here,

on → **true**, the box is checked. on → **false**, the box is cleared.

str → The string passed in *str* becomes the new label associated with the invoking check box.

Handling Check Boxes :

- Each time a check box is selected or deselected, an item event is generated.
- This is sent to **ItemListener** that previously registered to receive item event notifications from that component.
- **ItemListener** interface defines the **itemStateChanged()** method.

- An **ItemEvent** object is supplied as the argument to this method. It contains information about the event

Example program:

```
import java.awt.*;
import java.applet.*;
import java.awt.event.*;
/*
<applet code="check1" width=1000 height=1000>
</applet>
*/

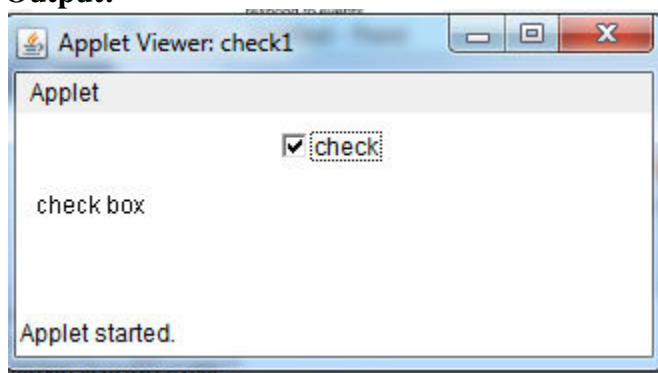
public class check1 extends Applet implements ItemListener
{
    String msg;
    Checkbox c;

    public void init()
    {
        c=new Checkbox("check");
        add(c);
        c.addItemListener(this);
    }

    public void itemStateChanged(ItemEvent ie)
    {
        msg ="check box";
    }

    public void paint(Graphics g)
    {
        g.drawString(msg,10,50);
    }
}
```

Output:



CheckboxGroup: Radio buttons

- It is possible to create a set of mutually exclusive check boxes in which one and only one check box in the group can be checked at any one time. These check boxes are often called *radio buttons*.
- To create a set of mutually exclusive check boxes, we must first define the group to which they will belong and then specify that group when we construct the check boxes.
- Check box groups are objects of type CheckboxGroup.

```
CheckboxGroup cbg=new CheckboxGroup();
```

getSelectedCheckbox() : It determines which check box in a group is currently selected.

setSelectedCheckbox() : It sets a check box.

Syntax :

```
Checkbox getSelectedCheckbox()
```

```
void setSelectedCheckbox(Checkbox which)
```

here,

which → check box that we want to be selected.

Example Program:

```
import java.awt.*;
import java.applet.*;
import java.awt.event.*;
/*
<applet code="checkgroup" width=1000 height=1000>
</applet>
*/
public class checkgroup extends Applet implements ItemListener
{
    String msg,msg1,msg2;
    Checkbox m,f;
    CheckboxGroup cbg;

    public void init()
    {
        cbg=new CheckboxGroup();
        m=new Checkbox("Male",cbg,true);
        f=new Checkbox("Female",cbg,false);
        add(m);
        add(f);
        m.addItemListener(this);
        f.addItemListener(this);
    }
}
```

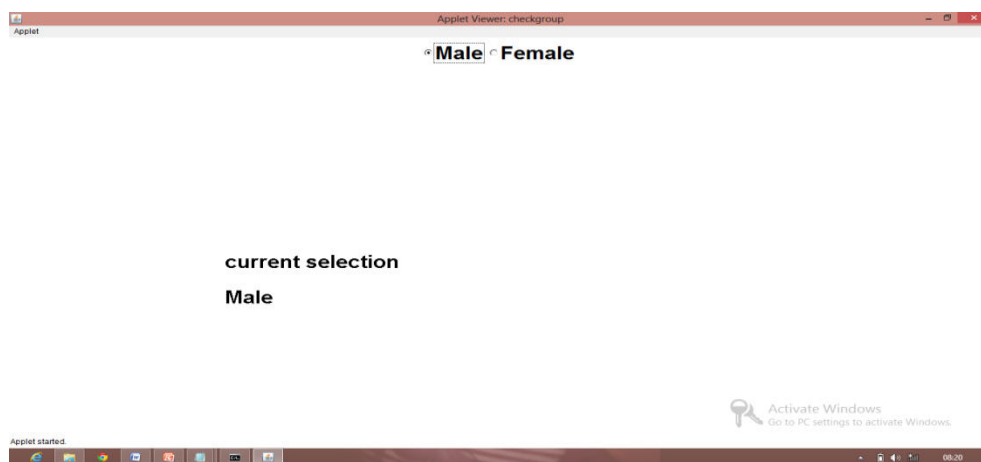


```

    public void itemStateChanged(ItemEvent ie)
    {
        repaint();
    }
    public void paint(Graphics g)
    {
        msg="current selection";
        g.drawString(msg,300,390);
        msg2=cbg.getSelectedCheckbox().getLabel();
        g.drawString(msg2,300,450);
    }
}

```

Output:



Choice Controls :

- The **Choice** class is used to create a *pop-up list* of items from which the user may choose.
- **Choice** control is a form of menu
- Choice defines only default constructor, which creates an empty list.

Syntax : **Choice()**

add() : Used to add item to the choice list.

Syntax : **void add(String name)**

here, *name* → name of item being added.

Note : Items are added to the list in the order in which calls to add() occur.

```
Choice c=new Choice();
```

```
c.add("CSE");
```

getSelectedItem() or getSelectedIndex() : To determine which item is currently selected

Syntax :

string getSelectedItem()

int getSelectedIndex()

getSelectedItem() - returns a string containing the name of the item.

getSelectedIndex() - returns the index of the item. The first item is at index 0.

getItemCount() : To obtain the number of items in the list

Syntax : **int getItemCount()**

getItem() : we can obtain the name associated with the item at that index.

Syntax : **String getItem(int index)**

here, *index* specifies the index of the desired item.

Handling Choice controls:

- Each time a choice is selected, an item event is generated. This is sent to **ItemListener** that previously registered to receive an item event notifications from that component.
- **ItemListener** interface defines the **itemStateChanged()** method.
- An **ItemEvent** object is supplied as the argument to this method. It contains information about the event

EXAMPLE PROGRAM:

```
import java.awt.*;
import java.applet.*;
import java.awt.event.*;
/*
<applet code=choice width=1000 height=1000>
</applet>
*/

public class choice extends Applet implements ItemListener
{
    String msg;
    Choice c;

    public void init()
    {
        Label l1=new Label("Branch");
        c = new Choice();

        c.add("CSE");
        c.add("ECE");
        c.add("EEE");

        add(l1);
        add(c);
    }
}
```

```

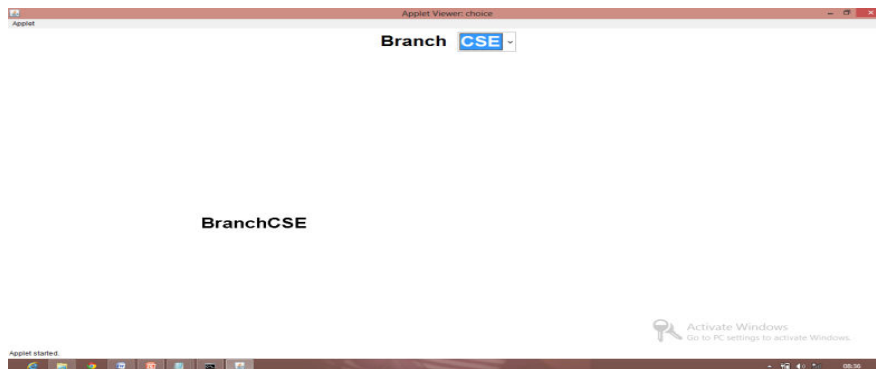
        c.addItemListener(this);
    }

    public void itemStateChanged(ItemEvent ie)
    {

        repaint();

    }
    public void paint(Graphics g)
    {
        msg="Branch"+branch.getSelectedItemAt();
        g.drawString(msg,300,420);
    }
}

```



List:

- The **List** class is used to create a pop-up list.
- **Choice** object, which shows only the single selected item in the menu,
- **List** object can be constructed to show any number of choices in the visible window
- From list it is possible to select more than one item.
- It provides multiple-choice, scrolling selection list.
- It can also be created to allow multiple selections.

List provides these constructors:

```

List( )
List(int numRows)
List(int numRows, boolean multipleSelect)

```

- The first version creates a **List** control that allows only one item to be selected at any one time.
- In the second form, the value of *numRows* specifies the number of entries in the list that will always be visible.
- In the third form, if *multipleSelect* is **true**, then the user may select two or more items at a time. If it is **false**, then only one item may be selected.

add(): It is used to add items to the list

Syntax: **void add(String *name*)**

void add(String *name*, int *index*)

Here, *name* is the name of the item added to the list. The first form adds items to the end of the list. The second form adds the item at the index specified by *index*. Indexing begins at zero.

getSelectedItem() :used to get the name of selected item.

getSelectedIndex():used to get the selected item index.

Syntax : **String getItem()**

int getSelectedIndex()

getSelectedItems() or getSelectedIndexes() :

Syntax :

 String[] getSelectedItems()

 int[] getSelectedIndexes()

getSelectedItems() - returns an array containing the names of the currently selected items.

getSelectedIndexes() - returns an array containing the indexes of the currently selected items.

getItemCount() : It obtain the number of items in the list

Syntax : **int getItemCount()**

getItem() : Given an index, we can obtain the name associated with the item at that index

Syntax : **String getItem(int *index*)**

here, *index* specifies the index of the desired item.

Handling Lists:

- Each time a item is selected in a list, an item event is generated. This is sent to **ItemListener** that previously registered to receive an item event notifications from that component.
- **ItemListener** interface defines the **itemStateChanged()** method.
- An **ItemEvent** object is supplied as the argument to this method. It contains information about the event

Example program:

```
import java.awt.*;
```

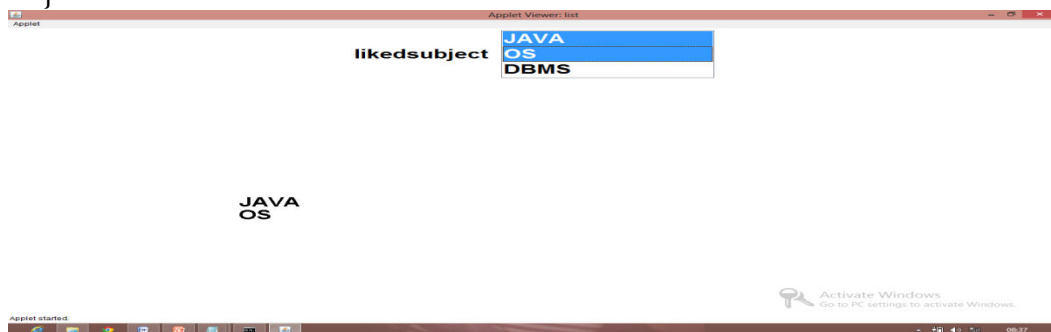
```

import java.applet.*;
import java.awt.event.*;
/*
<applet code=list width=1000 height=1000>
</applet>
*/
public class list extends Applet implements ItemListener
{
    String msg;
    List fav;
    public void init()
    {
        Label l1=new Label("likedsbjeet");
        fav=new List();
        fav.add("JAVA");
        fav.add("OS");
        fav.add("DBMS");
        add(l1);
        add(fav);
        fav.addItemListener(this);
    }

    public void itemStateChanged(ItemEvent ie)
    {
        repaint();
    }

    public void paint(Graphics g)
    {
        String idx[];
        msg="Current:";
        idx=fav.getSelectedItems();
        msg=idx[0];
        g.drawString(msg,300,420);
        msg=idx[1];
        g.drawString(msg,300,450);
        msg=idx[2];
        g.drawString(msg,300,480);
    }
}

```



TextFields :

- The **TextField** class implements a single-line text-entry area, usually called an *edit control*.
- Text fields allow the user to enter strings and to edit the text using the arrow keys, cut and paste keys, and mouse selections.
- **TextField** is a subclass of **TextComponent**.

TextField defines the following constructors:

TextField()

TextField(int *numChars*)

TextField(String *str*)

TextField(String *str*, int *numChars*)

- The first version creates a default text field.
- The second form creates a text field that is *numChars* characters wide.
- The third form initializes the text field with the string contained in *str*.
- The fourth form initializes a text field and sets its width.

Methods:

getText() : To obtain the string currently contained in the text field

setText() : To set the text.

Syntax :

String getText()

void setText(String *str*)

here, *str* is the new string.

select() : The user can select a portion of the text in a text field. Also, we can select a portion of text under program control

getSelectedText() : our program can obtain the currently selected text

Syntax :

String getSelectedText()

void select(int *startIndex*, int *endIndex*)

getSelectedText() returns the selected text.

The select() method selects the characters beginning at *startIndex* and ending at *endIndex*-1.

setEditable() : we can control whether the contents of a text field may be modified by the user or not.

isEditable() : we can determine editability

Syntax :

boolean isEditable()

void setEditable(boolean *canEdit*)

isEditable() returns true if the text may be changed and false if not.

In **setEditable()**, if *canEdit* is true, the text may be changed. If it is false, the text cannot be altered.

setEchoChar() :

- There may be times when we will want the user to enter text that is not displayed, such as a password. We can disable the echoing of the characters as they are typed.

- This method specifies a single character that the TextField will display when characters are entered (thus, the actual characters typed will not be shown and we can display like * or \$).

echoCharIsSet() : we can check a text field to see if it is in echoChar mode.

getEchoChar() : we can retrieve the echo character.

Syntax :

void setEchoChar(char *ch*)

boolean echoCharIsSet()

char getEchoChar()

here, *ch* specifies the character to be echoed.

Handling a TextField

- Since text fields perform their own editing functions, your program generally will not respond to individual key events that occur within a text field.
- However, you may want to respond when the user presses ENTER. When this occurs, an action event is generated.

TextArea :

- Sometimes a single line of text input is not enough for a given task. To handle these situations, the AWT includes a simple multiline editor that is textarea.
- In text area you may enter more than one line of text.

Constructors :

TextArea()

TextArea(int *numLines*, int *numChars*)

TextArea(String *str*)

TextArea(String *str*, int *numLines*, int *numChars*)

here, *numLines* → the height, in lines, of the text area,
numChars → its width, in characters.
str → Initial text.

It supports the `getText( )`, `setText( )`, `getSelectedText( )`, `select( )`, `isEditable( )`, and `setEditable( )` methods.

```
import java.awt.*;
import java.applet.*;
import java.awt.event.*;
/*
<applet code="textarea" width=1000 height=1000>
</applet>
*/
```

```
public class textarea extends Applet
{
    TextArea ta;
    String address;
    public void init()
    {

        ta=new TextArea(4,10);
        add(ta);
    }
    public void paint(Graphics g)
    {
        g.drawString("text area",300,360);
    }
}
```



Scroll bars :

- *Scroll bars* are used to select continuous values between a specified minimum and maximum.
- Scroll bars may be oriented horizontally or vertically.
- A scroll bar is actually a composite of several individual parts.
- Each end has an arrow that you can click to move the current value of the scroll bar one unit in the direction of the arrow.

- The current value of the scroll bar relative to its minimum and maximum values is indicated by the *slider box* (or *thumb*) for the scroll bar.
- The slider box can be dragged by the user to a new position.
- The scroll bar will then reflect this value.
- Scroll bars are encapsulated by the **Scrollbar** class.

Scrollbar defines the following constructors:

```
Scrollbar()  
Scrollbar(int style)  
Scrollbar(int style, int initialValue, int thumbSize, int min, int max)
```

The first form creates a vertical scroll bar.

- The second and third forms allow you to specify the orientation of the scroll bar.
- If *style* is **Scrollbar.VERTICAL**, a vertical scroll bar is created.
- If *style* is **Scrollbar.HORIZONTAL**, the scroll bar is horizontal.
- In the third form of the constructor, the initial value of the scroll bar is passed in *initialValue*. The number of units represented by the height of the thumb is passed in *thumbSize*. The minimum and maximum values for the scroll bar are specified by *min* and *max*

setValues() : If we construct a scroll bar by using one of the first two constructors, then we need to set its parameters

Syntax :

```
void setValues(int initialValue, int thumbSize, int min, int max)
```

getValue() : To obtain the current value of the scroll bar, returns the current setting.

setValue() : To set the current value

Syntax :

```
int getValue()  
void setValue(int newValue)
```

Here, *newValue* specifies the new value for the scroll bar.

Handling Scroll Bars :

- Each time a user interacts with a scroll bar, an `AdjustmentEvent` object is generated.
- This is sent to any `AdjustmentListener` that previously registered to receiving adjustment event notifications from that component.
- To process scroll bar events, we need to implement the `AdjustmentListener` interface..
- `AdjustmentListener` interface have `adjustmentValueChanged()` method.
- An `AdjustmentEvent` object is supplied as the argument to this method.

```

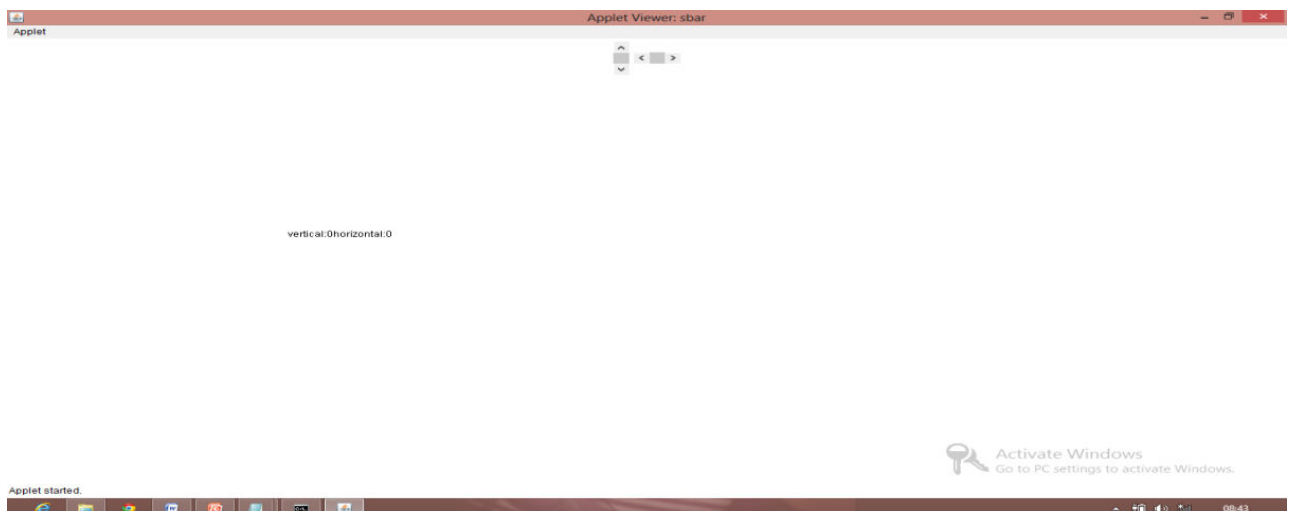
import java.awt.*;
import java.applet.*;
import java.awt.event.*;
/*
<applet code="sbar" width=1000 height=1000>
</applet>
*/

public class sbar extends Applet implements AdjustmentListener
{
    String msg="";
    Scrollbar vsb,hsb;

    public void init()
    {
        vsb=new Scrollbar(Scrollbar.VERTICAL,0,1,0,100);
        hsb=new Scrollbar(Scrollbar.HORIZONTAL,0,1,0,1000);
        add(vsb);
        add(hsb);
        vsb.addAdjustmentListener(this);
        hsb.addAdjustmentListener(this);
    }
    public void adjustmentValueChanged(AdjustmentEvent ae)
    {
        repaint();
    }

    public void paint(Graphics g)
    {
        msg="vertical:"+vsb.getValue();
        msg+="horizontal:"+hsb.getValue();
        g.drawString(msg,300,300);
    }
}

```



UNIT -V

Java Applet

Applet is a special type of program that is embedded in the webpage to generate the dynamic content.

It runs inside the browser and works at client side.

- An applet is a Java class that extends the `java.applet.Applet` class.
- An Applet class does not have any `main()` method.
- Applets are designed to be embedded within an HTML page.
- JVM creates an instance of the applet class and invokes **`init()`** method to initialize an Applet.

Advantage of Applet

There are many advantages of applet. They are as follows:

- It works at client side, so less response time.
- Secured
- It can be executed by browsers running under many platforms, including Linux, Windows, Mac Os etc.

How to run an Applet Program

An Applet program is compiled in the same way as you have been compiling your console programs. However there are two ways to run an applet.

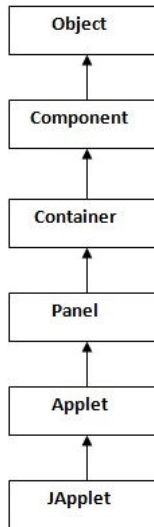
- Executing the Applet within **Java-compatible web browser**.
- **Using an Applet viewer**, such as the standard tool, **applet viewer**. An applet viewer executes your applet in a window.

Procedure using applet viewer:

Compilation: `javac filename.java`

Execution: `appletviewer filename.java`

Hierarchy of Applet



As displayed in the above diagram, Applet class extends Panel. Panel class extends Container which is the subclass of Component.

Applet class

Applet class provides all necessary support for applet execution, such as initializing and destroying of applet.

Applet provides several methods which are used to supports applet window based activities.

Some methods are shown below:

METHOD	DESCRIPTION
public void init()	init() is the first method to be called. It is used to initialize the Applet.
public void start()	It is automatically called after the init() method or browser is maximized.
public void stop()	It is used to stop the Applet. It is invoked when Applet is stop or browser is minimized.
public void paint()	It is used to paint the Applet. It is Invoked immediately after the start() method.
public void destroy()	destroy() method is called when your applet needs to be removed completely from memory.
boolean isActive()	Returns true if the applet has been started. It returns false if the applet has been stopped.
void showStatus(String str)	Displays str in the status window of the browser or applet viewer.

Life Cycle of an Applet(Applet skeleton)

There are 5 life cycle methods

1. `init()`
2. `start()`
3. `paint()`
4. `stop()`
5. `destroy()`

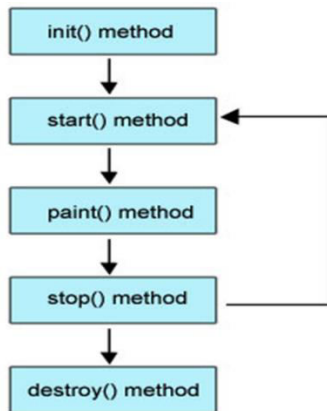


Figure: Life cycle of Applet

init():

- `init()` is the first method to be called.
- It is used to initialize the Applet(i.e., variables are initialized)
- This method is called only once during the runtime of applet.

Syntax: **`public void init();`**

start():

- It is automatically called after the `init()` method or browser is maximized.
- It is used to start the Applet.
- This method is called a number of times in the life cycle

Syntax: **`public void start();`**

stop():

- It is used to stop the Applet.
- It is invoked when Applet is stop or browser is minimized.
- This method is called a number of times in the life cycle

Syntax: **`public void stop();`**

destroy():

- It is used to destroy the Applet.
- destroy() method is called when your applet needs to be removed completely from memory.
- It is invoked only once.

Syntax: **public void destroy();**

paint():

- It is used to paint the Applet. It is Invoked immediately after the start() method
- It provides Graphics class object that can be used for drawing oval, rectangle, arc etc.
- This is called number of times in the execution.

Syntax: **public void paint(Graphics g);**

Example:

// An Applet skeleton.

```
import java.awt.*;
import java.applet.*;

/*
    <applet code="AppletSkel" width=300 height=100>
    </applet>
*/

public class AppletSkel extends Applet
{
    public void init()
    {
        // initialization called first
    }

    public void start()
    {
        // start or resume execution
    }

    public void stop()
    {
        // suspends execution
    }

    public void destroy()
    {
        // perform shutdown activities
    }
}
```

```

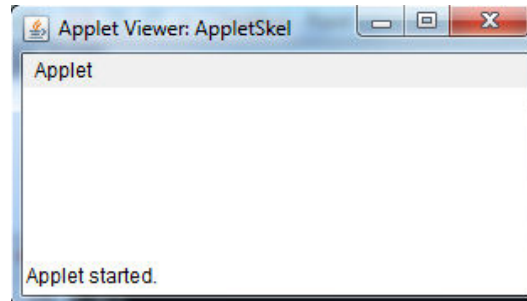
    public void paint(Graphics g)
    {
        // redisplay contents of window
    }
}

```

Compilation: javac AppletSkel.java

Execution: appletviewer AppletSkel.java

Output:



Simple Applet Display Methods

drawString():

- This method is used to output a string to an applet
- It is a member of the **Graphics** class.
- It is called from within either **update()** or **paint()**.
- The **drawString()** method will not recognize newline characters.

Syntax:

```
void drawString(String message, int x, int y);
```

Here, *message* is the string to be output to an applet.
x,y specifies the location in an applet window.

setBackground():

To set the background color of an applet's window

```
void setBackground(Color newColor);
```

setForeground():

To set the foreground color of an applet's window.

```
void setForeground(Color newColor)
```


Here, *newColor* specifies the new color. The class **Color** defines the constants shown here that can be used to specify colors:

Color.black	Color.magenta
Color.blue	Color.orange
Color.cyan	Color.pink
Color.darkGray	Color.red
Color.gray	Color.white
Color.green	Color.yellow
Color.lightGray	

Examples:

```
setBackground(Color.green);
setForeground(Color.red);
```

getBackground()

It is used to obtain the current background color of an applet's window

Color getBackground()

getForeground()

It is used to obtain the current foreground color of an applet's window

Color getForeground()

Example:

*/** A simple applet that sets the **foreground and background** colors and outputs a string. **/*

```
import java.awt.*;
import java.applet.*;
```

```
/*
    <applet code="Sample" width=300 height=50>
    </applet>
*/
```

```
public class Sample extends Applet
{
    String msg;

    public void init()
    {
        setBackground(Color.cyan);
        setForeground(Color.red);
        msg = "Inside init() --";
    }
}
```

// Initialize the string to be displayed.

```
public void start()
{
    msg += " Inside start( ) --";
}
```

// Display msg in applet window.

```
public void paint(Graphics g)
{
    msg += " Inside paint( ).";
    g.drawString(msg, 10, 30);
}
}
```

repaint():

The **paint()** method is called by the JVM implicitly in two circumstances. One is when the first time frame is created and displayed. The other is when the frame is resized (by dragging the frame border with mouse) by the user.

If the programmer would like to call the **paint()** method in the middle of the coding, it is not permitted to call **paint()** method directly .

For example, if an applet is displaying a moving banner, what mechanism does the applet use to update the window each time this banner scrolls?

Whenever your applet needs to update the information displayed in its window, it simply calls **repaint()**.

The repaint() method calls automatically update() method and in turn update() method calls paint() method.

repaint() → update() → paint()

The **update()** method job is to **erase** the earlier drawings on the frame. On the cleared surface, the **paint()** method draws afresh with the latest graphics.

Syntax:

void repaint()

This version causes the entire window to be repainted.

void repaint(int left, int top, int width, int height)

This version specifies a region that will be repainted

Here, the coordinates of the upper-left corner of the region are specified by *left* and *top*, and the width and height of the region are passed in *width* and *height*. These dimensions are specified in pixels.

A Simple Banner Applet

/* A simple banner applet. This applet creates a thread that scrolls the message contained in msg right to left across the applet's window. */

```
import java.awt.*;
import java.applet.*;
```

```
/*
    <applet code="SimpleBanner" width=300 height=50>
    </applet>
*/
```

```
public class SimpleBanner extends Applet implements Runnable
{
```

```
    String msg = " A Simple Moving Banner.";
    Thread t = null;
    boolean stopFlag;
```

```
    public void init()
    {
        setBackground(Color.cyan);
        setForeground(Color.red);
    }
```

```
    // Start thread
    public void start()
    {
        t = new Thread(this);
        stopFlag = false;
        t.start();
    }
```

```
    // Entry point for the thread that runs the banner.
```

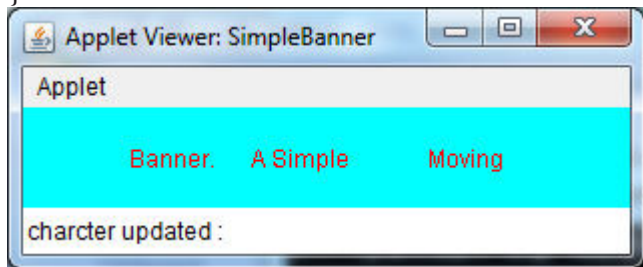
```
    public void run()
    {
        char ch;
        // Display banner
        for( ; ; )
        {
            try
            {
                repaint();
                Thread.sleep(10000);
            }
        }
    }
}
```

```

        ch = msg.charAt(0);
        msg = msg.substring(1);
        //create a substring from starting index 1 to end
        msg += ch;
        if(stopFlag)
            break;
    }
    catch(InterruptedException e) {}
}
}
// Pause the banner.
public void stop()
{
    stopFlag = true;
    t = null;
}

// Display the banner.
public void paint(Graphics g)
{
    g.drawString(msg, 50, 30);
}
}

```



showStatus

The **showStatus()** method is used to display the information in applet windows status bar of the browser or Appletviewer.

This is used for debugging, because it gives you an easy way to output message.

The status window is a good place to give the user feedback about what is occurring in the applet, suggest options, or possibly report some types of errors.

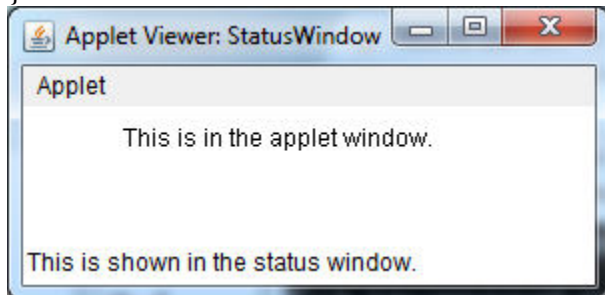
// Using the Status Window.

```

import java.awt.*;
import java.applet.*;
/*
    <applet code="StatusWindow" width=300 height=50>
    </applet>
*/

```

```
public class Status extends Applet
{
    // Display msg in applet window.
    public void paint(Graphics g)
    {
        g.drawString("This is in the applet window.", 10, 20);
        showStatus("This is shown in the status window.");
    }
}
```



Event Handling:

- Event handling is the mechanism that controls the event and decides what should happen if an event occurs.
- This mechanism has the code which is known as event handler that is executed when an event occurs.
- Java uses a Delegation Event Model to handle the events. This model defines the standard mechanism to generate and handle events.

Delegation Event Model :

- The modern approach to handling events is based on the *delegation event model*.
- It defines a standard and consistent mechanisms to generate and process events.
- The delegation event model has two parts: **sources** and **listeners**.
- The concept of Delegation Event Model is that “a *source* generates an event and sends it to one or more *listeners*. In this scheme, the listener simply waits until it receives an event. Once an event is received, the listener processes the event and then returns.
- **Advantage of this design :** The application logic that processes events is cleanly separated from the user interface logic that generates those events.
- In this model, listeners must register with a source in order to receive an event notification. So that notifications are sent only to listeners that want to receive them.

Events:

An **event** is an object that describes a “state change” in a source. Source is responsible for providing information of the occurred event to its handler.

- It can be generated as a consequence of a person interacting with the elements in a GUI (Graphical User Interface).
- Some of the activities that generate the events are pressing a button, entering a character via the keyboard, selecting an item in a list, and clicking the mouse.
- Events may also occur that are not directly caused by interactions with a user interface. For example, an event may be generated when a timer expires, a counter exceeds a value, a software or hardware failure occurs, or an operation is completed

Event Sources:

A **source** is an object that generates an event. A source is an object on which event occurs.

- It(event) occurs when the internal state of that(event source) object changes.
- Sources may generate more than one type of event.
- A “source” must register “listeners”. In order for the listeners to receive notifications about a specific type of event.
- Each type of event has its own registration method.

Syntax :

```
public void addTypeListener(TypeListener el);
```

Here, *Type* is the name of the event ,*el* is a reference to the event listener.

Example: the method that registers keyboard event listener is called **addKeyListener()**.

mouse motion event listener is called **addMouseMotionListener**

Event Source examples:

Event Source	Description
Button	Generates action events when the button is pressed.
Check box	Generates item events when the check box is selected or deselected.
Choice	Generates item events when the choice is changed.
List	Generates action events when an item is double-clicked; generates item events when an item is selected or deselected.
Menu Item	Generates action events when a menu item is selected; generates item events when a checkable menu item is selected or deselected.
Scroll bar	Generates adjustment events when the scroll bar is manipulated.
Text components	Generates text events when the user enters a character.
Window	Generates window events when a window is activated, closed, deactivated, deiconified, iconified, opened, or quit.

Event Listeners :

- Listener is an object that is notified when an event occurs. It is also known as event handler. It is responsible for generating response to an event.
- It has 2 major requirements.
 - 1) It must have been registered with one or more sources to receive notifications about specific types of events.
 - 2) It must implement methods to receive and process these notifications.

The methods that receive and process events are defined in a set of interfaces found in **java.awt.event**. For example, the **MouseMotionListener** interface defines two methods to receive notifications when the mouse is dragged or moved. Any object may receive and process one or both of these events if it provides an implementation of this interface.

Event Classes :

- The classes that represent events are at the core of Java's event handling mechanism.
- At the root of the Java event class hierarchy is **EventObject**, which is in **java.util**. It is the superclass for all events.
- Its one constructor is shown here:

EventObject(Object src);

Here, *src* is the object that generates this event.

- **EventObject** contains two methods: **getSource()** and **toString()**.

getSource(); It returns the source of the event.

Syntax : **Object getSource();**

toString(): It returns the string equivalent of the event.

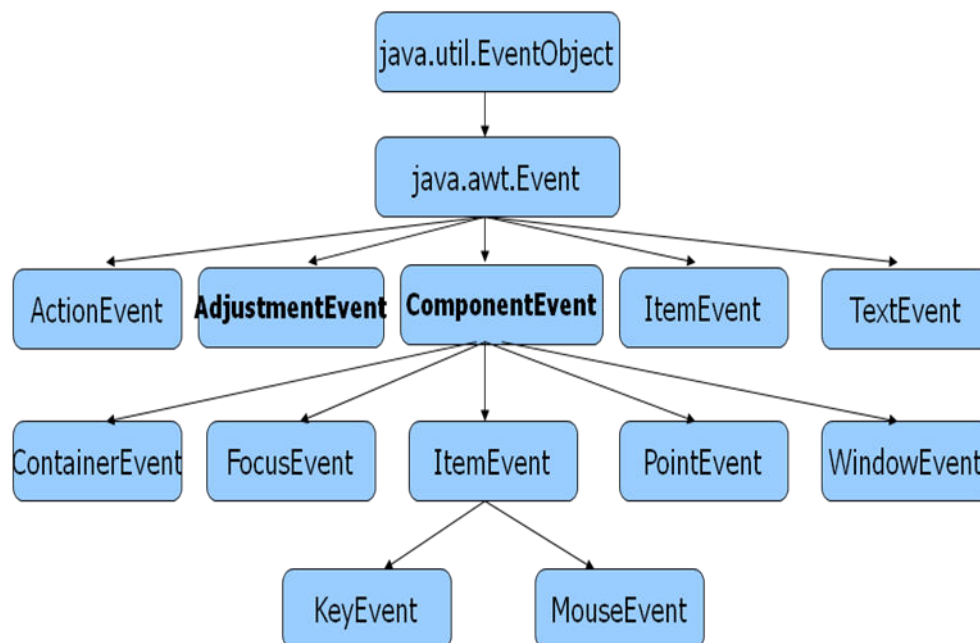
AWTEvent class:

The class **AWTEvent**, defined within the **java.awt** package, is a subclass of **EventObject**. It is the super class (either directly or indirectly) of all AWT-based events used by the delegation event model.

Its getId() method can be used to determine the type of the event.

Syntax: **int getId();**

Hierarchy of Event Classes :



Main Event Classes in java.awt.event:

Event Class	Description
ActionEvent	Generated when a button is pressed, a list item is double-clicked, or a menu item is selected.
AdjustmentEvent	Generated when a scroll bar is manipulated.
ComponentEvent	Generated when a component is hidden, moved, resized, or becomes visible.
ContainerEvent	Generated when a component is added to or removed from a container.
FocusEvent	Generated when a component gains or loses keyboard focus.
InputEvent	Abstract superclass for all component input event classes.
ItemEvent	Generated when a check box or list item is clicked; also occurs when a choice selection is made or a checkable menu item is selected or deselected.
KeyEvent	Generated when input is received from the keyboard.
MouseEvent	Generated when the mouse is dragged, moved, clicked, pressed, or released; also generated when the mouse enters or exits a component.
MouseWheelEvent	Generated when the mouse wheel is moved.
TextEvent	Generated when the value of a text area or text field is changed.
WindowEvent	Generated when a window is activated, closed, deactivated, deiconified, iconified, opened, or quit.

ActionEvent Class :

- An **ActionEvent** is generated when a button is pressed, a list item is double-clicked, or a menu item is selected.
- The **ActionEvent** class defines four integer constants that can be used to identify any modifiers associated with an action event: **ALT_MASK**, **CTRL_MASK**, **META_MASK**, and **SHIFT_MASK**.
- In addition, there is an integer constant, **ACTION_PERFORMED**, which can be used to identify action events.

ActionEvent has these three constructors:

ActionEvent(Object src, int type, String cmd)

ActionEvent(Object src, int type, String cmd, int modifiers)

ActionEvent(Object src, int type, String cmd, long when, int modifiers)

Here, *src* - reference to the object that generated this event

type - type of the event

cmd - its command string

modifiers - which modifier keys (ALT, CTRL, META, and/or SHIFT) were pressed when the event was generated.

when - when the event occurred

Methods:

getActionCommand(): It is used to get the label on the button when button is pressed.

Syntax: String getActionCommand()

For example, when a button is pressed, an action event is generated that has a command name equal to the label on that button.

getModifiers() :

method returns a value that indicates which modifier keys (ALT, CTRL, META, and/or SHIFT) were pressed when the event was generated.

Syntax : int getModifiers();

getWhen() :

It returns the time at which the event took place. This is called the event's *timestamp*.

Syntax: long getWhen();

Action Event Sources:

Button - Generates action events when the button is pressed.

List - Generates action events when an item is double-clicked

Menu Item- Generates action events when a menu item is selected

ActionListener Interface:

This interface defines the one method that is actionPerformed() method .

It is invoked when an action event occurs.

Syntax : void actionPerformed(ActionEvent ae)

AdjustmentEvent Class :

- Generated when a scroll bar is manipulated.
- There are 5 types of adjustment events.
- The AdjustmentEvent class defines integer constants that can be used to identify them.

The constants and their meanings are shown here:

BLOCK_DECREMENT	The user clicked inside the scroll bar to decrease its value.
BLOCK_INCREMENT	The user clicked inside the scroll bar to increase its value.
TRACK	The slider was dragged.
UNIT_DECREMENT	The button at the end of the scroll bar was clicked to decrease its value.
UNIT_INCREMENT	The button at the end of the scroll bar was clicked to increase its value.

NOTE :

In addition, there is an integer constant, ADJUSTMENT_VALUE_CHANGED,

that indicates that a change has occurred.

Constructor:

AdjustmentEvent(Adjustable <i>src</i>, int <i>id</i>, int <i>type</i>, int <i>data</i>)
--

Here,

src - reference to the object that generated this event.
id – equals **ADJUSTMENT_VALUE_CHANGED**.
type – type of the event
data - its associated data

Methods:

- **getAdjustable() :**
returns the object that generated the event.
Syntax:
Adjustable getAdjustable()
- **getAdjustmentType() :**
obtains the type of the adjustment event.
-returns one of the constants defined by AdjustmentEvent
Syntax :
int getAdjustmentType()
- **getValue() :**
-Obtains the amount of the adjustment.
Syntax :
int getValue()

AdjustmentListener:

adjustmentValueChanged() :

- This interface defines the adjustmentValueChanged() method that is invoked when an adjustment event occurs.

Syntax :

void adjustmentValueChanged(AdjustmentEvent *ae*)

Component Eventclass :

- A **ComponentEvent** is generated when the size, position, or visibility of a component is Changed, that is when a component is hidden, moved, resized, or becomes visible.
- There are 4 types of component events.
- The ComponentEvent class defines integer constants that can be used to identify them.

COMPONENT_HIDDEN	The component was hidden.
COMPONENT_MOVED	The component was moved.
COMPONENT_RESIZED	The component was resized.
COMPONENT_SHOWN	The component became visible.

NOTE: **ComponentEvent** is the superclass either directly or indirectly of **ContainerEvent**, **FocusEvent**, **KeyEvent**, **MouseEvent**, and **WindowEvent**.

- ComponentEvent class has one Constructor:

ComponentEvent(Component <i>src</i> , int <i>type</i>)

Here, *src* - reference to the object that generated this event.
type – type of the event

- ComponentEvent class has one **Method** :

getComponent() : returns the component that generated the event.

Syntax : Component getComponent()

ComponentListener:

- This interface defines 4 methods that are invoked when a component is resized, moved, shown, or hidden. Their general forms are shown here:

Syntax :

void componentResized(ComponentEvent *ce*)

void componentMoved(ComponentEvent *ce*)

void componentShown(ComponentEvent *ce*)

void componentHidden(ComponentEvent *ce*)

ContainerEvent class:

- ComponentEvent is Generated when a component is added to or removed from container.
- There are 2 types of container events.
- The ContainerEvent class defines int constants that can be used to identify them:

Constants	Meaning
COMPONENT_ADDED	component has been added
COMPONENT_REMOVED.	component has been removed

Constructor :

ContainerEvent(Component *src*, int *type*, Component *comp*)

here,

src - reference to the container that generated this event. *type* - type of the event
comp - component that has been added to or removed from the container.

Methods:

- **getContainer() :**
- we can obtain a reference to the container that generated this event
Syntax : Container getContainer()
- **getChild() :**
- returns a reference to the component that was added to or removed from the container.
Syntax : Component getChild();

ContainerListener:

- This interface contains 2 methods. When a component is added to a container, **componentAdded()** is invoked. When a component is removed from a container, **componentRemoved()** is invoked. Their general forms are shown here:

Syntax :

void componentAdded(ContainerEvent *ce*)
void componentRemoved(ContainerEvent *ce*)

FocusEvent Class :

- A FocusEvent is generated when a component gains or loses input (keyboard) focus.
- These events are identified by the integer constants **FOCUS_GAINED** and **FOCUS_LOST**.
- **FocusEvent** is a subclass of **ComponentEvent**.

Constants	Meaning
FOCUS_GAINED	Component gains the focus
FOCUS_LOST	Component loses the focus

Constructors:

- **FocusEvent** has these constructors:

```
FocusEvent(Component src, int type)
FocusEvent(Component src, int type, boolean temporaryFlag)
FocusEvent(Component src, int type, boolean temporaryFlag,
            Component other)
```

here,

src - a reference to the component that generated this event.

type - type of the event

temporaryFlag - set to **true** if the focus event is temporary.
Otherwise, it is set to **false**.

other - The other component involved in the focus change, called the *opposite component*, is passed in *other*.

Therefore, if a **FOCUS_GAINED** event occurred, *other* will refer to the component that lost focus. Conversely, if a **FOCUS_LOST** event occurred, *other* will refer to the component that gains focus.

Methods:

- **getOppositeComponent() :**
 - we can determine the other component by calling this method.
 - Note : opposite component is returned

Syntax : Component getOppositeComponent()

- **isTemporary() :**
 - This method indicates if this focus change is temporary.

Syntax : boolean isTemporary()

Note : The method returns true if the change is temporary. Otherwise, it returns false.

FocusEvent Listener :

- This interface defines two methods.
- When a component obtains keyboard focus, **focusGained()** is invoked.
- When a component loses keyboard focus, **focusLost()** is called.

Syntax :

```
void focusGained(FocusEvent fe)
```

```
void focusLost(FocusEvent fe)
```

InputEvent Class:

- The abstract class **InputEvent** is a subclass of **ComponentEvent** and is the superclass for component input events. Its subclasses are **KeyEvent** and **MouseEvent**.

- **InputEvent** defines several integer constants that represent any modifiers(such as the control key being pressed) that might be associated with the event.
- The **InputEvent** class defined the following eight values to represent the modifiers:

ALT_DOWN_MASK	BUTTON2_DOWN_MASK	META_DOWN_MASK
ALT_GRAPH_DOWN_MASK	BUTTON3_DOWN_MASK	SHIFT_DOWN_MASK
BUTTON1_DOWN_MASK	CTRL_DOWN_MASK	

When writing new code, it is recommended that you use the new, extended modifiers rather than the original modifiers.

To test if a modifier was pressed at the time an event is generated, use the **isAltDown()**, **isAltGraphDown()**, **isControlDown()**, **isMetaDown()**, and **isShiftDown()** methods.

The forms of these methods are shown here:

boolean isAltDown()
boolean isAltGraphDown()
boolean isControlDown()
boolean isMetaDown()
boolean isShiftDown()

You can obtain a value that contains all of the original modifier flags by calling the **getModifiers()** method.

Syntax: int getModifiers()

You can obtain the extended modifiers by calling **getModifiersEx()**.

Syntax: int getModifiersEx()

ItemEvent Class :

- ItemEvent is generated when a check box or list item is clicked; also occurs when a choice selection is made or a checkable menu item is selected or deselected.
- There are 2 types of item events, which are identified by 2 integer constants.

DESELECTED	The user deselected an item.
SELECTED	The user selected an item.

NOTE: In addition, ItemEvent defines one integer constant, **ITEM_STATE_CHANGED**, that signifies a change of state

constructor:

ItemEvent(ItemSelectable *src*, int *type*, Object *entry*, int *state*)

here,

src - It is a reference to the component that generated this event. eg, it might be a list or choice element.

type – The type of the event.

entry – The specific item that generated the item event is passed in entry.

state - the current state of the item.

Methods:

getItem() :

- used to obtain a reference to the item that generated an event.

Syntax : Object getItem()

- **getItemSelectable() :**

-used to obtain a reference to the ItemSelectable object that generated an event.

Syntax : ItemSelectable getItemSelectable()

Eg: Lists and choices are user interface elements that implement the **ItemSelectable** interface.

- **getStateChange() :**

- method returns the state change (i.e., SELECTED or DESELECTED) for the event.

Syntax : int getStateChange

ItemEvent Source :

- Checkbox - Generates item events when the check box is selected or deselected.
- Choice - Generates item events when the choice is changed.
- List - Generates item events when an item is selected or deselected.
- Menu Item - Generates item events when a checkable menu item is selected or deselected.

ItemListener:

This interface defines itemStateChanged() method that is invoked when the state of an item changes.

Syntax : void itemStateChanged(ItemEvent *ie*)

KeyEvent Class :

- **AKeyEvent** is generated when keyboard input occurs. Generated when input is received from the keyboard
- There are three types of key events, which are identified by these integer constants: **KEY_PRESSED**, **KEY_RELEASED**, and **KEY_TYPED**.
- **KeyEvent** is a subclass of **InputEvent**.

Constants	Meaning
KEY_PRESSED	generated when any key is pressed
KEY_RELEASED	generated when any key is released
KEY_TYPED	occurs only when a character is generated.

Constructor:

`KeyEvent(Component src, int type, long when, int modifiers, int code)`

`KeyEvent(Component src, int type, long when, int modifiers, int code, char ch)`

Here, *src* - reference to the component that generated the event.

type - type of the event
when - system time at which the key was pressed.
modifiers - argument indicates which modifiers were pressed when this key event occurred.
code - virtual key code,
ch - character equivalent

Methods: The **KeyEvent** class defines several methods, but the most commonly used ones are

getKeyChar() : which returns the character that was entered. If no valid character is available, then **getKeyChar()** returns **CHAR_UNDEFINED**.

Syntax : `char getKeyChar()`

getKeyCode() : which returns the key code. When a **KEY_TYPED** event occurs, **getKeyCode()** returns **VK_UNDEFINED**.

Syntax : `int getKeyCode();`

KeyListener

- This interface defines 3 methods. The `keyPressed()` and `keyReleased()` methods are invoked when a key is pressed and released, respectively. The `keyTyped()` method is invoked when a character has been entered.
- Eg, if a user presses and releases the A key, three events are generated in sequence: key pressed, typed, and released. If a user presses and releases the HOME key, two key events are generated in sequence: key pressed and released.

Syntax :

```
void keyPressed(KeyEvent ke)
void keyReleased(KeyEvent ke)
void keyTyped(KeyEvent ke)
```

MouseEvent Class:

- MouseEvent is generated when the mouse is dragged, moved, clicked, pressed or released; also generated when the mouse enters or exits a component.
- There are 8 types of mouse events. **MouseEvent** class defines the following integer constants that can be used to identify them:

Constants	Meaning
MOUSE_CLICKED	It is occur when the user clicked the mouse.
MOUSE_DRAGGED	It is occur when the user dragged the mouse.
MOUSE_ENTERED	It is occur when the mouse entered a component.
MOUSE_EXITED	It is occur when the mouse exited from a component.
MOUSE_MOVED	It is occur when the mouse moved.
MOUSE_PRESSED	It is occur when the mouse was pressed.
MOUSE_RELEASED	It is occur when the mouse was released.

Constructor:

MouseEvent(Component <i>src</i> , int <i>type</i> , long <i>when</i> , int <i>modifiers</i> , int <i>x</i> , int <i>y</i> , int <i>clicks</i> , boolean <i>triggersPopup</i>)

Here,

src - reference to the component that generated the event.

type - type of the event

when - system time at which the mouse event occurred

modifiers - argument indicates which modifiers were pressed when a mouse event occurred.

x and *y* - coordinates of the mouse

clicks - click count

triggersPopup - flag indicates if this event causes a pop-up menu to appear on this platform.

Methods:

getX() and getY() : return the X and Y coordinates of the mouse when the event occurred.

Syntax :

```
int getX() ;  
int getY();
```

getPoint(): used to obtain the coordinates of the mouse.

Syntax : Point getPoint();

Note : It returns a Point object that contains the X, Y coordinates in its integer members : x and y.

translatePoint() : It changes the location of the event.

Syntax : void translatePoint(int x, int y);

Here, x & y are added to the coordinates of event.

getClickCount() : It obtains the number of mouse clicks for this event.

Syntax : int getClickCount();

isPopupTrigger() : This method tests if this event causes a pop-up menu to appear on this platform.

Syntax : boolean isPopupTrigger();

getButton() : It returns a value that represents the button that caused the event. The return value will be one of these constants defined by **MouseEvent**:

NOBUTTON	BUTTON1	BUTTON2	BUTTON3
----------	---------	---------	---------

The **NOBUTTON** value indicates that no button was pressed or released.

MouseListener:

- This interface defines 5 methods. If the mouse is pressed and released at the same point, mouseClicked() is invoked. When the mouse enters a component, the mouseEntered() method is called. When it leaves, mouseExited() is called. The mousePressed() and mouseReleased() methods are invoked when the mouse is pressed and released, respectively.

Syntax :

```
void mouseClicked(MouseEvent me)  
void mouseEntered(MouseEvent me)  
void mouseExited(MouseEvent me)  
void mousePressed(MouseEvent me)
```

```
void mouseReleased(MouseEvent me)
```

MouseMotionListener:

- This interface defines 2 methods.
- The `mouseDragged()` method is called multiple times as the mouse is dragged. The `mouseMoved()` method is called multiple times as the mouse is moved.

Syntax :

```
void mouseDragged(MouseEvent me)  
void mouseMoved(MouseEvent me)
```

WindowEvent class:

- WindowEvent is generated when a window is activated, closed, deactivated, deiconified, iconified, opened, or quit.
- There are 10 types of window events. The WindowEvent class defines integer constants that can be used to identify them.

WINDOW_ACTIVATED	The window was activated.
WINDOW_CLOSED	The window has been closed.
WINDOW_CLOSING	The user requested that the window be closed.
WINDOW_DEACTIVATED	The window was deactivated.
WINDOW_DEICONIFIED	The window was deiconified.
WINDOW_GAINED_FOCUS	The window gained input focus.
WINDOW_ICONIFIED	The window was iconified.
WINDOW_LOST_FOCUS	The window lost input focus.
WINDOW_OPENED	The window was opened.
WINDOW_STATE_CHANGED	The state of the window changed.

NOTE : WindowEvent is a subclass of ComponentEvent.

Constructor:

```
WindowEvent(Window src, int type)  
WindowEvent(Window src, int type, Window other)  
WindowEvent(Window src, int type, int fromState, int toState)  
WindowEvent(Window src, int type, Window other, int fromState, int toState)
```

here,

src - reference to the object that generated this event

type - type of the event

other - specifies the opposite window when a focus event occurs.

fromState - specifies the prior state of the window

toState - specifies the new state that the window will have when a window state change occurs.

Methods:

getWindow() : It returns Window object that generated event.

Syntax : Window getWindow();

getOppositeWindow() : It return the opposite window (when a focus event has occurred).

Syntax : Window getOppositeWindow();

getOldState() : return the previous window state

Syntax : int getOldState();

getNewState() : Return the current window state.

Syntax : int getNewState();

WindowListener:

- This interface defines 7 methods.

Syntax :

```
void windowActivated(WindowEvent we)
void windowClosed(WindowEvent we)
void windowClosing(WindowEvent we)
void windowDeactivated(WindowEvent we)
void windowDeiconified(WindowEvent we)
void windowIconified(WindowEvent we)
void windowOpened(WindowEvent we)
```

TextEvent class:

- Generated by text fields and text areas when characters are entered by a user or program.
- TextEvent defines the integer constant

Constant	Meaning
TEXT_VALUE_CHANGED	character entered by a user

Constructor :

TextEvent(Object *src*, int *type*)

Here,

src - reference to object that generated this event.

type - type of the event

TextListener :

- This interface defines the **textChanged()** method that is invoked when a change occurs in a text area or text field.

Syntax :

void textChanged(TextEvent *te*)

Event Listener Interfaces:

As explained, the delegation event model has two parts: sources and listeners. Listeners are created by implementing one or more of the interfaces defined by the **java.awt.event** package.

When an event occurs, the event source invokes the appropriate method defined by the listener and provides an event object as its argument. Table 22-3 lists commonly used listener interfaces and provides a brief description of the methods that they define.

Interface	Description
ActionListener	Defines one method to receive action events.
AdjustmentListener	Defines one method to receive adjustment events.
ComponentListener	Defines four methods to recognize when a component is hidden, moved, resized, or shown.
ContainerListener	Defines two methods to recognize when a component is added to or removed from a container.
FocusListener	Defines two methods to recognize when a component gains or loses keyboard focus.
ItemListener	Defines one method to recognize when the state of an item changes.
KeyListener	Defines three methods to recognize when a key is pressed, released, or typed.
MouseListener	Defines five methods to recognize when the mouse is clicked, enters a component, exits a component, is pressed, or is released.
MouseMotionListener	Defines two methods to recognize when the mouse is dragged or moved.
MouseWheelListener	Defines one method to recognize when the mouse wheel is moved.
TextListener	Defines one method to recognize when a text value changes.
WindowFocusListener	Defines two methods to recognize when a window gains or loses input focus.
WindowListener	Defines seven methods to recognize when a window is activated, closed, deactivated, deiconified, iconified, opened, or quit.

The ActionListener Interface:

- This interface defines the **actionPerformed()** method that is invoked when an action event occurs.

Syntax: **void actionPerformed(ActionEvent ae)**

The AdjustmentListener Interface:

➤ This interface defines the **adjustmentValueChanged()** method that is invoked when an adjustment event occurs.

Syntax: **void adjustmentValueChanged(AdjustmentEvent ae)**

The ComponentListener Interface

➤ This interface defines four methods that are invoked when a component is resized, moved, shown, or hidden.

Syntax:

void componentResized(ComponentEvent ce)
void componentMoved(ComponentEvent ce)
void componentShown(ComponentEvent ce)
void componentHidden(ComponentEvent ce)

The ContainerListener Interface:

This interface contains two methods.

- When a component is added to a container, **componentAdded()** is invoked.
- When a component is removed from a container, **componentRemoved()** is invoked.

Syntax: **void componentAdded(ContainerEvent ce)**
 void componentRemoved(ContainerEvent ce)

The FocusListener Interface:

This interface defines two methods.

- When a component obtains keyboard focus, **focusGained()** is invoked.
- When a component loses keyboard focus, **focusLost()** is called.

Syntax: **void focusGained(FocusEvent fe)**
 void focusLost(FocusEvent fe)

The ItemListener Interface:

- This interface defines the **itemStateChanged()** method that is invoked when the state of an item changes.

Syntax: void itemStateChanged(ItemEvent *ie*)

The KeyListener Interface:

This interface defines three methods.

- The **keyPressed()** and **keyReleased()** methods are invoked when a key is pressed and released, respectively.
- The **keyTyped()** method is invoked when a character has been entered.

For example, if a user presses and releases the A key, three events are generated in sequence: key pressed, typed, and released. If a user presses and releases the HOME key, two key events are generated in sequence: key pressed and released.

Syntax:

```
void keyPressed(KeyEvent ke)  
void keyReleased(KeyEvent ke)  
void keyTyped(KeyEvent ke)
```

The MouseListener Interface:

This interface defines five methods.

- If the mouse is pressed and released at the same point, **mouseClicked()** is invoked.
- When the mouse enters a component, the **mouseEntered()** method is called.
- When it leaves, **mouseExited()** is called.
- The **mousePressed()** and **mouseReleased()** methods are invoked when the mouse is pressed and released, respectively.

Syntax: void mouseClicked(MouseEvent *me*)
 void mouseEntered(MouseEvent *me*)
 void mouseExited(MouseEvent *me*)
 void mousePressed(MouseEvent *me*)
 void mouseReleased(MouseEvent *me*)

The MouseMotionListener Interface:

This interface defines two methods.

- The **mouseDragged()** method is called multiple times as the mouse is dragged.
- The **mouseMoved()** method is called multiple times as the mouse is moved.

Syntax: void mouseDragged(MouseEvent *me*)
 void mouseMoved(MouseEvent *me*)

The MouseWheelListener Interface:

This interface defines the **mouseWheelMoved()** method that is invoked when the mouse wheel is moved.

Syntax: void mouseWheelMoved(MouseWheelEvent *mwe*)

The TextListener Interface:

This interface defines the **textChanged()** method that is invoked when a change occurs in a text area or text field.

Syntax: void textChanged(TextEvent *te*)

The WindowFocusListener Interface:

This interface defines two methods:

- **windowGainedFocus()** and **windowLostFocus()**. These are called when a window gains or loses input focus.

Syntax: void windowGainedFocus(WindowEvent *we*)
 void windowLostFocus(WindowEvent *we*)

The WindowListener Interface:

This interface defines seven methods.

- The **windowActivated()** and **windowDeactivated()** methods are invoked when a window is activated or deactivated, respectively.
- If a window is iconified, the **windowIconified()** method is called.
- When a window is deiconified, the **windowDeiconified()** method is called.
- When a window is opened or closed, the **windowOpened()** or **windowClosed()** methods are called, respectively.
- The **windowClosing()** method is called when a window is being closed.

Syntax: void windowActivated(WindowEvent *we*)
 void windowClosed(WindowEvent *we*)
 void windowClosing(WindowEvent *we*)
 void windowDeactivated(WindowEvent *we*)
 void windowDeiconified(WindowEvent *we*)
 void windowIconified(WindowEvent *we*)

Handling Mouse Events:

- To handle mouse events, we must implement the `MouseListener` and `MouseMotionListener` interfaces.
- `MouseListener` interface defines 5 methods.

1. **`mouseClicked()`** :It is invoked when mouse is pressed and released.
2. **`mouseEntered()`**: it is invoked when mouse is entered in to component.
3. **`mouseExited()`**: it is invoked when mouse leaves the component.
4. **`mousePressed()`**: It is invoked when mouse is pressed.
5. **`mouseReleased()`**: It is invoked when mouse is released.

Syntax :

```
void mouseClicked(MouseEvent me)
void mouseEntered(MouseEvent me)
void mouseExited(MouseEvent me)
void mousePressed(MouseEvent me)
void mouseReleased(MouseEvent me)
```

MouseMotionListener

- This interface defines 2 methods.
- The `mouseDragged()` method is called multiple times as the mouse is dragged. The `mouseMoved()` method is called multiple times as the mouse is moved.

Syntax :

```
void mouseDragged(MouseEvent me)
void mouseMoved(MouseEvent me)
```

following program illustrates mouse handling

```
import java.applet.*;
import java.awt.event.*;
import java.awt.*;
/*<applet code=mousedemo width=300 height=300>
</applet>*/
public class mousedemo extends Applet implements MouseListener,
                                                    MouseMotionListener
{
    int x=0,y=0;
    String msg="";
    public void init()
    {
        addMouseListener(this);
        addMouseMotionListener(this);
    }
    public void mouseEntered(MouseEvent me)
    {
        x=0;
        y=10;
        msg="Mouse Entered";
```

```

        repaint();
    }
    public void mouseClicked(MouseEvent me)
    {
        x=0;
        y=10;
        msg="Mouse Clicked";
        repaint();
    }

    public void mouseExited(MouseEvent me)
    {
        x=0;
        y=10;
        msg="Mouse Exited";
        repaint();
    }

    public void mousePressed(MouseEvent me)
    {
        x=me.getX();
        y=me.getY();
        msg="Mouse Pressed";
        repaint();
    }
    public void mouseReleased(MouseEvent me)
    {
        x=me.getX();
        y=me.getY();
        msg="Mouse Released";
        repaint();
    }

    public void mouseDragged(MouseEvent me)
    {
        x=me.getX();
        y=me.getY();
        showStatus("Currently mouse dragged"+x+" "+y);
        repaint();
    }
    public void mouseMoved(MouseEvent me)
    {
        x=me.getX();
        y=me.getY();
        showStatus("Currently mouse is at"+x+" "+y);
        repaint();
    }
    public void paint(Graphics g)
    {

```

```

        g.drawString("Handling Mouse Events",30,20);
        g.drawString(msg,x,y);
    }
}

```

Handling Keyboard Events:

To handle keyboard events, we must implement the `KeyListener` interface .

When a key is pressed, a **KEY_PRESSED** event is generated. This results in a call to the **keyPressed()** event handler.

When the key is released, a **KEY_RELEASED** event is generated and the **keyReleased()** handler is executed.

If a character is generated by the keystroke, then a **KEY_TYPED** event is sent and the **keyTyped()** handler is invoked.

Thus, each time the user presses a key, at least two and often three events are generated

The following program demonstrates the handling of keyboard events.

```

import java.awt.*;
import java.awt.event.*;
import java.applet.*;

/*
<applet code=key width=500 height=500>
</applet>
*/

public class key extends Applet implements KeyListener
{
    String msg="";
    int x=10,y=20;
    public void init()
    {
        setBackground(Color.green);
        setForeground(Color.red);
        addKeyListener(this);
    }
    public void keyPressed(KeyEvent ke)
    {
        showStatus("KEY DOWN");
    }
    public void keyReleased(KeyEvent ke)
    {
        showStatus("KEY UP");
    }
    public void keyTyped(KeyEvent ke)

```

```

    {
        msg+=ke.getKeyChar();
        repaint();
    }
    public void paint(Graphics g)
    {
        g.drawString(msg,x,y);
    }
}

```

Adapter Classes:

It is a class, that can simplify the creation of event handlers in certain situations.

An adapter class provides all the methods of event listener interface with empty implementation.

Adapter classes are useful when we want to receive and process only some of the events that are handled by a particular event listener interface.

We can define a new class to act as an event listener by extending one of the adapter classes and implementing only those events in which we are interested.

Example: the MouseMotionAdapter class has 2 methods, mouseDragged() and mouseMoved().

Note : The signatures of these empty methods are exactly as defined in the MouseMotionListener interface.

- If we were interested in only mouse drag events, then we could simply extend MouseMotionAdapter and implement mouseDragged() only.
- The empty implementation of mouseMoved() would handle the mouse motion events for us.

Commonly used Listener interfaces implemented by Adapter Classes :

Adapter Class	Listener Interface
ComponentAdapter	ComponentListener
ContainerAdapter	ContainerListener
FocusAdapter	FocusListener
KeyAdapter	KeyListener
MouseAdapter	MouseListener
MouseMotionAdapter	MouseMotionListener
WindowAdapter	WindowListener

Example program:

```
import java.applet.*;
import java.awt.event.*;
import java.awt.*;

/*
<applet code=ademo width=300 height=300>
</applet>
*/

public class ademo extends Applet
{
    public void init()
    {
        myadopter m=new myadopter(this);
        addMouseListener(m);
    }
}

class myadopter extends MouseAdapter
{
    String msg;
    ademo ad;
    myadopter(ademo m)
    {
        ad=m;
    }
    public void mouseEntered(MouseEvent me)
    {
        Graphics g=ad.getGraphics();
        g.drawString("entered",50,50);
    }

    public void mouseExited(MouseEvent me)
    {
        Graphics g=ad.getGraphics();
        g.drawString("Exited",100,100);
    }
}
```

Unit-V

Swings

Swing library is an official Java GUI toolkit released by Sun Microsystems. Swing is probably the most advanced toolkit. It has a rich set of widgets. From basic widgets like Buttons, Labels, Scrollbars to advanced widgets like Trees and Tables. Swing is written in 100% java. Swing is a part of JFC, Java Foundation Classes. It is a collection of packages for creating full featured desktop applications. JFC consists of AWT, Swing, Accessibility. Swing was released in 1997 with JDK 1.2. It is a mature toolkit. The Java platform has Java2D library, which enables developers to create advanced 2D graphics and imaging.

There are basically two types of widget toolkits.

- Lightweight (consumes less resources)
- Heavyweight (consumes more resources)

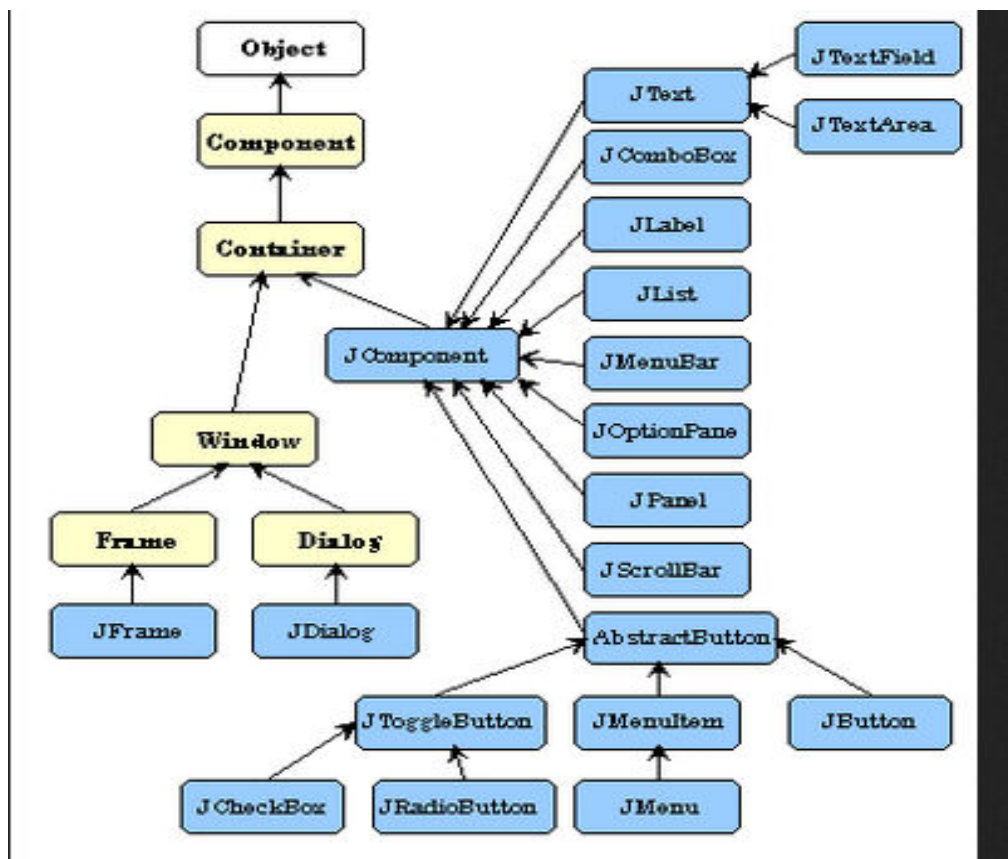
Swing is a lightweight toolkit. The main characteristics of the Swing toolkit

- platform independent
- customizable
- extensible
- configurable
- lightweight

some of the Swing packages are

- javax.swing
- javax.swing.border
- javax.swing.event
- javax.swing.table
- javax.swing.text

SWING HIERARCHY:



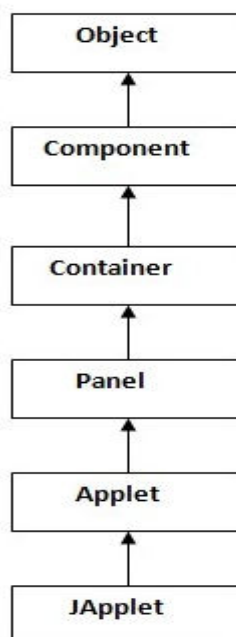
AWT Classes	Swing classes
Applet	JApplet
Component	JComponent
Frame	JFrame
Label	JLabel
Button	JButton
Checkbox	JCheckBox
CheckboxGroup	JRadioButton
TextField	JTextField

JApplet:

An applet is a small Internet-based program written in Java, a programming language for the Web, which can be downloaded by any computer.

- JApplet is similar to the Applet but completely written by using Java Programming language.

Hierarchy of Applet



- Applet class is the super class of JApplet class.
- It provides rich functionality which is not found in Applet
- For adding components to the Japplet window, add() method can't be used directly .

Procedure for adding components to Japplet window

- first get the contentPane object.
- JApplet class defined getContentPane() method that returns contentPane object.

Syntax: Container getContentPane()

- The add() method of Container can be used to add a component to a content pane
void add(comp)

Here, comp is the component to be added to the content pane.

```
Ex:  Container c=getContentPane();  
      JLabel l1=new JLabel("Example");  
      c.add(l1);
```

JFrame:

- Frame represents a window with a title bar and borders.
- Frame is a display area where components are added(displayed)
- In swings frame can be created by using JFrame class.

Creation of Frame by using swings:

```
Jframe jf = new Jframe();
```

frame is created without title.

```
Jframe jf=new Jframe("title");
```

frame is created with title

Methods:

Method Name	Description
setSize(int x, int y)	Sets the size of the JFrame in pixels
setLocation(int x, int y)	Sets the location of the JFrame on the screen in pixels
setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE)	Ensures that the application exits when the close button is clicked
setTitle(String title)	Sets the title of the JFrame. The title can also be set in the constructor.
getContentPane()	Retrieves a reference to the content pane, so other components can be added

Example program:

```
import javax.swing.*;
```

```

class frame
{
    public static void main(String args[])
    {
        JFrame jf=new JFrame("MYFRAME");
        jf.setSize(200,200);
        jf.setVisible(true);
    }
}

```

JComponent :

- Jcomponent class of javax.swing package is the subclass of the Component class of java.awt package.
- So whatever methods are available in Component class are also available to Jcomponent.
- All Swing components derive from the abstract javax.swing.JComponent class.

Method	Description
public int getHeight ()	Returns the current height of this component in pixel.
public int getX()	Returns the current x coordinate of the component's top-left corner.
public int getY ()	Returns the current y coordinate of the component's top-left corner.
public java.awt.Graphics getGraphics()	Returns this component's Graphics object you can draw on. This is useful if you want to change the appearance of a component.
public void setBackground (java.awt.Color bg)	Sets this component's background color.
public void setFont (java.awt.Font font)	Set the font used to print text on this component.
public void setForeground (java.awt.Color fg)	Set this component's foreground color.
public void setVisible (boolean visible)	Sets whether or not this component is visible.

Icons:

- In swing, icons are encapsulated by the ImageIcon class,

ImageIcon : Which is used to paints an icon from images.

Constructors of ImageIcon:

ImageIcon(String filename)

ImageIcon(URL url)

Here filename is the name of icon file(image file);

url is the complete location of file.

Ex: ImageIcon ii = new ImageIcon("Sunset.jpg");

Method	Description
int getIconHeight()	Returns the height of the icon in pixels.
int getIconWidth()	Returns the width of the icon in pixels.
void paintIcon(Component comp, Graphics g,int x, int y)	Paints the icon at position x, y onthe graphics context g.

Example:

```
import java.awt.*;
import javax.swing.*;
/*
<applet code=jlabel width=500 height=500>
</applet>
*/
public class jlabel extends JApplet
{
    public void init()
    {
        // Get content pane

        Container c= getContentPane();
        c.setLayout(new FlowLayout());

        JLabel jl = new JLabel("tree");
        ImageIcon i2 = new ImageIcon("temple.jpg");
```

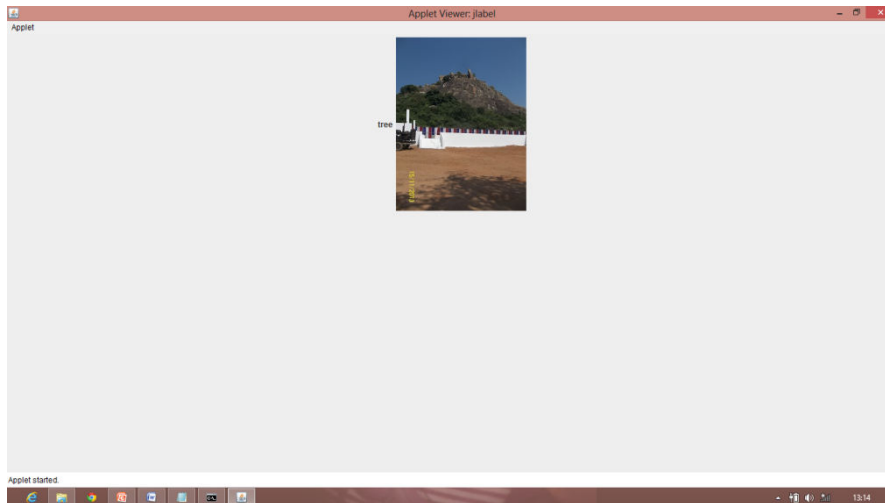
```

        // Create a label
        JLabel j2 = new JLabel(i2);

        // Add label to the content pane
        c.add(j1);
        c.add(j2);
    }
}

```

Output:



Jlabel:

- Swing labels are instances of the JLabel class
- It can display text or images as labels

Constructors of JLabel class:

JLabel(Icon i)

JLabel(String s)

JLabel(String s, Icon I, int align)

- first form creates a label with the specified image;
- Second form creates a label with the string specified in str.
- Third form creates a label with the specified text, images and alignment.

Align values are either LEFT, RIGHT, CENTER

Methods:

getIcon(): Used to get the icon of label

Syntax: Icon getIcon();

getText(): used to get the text of label

Syntax: String getText();

setIcon(Icon i): Used to set icon

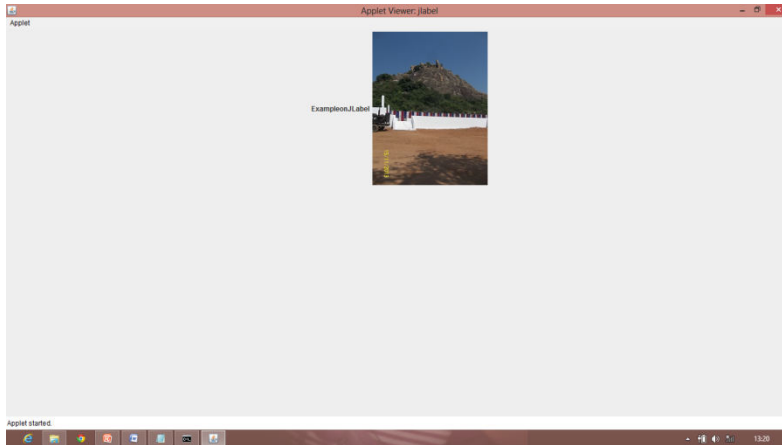
Syntax: void setIcon(Icon i);

void setText(String s): used to set text to the label

Syntax: void setText(String s)

Example Program:

```
import java.awt.*;
import javax.swing.*;
/*
<applet code=jlabel width=500 height=500>
</applet>
*/
public class jlabel extends JApplet
{
    public void init()
    {
        Container c= getContentPane();
        c.setLayout(new FlowLayout());
        JLabel jl = new JLabel("ExampleonJLabel");
        ImageIcon i2 = new ImageIcon("temple.jpg");
        JLabel j2 = new JLabel(i2);
        c.add(jl);
        c.add(j2);
    }
}
```



JTextField:

- JTextField is a lightweight component that allows the editing of a single line of text

Constructors :

JTextField()

JTextField(int cols)

JTextField(String s, int cols)

JTextField(String s)

- Here, s is the string to be presented, and cols is the number of columns in the text field.

Methods:

setEditable(): used to set edit ability to text field.

Syntax: Void setEditable(boolean b);

isEditable(): Used to determine whether textfield is editable or not.

Syntax: boolean isEditable():

setText(): used to set text to textfield.

Syntax: Void setText(String s)

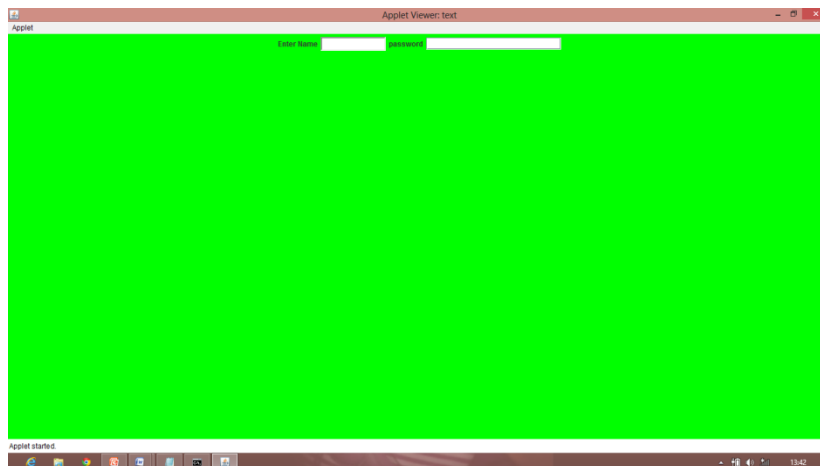
getText():used to get the text from text field

Syntax: String getText();

Example Program:

```
import java.awt.*;
import java.applet.*;
import javax.swing.*;
/*
<applet code="text" width=1000 height=1000>
</applet>
*/
public class text extends JApplet
{
    TextField t1;
    JPasswordField pass;
    public void init()
    {
        Container c=getContentPane();
        c.setLayout(new FlowLayout());
        JLabel l1=new JLabel("Enter Name");
        t1=new TextField(12);
        c.add(l1);
        c.add(t1);
        c.setBackground(Color.green);
        JLabel l2=new JLabel("password");
        pass=new JPasswordField(20);
        pass.setEchoChar('+');
        c.add(l2);
        c.add(pass);
    }
}
```

Output:



JButton:

- A button is a component that contains a label and that generates an event when it is pressed.
- The JButton is a push button.
- Push buttons are objects of type JButton.

Constructors :

JButton(Icon i)
JButton(String s)
JButton(String s, Icon i)

- In first form button is created with specified image
- In second form button is created with specified string s
- In third form button is created with specified string s and image

Methods:

setActionCommand(): used to set action command to button.

Syntax: setActionCommand(String s);

getActionCommand(): used to identify the pressed button

Syntax: String getActionCommand()

Handling Events:

1. Each time a button is pressed, an action event is generated.
2. This is sent to ActionListener that previously registered to receiving an action event notifications from that component..
3. ActionListenerinterface defines the actionPerformed() method, which is called when an event occurs.
4. An(ActionEvent) object is supplied as the argument to this method.

Example Program:

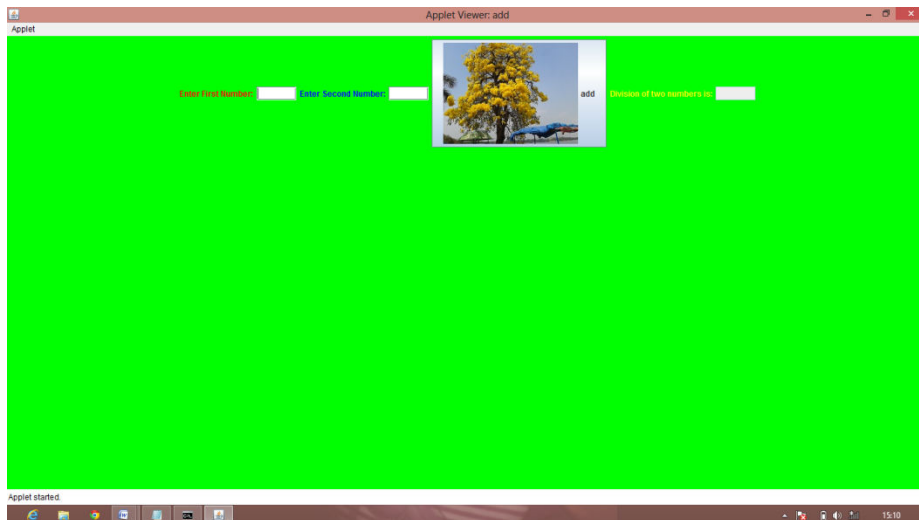
```
import java.awt.*;  
import java.awt.event.*;  
import javax.swing.*;  
  
/* <applet code="add" width=500 height=200> </applet> */  
  
public class add extends JApplet implements ActionListener
```

```

{
    JTextField t1,t2,t3;
    JLabel l1,l2,l3,h;

    public void init()
    {
        Container c=getContentPane();
        c.setLayout(new FlowLayout());
        c.setBackground(Color.green);
        l1=new JLabel("Enter First Number:");
        l1.setForeground(Color.red);
        t1=new JTextField(5);
        l2=new JLabel("Enter Second Number:");
        l2.setForeground(Color.blue);
        t2=new JTextField(5);
        t2.setForeground(Color.red);
        l3=new JLabel("Division of two numbers is:");
        l3.setForeground(Color.yellow);
        t3=new JTextField(5);
        t3.setEditable(false);
        ImageIcon ii = new ImageIcon("tree.jpg");
        JButton b=new JButton("add",ii);
        h=new JLabel(" ");
        h.setVisible(false);
        c.add(l1);
        c.add(t1);
        c.add(l2);
        c.add(t2);
        c.add(b);
        c.add(l3);
        c.add(t3);
        c.add(h);
        b.addActionListener(this);
    }
    public void actionPerformed(ActionEvent ae)
    {
        String n1,n2;
        int a,b,c;
        n1=t1.getText();
        a=Integer.parseInt(n1);
        n2=t2.getText();
        b=Integer.parseInt(n2);
        int sum=a+b;
        t3.setText(Integer.toString(sum));
    }
}

```



JCheckBox:

- Checkbox is a collection of items user has to select multiple items from a group.
- It consists of a small box that can either contain a check mark or not.
- The JCheckBox is a widget that lets you select more than one attribute at a time on the screen by 'checking' i.e. ticking selections in a list.
- This is useful when multiple choices are involved,

Constructors :

JCheckBox(Icon i)

JCheckBox(Icon i, boolean state)

JCheckBox(String s)

JCheckBox(String s, boolean state)

JCheckBox(String s, Icon i)

JCheckBox(String s, Icon i, boolean state)

- Here I is icon for the button
- Text is specified by s.
- If state is true, the checkbox is initially checked otherwise unchecked.

Methods:

getItem() : used to get the JCheckBox object that generated the event.

getText() : method gets the text for that check box and uses it to set the text inside the text field

isSelected():used to determine if the box is selected or not.

Handling Check Boxes :

1. Each time a check box is selected or deselected, an item event is generated.
2. This is sent to any listeners that previously registered an interest in receiving item event notifications from that component.
3. Each listener implements the ItemListener interface.
4. That interface defines the itemStateChanged() method.
5. An ItemEvent object is supplied as the argument to this method. It contains information about the event (for example, whether it was a selection or deselection).

Example Program:

```
import java.awt.*;
import java.applet.*;
import javax.swing.*;

/*
<applet code="check" width=1000 height=1000>
</applet>
*/

public class check extends JApplet
{
    JCheckBox c1,c2,c3,c4;

    public void init()
    {

        Container c=getContentPane();
        c.setLayout(new GridLayout());
        c1=new JCheckBox("JAVA");
        c2=new JCheckBox("DBMS");
        c3=new JCheckBox("OS");
        ImageIcon i=new ImageIcon("tree.jpg");
```

```

        c.add(c1);
        c4=new JCheckBox(i);
        c.add(c4);
        c.add(c2);
        c.add(c3);

    }

}

```

RadioButtons

- Radiobuttons are a group of mutually exclusive buttons, in which only one button can be selected at any time.
- JRadioButton class is used to create RadioButtons.
- It is used with a ButtonGroup object to create a group of buttons in in which only one button can be selected at any time.
- So first a ButtonGroup object is created and its add() method is used to add RadioButton objects to this group.

Constructors:

JRadioButton(Icon ic)

JRadioButton(Icon ic,boolean state)

JRadioButton(String str)

JRadioButton(String str,boolean state)

JRadioButton(String str,Icon ic)

JRadioButton(String str,Icon ic,boolean state)

JRadioButton(String str,boolean state)

Here icon ic is image and str is string. if state is true RadioButton is checked.otherwise unchecked.

ExapleProgram:

```
import java.awt.*;
```

```
import java.awt.event.*;
```

```

import java.applet.*;
import javax.swing.*;

/*
<applet code="radio" width=1000 height=1000>
</applet>
*/

public class radio extends JApplet implements ActionListener
{
    String msg="";
    JRadioButton rb1,rb2,rb3;
    JLabel h;

    public void init()
    {

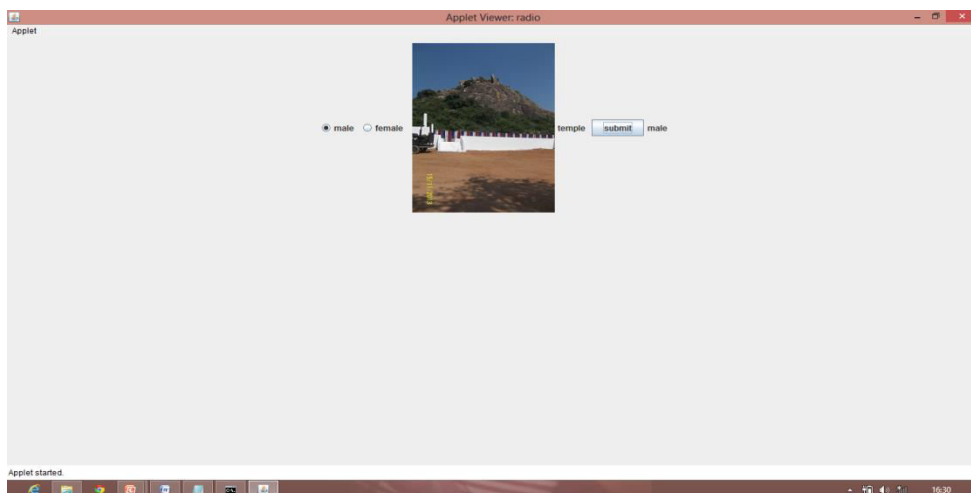
        Container c=getContentPane();
        c.setLayout(new FlowLayout());
        rb1=new JRadioButton("male");
        rb2=new JRadioButton("female");
        ImageIcon i=new ImageIcon("temple.jpg");
        rb3=new JRadioButton("temple",i);
        ButtonGroup bg=new ButtonGroup();
        bg.add(rb1);
        bg.add(rb2);
        bg.add(rb3);
        JButton jb=new JButton("submit");
        h=new JLabel("");
        h.setVisible(false);
        c.add(rb1);
        c.add(rb2);
        c.add(rb3);
        c.add(jb);
        c.add(h);
        jb.addActionListener(this);
    }
}

```

```

    }
    public void actionPerformed(ActionEvent ae)
    {
        if(rb1.isSelected())
            msg+=rb1.getText();
        if(rb2.isSelected())
            msg+=rb2.getText();
        if(rb3.isSelected())
            msg+=rb3.getText();
        h.setText(msg);
        h.setVisible(true);
    }
}

```



Comboboxes(JComboBox):

- JComboBox class is used to create a drop-down list or pop-up list.
- The user has to select a value from drop-down list.

By default only one item is visible in the combobox.

Constructors:

JCoboBox()

JComboBox(Object[] items)

JComboBox(Vector items)

- First form creates a JComboBox with no elements in it.
- Second form creates a JComboBox that contains the elements in the specified array.
- Third form creates a JComboBox that contains the elements in the specified vector.

Example Program:

```
import java.awt.*;
import java.applet.*;
import javax.swing.*;
import javax.swing.event.*;
import java.awt.event.*;

/*
<applet code="combo" width=1000 height=1000>
</applet>
*/

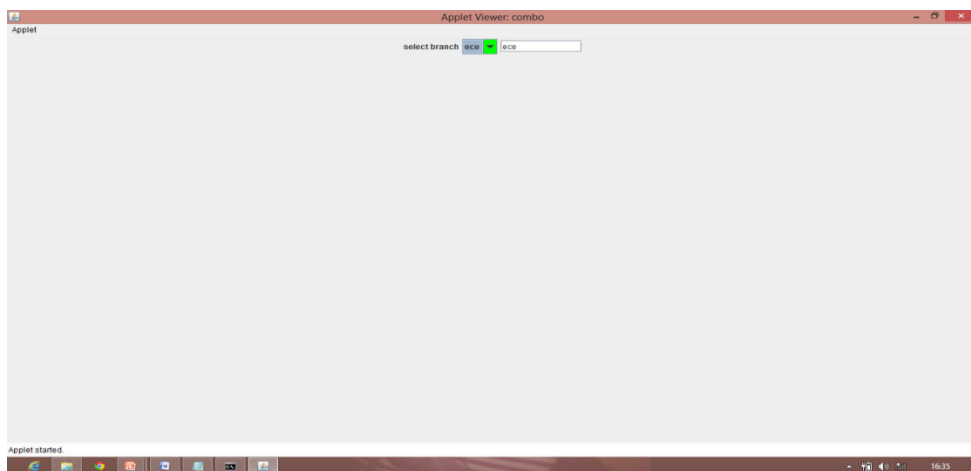
public class combo extends JApplet implements ItemListener
{
    JTextField txt;
    JComboBox combo;
    public void init()
    {
        Container c=getContentPane();
        c.setLayout(new FlowLayout());
        String branch[] = {"cse","ece","it"};
        combo = new JComboBox(branch);
        JLabel l1=new JLabel("select branch");
        combo.setBackground(Color.green);
        combo.setForeground(Color.red);
        txt = new JTextField(10);
        c.add(l1);
        c.add(combo);
        c.add(txt);
        combo.addItemListener(this);
    }
    public void itemStateChanged(ItemEvent ie)
```



```

{
    String str = (String)combo.getSelectedItem();
    txt.setText(str);
}
}

```



JTable:

- A table is a component that displays rows and columns of data.
- JTable is used to display two-dimensional tables of cells.

Constructors:

JTable(int nrows,int ncols)

JTable(Object rowdata[][],Object colnames[])

- First form creates table with nrows and ncols of empty cells.
- Second form creates table to display the values in the two-dimensional array rowdata ,with column names colnames.
- Here, rowdata is a two dimensional array of the information to be presented, and colnames is a one dimensional array with column headings.

Here are the steps for using a table in an applet:

1. Create a JTable object
2. Create a JScrollPane object.
3. Add the table to the scroll pane.
4. Add the scroll pane to the content pane of the applet.

```
import java.awt.*;
```

```

import java.applet.*;
import javax.swing.*;

/*
<applet code="table" width=1000 height=1000>
</applet>
*/

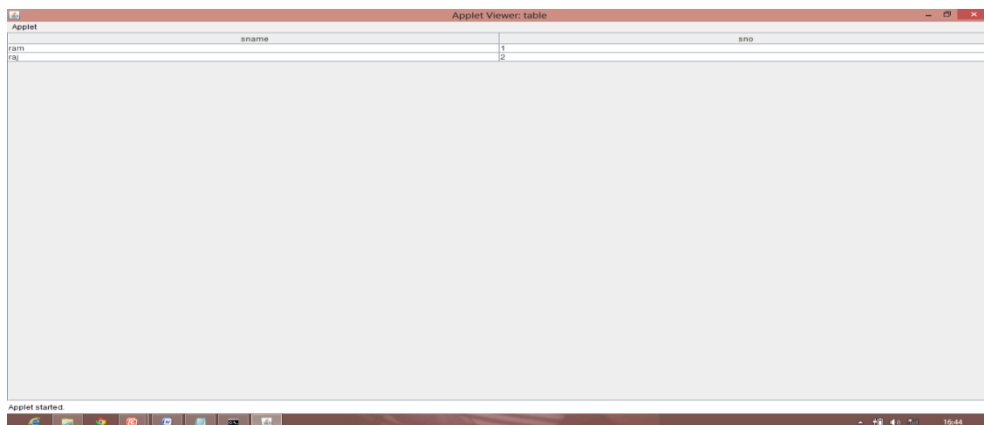
public class table extends JApplet
{
    JTextField txt;
    JComboBox combo;

    public void init()
    {
        Container c=getContentPane();
        c.setLayout(new GridLayout());

        final String[] colheads={"sname","sno"};
        final Object data[][]={ {"ram","1"}, {"raj","2"} };
        JTable table=new JTable(data, colheads);
        int v=ScrollPaneConstants.VERTICAL_SCROLLBAR_AS_NEEDED;
        int h=ScrollPaneConstants.HORIZONTAL_SCROLLBAR_AS_NEEDED;

        JScrollPane jsp=new JScrollPane(table,v,h);
        c.add(jsp);
    }
}

```



TabbedPanels:

- A tabbed pane is a component that appears as a group of folders in a file cabinet.
- Each folder has a title. When a user selects a folder, its contents become visible. Only one of the folders may be selected at a time.
- Tabbed panes are encapsulated by the **JTabbedPane** class, which extends **JComponent**.
- JTabbedPane is a container with the labelled tabs.
- JTabbedPane allows the user to switch between a group componets by clicking on a tab with a given title or icon

addTab():It is used to add tabs to TabbedPane.

Constructors:

JTabbedPane()

JtabbedPane(int tabpalcement)

- First form creates an empty TabbedPane with a default tab placement.i.e Jtabbed.Top
- Secondform creates a TabbedPane with specified tab placement.
- tabpalcement ispecifies one of following
- JTabbedPane.Top, JTabbedPane.BOTTOM, JTabbedPane.LEFT etc

The general procedure to use a tabbed pane in the an applet is outlined here:

1. Create a JTabbedPane object.
2. Call addTab() to add a tab to the pane.
3. Repeat step2 for each tab.
4. Add the tabbed pane to the content pane of the applet

Example program:

```
import java.awt.*;
import javax.swing.*;
/*
<applet code="tabbed" width=500 height=500>
</applet>
*/
class colors extends JPanel
{
```

```

public colors()
{
    JButton b1=new JButton("Red");
    JButton b2=new JButton("Green");
    add(b1);
    add(b2);
}
}

class movies extends JPanel
{
    public movies()
    {
        JRadioButton rb1=new JRadioButton("Siva");
        JRadioButton rb2=new JRadioButton("Chanti");
        JRadioButton rb3=new JRadioButton("Sivaji");
        add(rb1);
        add(rb2);
        add(rb3);
    }
}

class crickt extends JPanel
{
    public crickt()
    {
        JCheckBox cb1=new JCheckBox("Sachin");
        JCheckBox cb2=new JCheckBox("Ganguly");
        JCheckBox cb3=new JCheckBox("Kohli");
        add(cb1);
        add(cb2);
        add(cb3);
    }
}

public class tabbed extends JApplet
{

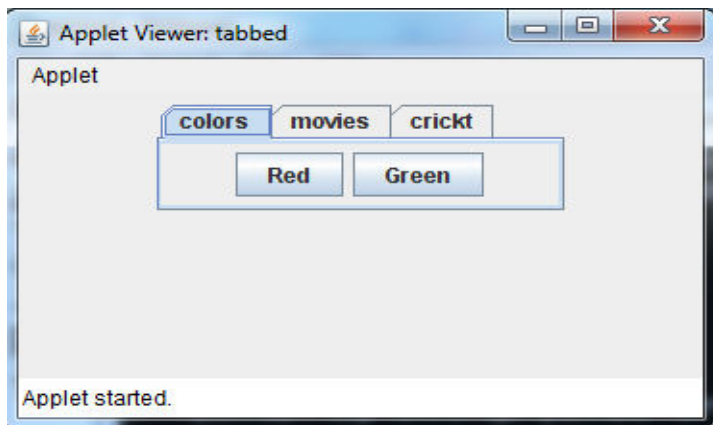
```

```

public void init()
{
    Container c=getContentPane();
    c.setLayout(new FlowLayout());
    JTabbedPane jtp=new JTabbedPane(JTabbedPane.TOP);
    colors co=new colors();
    crickt ck=new crickt();
    movies m=new movies();
    jtp.addTab("colors",co);
    jtp.addTab("movies",m);
    jtp.addTab("cricht",ck);

    c.add(jtp);
}
}

```



JScrollPane

- A JScrollPane is a container class that can hold one component.
- A scroll pane is a component that presents a rectangular area in which a component may be viewed.
- if necessary Horizontal and / or vertical scroll bars may be provided
- Scroll panes are implemented in Swing by the **JScrollPane** class, which extends **JComponent**.

JScrollPane class constructors :

`JScrollPane(Component comp)`

`JScrollPane(Component comp, int vsb, int hsb)`

- Here comp is the component to be added to the scroll pane. vsb and hsb are int constants that defined when vertical and horizontal scroll bars for this scroll pane.

Constant

Description

`HORIZONTAL_SCROLLBAR_ALWAYS`

always provide horizontal
scroll bar

`HORIZONTAL_SCROLLBAR_AS_NEEDED`

provide horizontal scroll bar,
if needed

`VERTICAL_SCROLLBAR_ALWAYS`

always provide vertical
scrollbar

`VERTICAL_SCROLLBAR_AS_NEEDED`

provide vertical scroll bar,
if needed

Example program:

```
import java.awt.*;
import javax.swing.*;

/*
<applet code="jspane" width=200 height=200>
</applet>
*/

class mypanel extends JPanel
{
    public mypanel()
    {
        setLayout(new GridLayout(20,10));
        for(int i=1;i<=20;i++)
            for(int j=1;j<=10;j++)
                add(new JLabel(""+(i*j)));
    }
}

public class jspane extends JApplet
{
    public void init()
```

```

        {
            Container c=getContentPane();
            c.setLayout(new BorderLayout());
int v=ScrollPaneConstants.VERTICAL_SCROLLBAR_ALWAYS;
int h=ScrollPaneConstants.HORIZONTAL_SCROLLBAR_AS_NEEDED;
mypanel my=new mypanel();
            JScrollPane jsp=new JScrollPane(my,v,h);
            c.add(jsp);
        }
    }

```

JTree:

- A tree is a component that presents a hierarchical view of data.
- JTree class is useful to create a tree structure where a set of nodes can be displayed in a hierarchical manner.

Some of its constructors shown bellow

```

JTree(Hashtable ht)
JTree(Object obj[])
JTee(TreeNode tn)
Jtree(Vector v)

```

- The first form creates a tree in which each element of the hash table ht is a child node.
- Each element of the array obj is a child node in the second form.
- the tree node tn is the root of the tree in the third form.
- The last form uses the elements of vector v as child nodes.

Ex:

```
JTree tree=new JTree(root);
```

- Here root represents root node of the tree where other nodes will span.

Creation of root node and other nodes:

DefaultMutableTreeNode():It is used to create nodes.

Syntax:

```
DefaultMutableTreeNode(Object obj)
```

Here, obj is the node to be enclosed in this tree node.

add():It is used to add nodes to root node.

```
void add(MutableTreeNode child)
```

Here, child is a mutable tree node that is to be added as a child to the current node.

Procedure:

1. Create a root node by using DefaultMutableTreeNode() class.
2. Create child nodes by using DefaultMutableTreeNode() class.
3. Add child nodes to root node.
4. And then create a tree with root node by using JTree class
5. Add tree to the contentpane.

Example program:

```
import java.awt.*;
import javax.swing.*;
import javax.swing.tree.*;

/*
    <applet code=tree width=500 height=500>
    </applet>
*/
public class tree extends JApplet
{
    JTree tree;
    DefaultMutableTreeNode root,dir1,dir2,f1,f2,f3,f4;
    public void init()
    {
        Container c= getContentPane();
        c.setLayout(new FlowLayout());
        root=new DefaultMutableTreeNode("");
        dir1=new DefaultMutableTreeNode("ECE");
        dir2=new DefaultMutableTreeNode("CSE");

        f1=new DefaultMutableTreeNode("EDC");
        f2=new DefaultMutableTreeNode("DE");
        f3=new DefaultMutableTreeNode("JAVA");
```



```
f4=new DefaultMutableTreeNode("DBMS");  
root.add(dir1);  
root.add(dir2);
```

```
dir1.add(f1);  
dir1.add(f2);  
dir2.add(f3);  
dir2.add(f4);
```

```
c.setBackground(Color.green);  
JTree tree=new JTree(root);  
c.add(tree);
```

```
}  
}
```

